A project report on

DEEP LEARNING AND IOT-BASED REAL-TIME VARIABLE-RATE CHEMICAL SPRAYING SYSTEM FOR DISEASE CONTROL IN PADDY CROP

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

SOURISH DAS (23BCE1179)
ADITYA SINGH (23BCE1293)



School of Computer Science and Engineering
NOVEMBER, 2026

DEEP LEARNING AND IOT-BASED REAL-TIME VARIABLE-RATE CHEMICAL SPRAYING SYSTEM FOR DISEASE CONTROL IN PADDY CROP

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

SOURISH DAS (23BCE1179)
ADITYA SINGH (23BCE1293)



School of Computer Science and Engineering
NOVEMBER, 2026

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "Deep Learning and IoT-Based Real-Time Variable-Rate Chemical Spraying System for Disease Control in Paddy Crop" submitted by me (Sourish Das, 23BCE1179), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai, is a record of bonafide work carried out by me under the supervision of Dr. Vergin Raja Sarobin M..

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: Signature of the Candidate

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "Deep Learning and IoT-Based Real-Time Variable-Rate Chemical Spraying System for Disease Control in Paddy Crop" is prepared and submitted by Sourish Das (23BCE1179) to Vellore Institute of Technology, Chennai, in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering, and is a bonafide record of work carried out under my guidance. The project fulfils the requirements as per the regulations of this University, and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma, and the same is certified.

Signature of the Guide:

Name: Dr. Vergin Raja Sarobin M.

Date:

Signature of the Examiner 1 Signature of the Examiner 2

Name: Name:

Date: Date:

Approved by the Head of the Department,
B.Tech. Computer Science and Engineering

Name: [[ENTER HOD NAME]]

Date:

(Seal of the School of Computer Science and Engineering)

ABSTRACT

Farmers usually spray the same amount of chemical on every plant in a field, no matter how healthy or how diseased that plant is. This wastes chemical, raises cost and pollutes the soil and nearby water. We built a variable rate spraying system for paddy that decides how much chemical to spray for each plant based on what disease it has and how much of the leaf is affected. A camera image of a leaf is classified into one of six classes, a healthy leaf and five diseases, using a trained convolutional neural network. For a diseased leaf, a separate segmentation model finds the diseased pixels and reports the diseased area as a percentage of the leaf area. This severity percentage is mapped to a category and a base dosing coefficient, which is then scaled by a factor that depends on the diagnosed disease, since some diseases need a relatively higher dose at the same severity level. The resulting coefficient sets the chemical volume per plant and the time the spray valve should stay open. A compact hardware design built around an ESP32-CAM and an Arduino Nano is specified to carry out this decision in the field, with a Blynk dashboard for remote monitoring and a web demonstration interface for showing the full decision process on one uploaded photograph. The classifier reached 91.5 percent accuracy on an initial three class test set and 83.80 to 87.08 percent on the harder six class merged dataset. A full software simulation on 49 real test images showed a 40 to 43 percent reduction in chemical use against a constant full rate baseline, ahead of the 33.88 percent minimum saving reported by the base paper this project builds on. The physical hardware prototype's bench, integration and field test results depend on evidence the team supplies separately and are reported in this document exactly as available at the time of writing.

ACKNOWLEDGEMENT

It is my pleasure to express my deep gratitude to Dr. Vergin Raja Sarobin M., [[ENTER GUIDE DESIGNATION, e.g. Associate Professor]], School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for the constant guidance, the patience and the steady encouragement given throughout this project. Working under this guidance gave me the chance to learn from someone with real expertise in [[ENTER GUIDE'S AREA OF EXPERTISE]], and that experience went well beyond the academic requirements of this course.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and for inspiring all of us during the course of this programme.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for sharing their time and their knowledge and for helping us at every stage of this project.

I sincerely thank [[ENTER HOD NAME]], Head of the Department, B.Tech. Computer Science and Engineering, and the Project Coordinators for their support and encouragement in taking up and completing this project.

My sincere thanks also go to all the faculty and staff at Vellore Institute of Technology, Chennai, who helped me learn what this project needed. I thank my parents for their support, and my friends who encouraged me to take up and finish this work. This report was written with the help of AI based writing and drafting tools, used under my own direction and checked by me; the system design, the code and the results described in it are the team's own work.

Place: Chennai

Date: Sourish Das

CONTENTS

CHAPTER 1 1

INTRODUCTION 1

1.1 BACKGROUND 1

1.2 THE PROBLEM WITH UNIFORM SPRAYING 1

1.3 CHEMICAL OVERUSE AND ITS WIDER EFFECT 2

1.4 MOTIVATION 2

1.5 PROBLEM STATEMENT 3

1.6 OBJECTIVES 3

1.7 SCOPE OF THE PROJECT 4

CHAPTER 2 5

LITERATURE SURVEY AND PROJECT GOALS 5

2.1 OVERVIEW 5

2.2 VARIABLE RATE SPRAYING SYSTEMS 5

2.3 WEED DETECTION AND SITE SPECIFIC SPRAYING 7

2.4 DEEP LEARNING FOR PLANT AND RICE DISEASE DETECTION 7

2.5 RESEARCH GAP 9

2.6 PROBLEM STATEMENT (RESTATED FOR DESIGN) 9

2.7 OBJECTIVES AND NOVELTY OVER THE BASE PAPER 10

CHAPTER 3 11

TECHNICAL SPECIFICATION 11

3.1 OVERVIEW 11

3.2 FUNCTIONAL REQUIREMENTS 11

3.3 NON FUNCTIONAL REQUIREMENTS 12

3.4 FEASIBILITY STUDY 13

3.4.1 Technical feasibility 13

3.4.2 Economic feasibility 13

3.4.3 Social feasibility....13
3.5 HARDWARE SPECIFICATION....14
3.6 SOFTWARE SPECIFICATION....14
3.7 ASSUMPTIONS AND CONSTRAINTS....15
CHAPTER 4....16
SYSTEM DESIGN AND METHODOLOGY....16
4.1 SYSTEM ARCHITECTURE....16
4.2 ONE SPRAY CYCLE, STEP BY STEP....18
4.3 DATASETS....19
4.4 CLASSIFICATION MODEL....22
4.5 SEVERITY ESTIMATION....23
4.6 DOSE AND SPRAY TIME EQUATIONS....24
4.7 DESIGN DIAGRAMS....25
4.8 CONSTRAINTS AND DESIGN TRADE OFFS....30
CHAPTER 5....32
HARDWARE IMPLEMENTATION....32
5.1 OVERVIEW....32
5.2 BILL OF MATERIALS....32
5.3 CIRCUIT AND WIRING....33
5.4 POWER SYSTEM....36
5.5 LIQUID PATH....38
5.6 FIRMWARE BEHAVIOUR....38
5.6.1 Real problems found during firmware bring up....39
5.7 CLOUD API AND BLYNK DASHBOARD....42
5.8 ASSEMBLY AND CALIBRATION PROCEDURE....44
5.8.1 Measured flow rate and calibration result....45
5.8.2 Bench and integration test results....45

5.8.3 Real problems faced and fixes 48
5.9 KNOWN HARDWARE RISKS 48
5.10 CROSS CHECK WITH THE HARDWARE PLAN 49
5.11 PROCUREMENT STATUS 50
CHAPTER 6 52
SOFTWARE IMPLEMENTATION 52
6.1 OVERVIEW 52
6.2 TRAINING PIPELINE 52
6.2.1 Classifier training 52
6.2.2 Segmentation model training 52
6.2.3 Saved model artefacts 53
6.3 SEVERITY MODULE 53
6.4 INFERENCE API 54
6.5 WEB DEMONSTRATION INTERFACE 54
6.6 MODULE DESCRIPTION 59
CHAPTER 7 61
SCHEDULE, TASKS AND MILESTONES 61
7.1 OVERVIEW 61
7.2 TASK TABLE 61
7.3 MEMBER RESPONSIBILITIES 62
7.4 MILESTONES ACHIEVED 62
CHAPTER 8 64
TESTING 64
8.1 TESTING STRATEGY 64
8.2 UNIT TESTS 64
8.3 API INTEGRATION TESTS 66
8.4 SYSTEM AND FIELD TESTS (HARDWARE) 66

8.5 TEST CASE SUMMARY 67
CHAPTER 9 69
RESULTS AND DISCUSSION 69
9.1 OVERVIEW 69
9.2 CLASSIFIER PERFORMANCE 69
9.2.1 Three class model (initial UCI dataset) 69
9.2.2 Six class merged model 70
9.2.3 Discussion of classifier results 73
9.3 SEGMENTATION QUALITY 74
9.4 PIPELINE SIMULATION ON REAL TEST IMAGES 74
9.5 API RESPONSE TIME 76
9.6 DOSE AND SPRAY TIME WORKED EXAMPLES 76
9.7 COMPARISON WITH THE BASE PAPER AND RELATED WORK 77
9.8 LIMITATIONS 79
CHAPTER 10 81
CONCLUSION AND FUTURE WORK 81
10.1 SUMMARY OF WHAT WAS BUILT 81
10.2 WHAT WORKED WELL 81
10.3 WHAT DID NOT FULLY SUCCEED 82
10.4 COMPARISON WITH THE PLANNED TARGETS 82
10.5 OWN CONTRIBUTION VERSUS EXISTING WORK 83
10.6 INSIGHTS GAINED 83
10.7 FUTURE WORK 84
REFERENCES 85
APPENDICES 86
Appendix 1: Severity and Dosing Logic (model/severity.py) 87
Appendix 2: Single Image Command Line Prediction (model/predict.py) 94

Appendix 3: Flask Inference API (app/server.py) 96

Appendix 4: ESP32-CAM Firmware
(hardware_evidence/firmware/esp32cam_sprayer/esp32cam_sprayer.ino) 100

Appendix 5: Arduino Nano Firmware
(hardware_evidence/firmware/valve_controller/valve_controller.ino) 104

LIST OF TABLES

Table 2.1: Comparison of the base paper and our proposed system. 10

Table 3.1: Functional requirements of the system. 11

Table 3.2: Non functional requirements of the system. 12

Table 3.3: Hardware specification summary (full bill of materials in Chapter 5). 14

Table 3.4: Software specification summary. 14

Table 4.1: Severity category thresholds and base coefficients, as implemented. 24

Table 4.2: Per disease scaling factor applied to the base coefficient. 24

Table 5.1: Bill of materials for the hardware prototype. 32

Table 5.2: ESP32-CAM pin usage, as implemented and flashed in esp32cam_sprayer.ino (Appendix 4). 34

Table 5.3: Arduino Nano pin usage. 35

Table 5.4: Approximate current draw per rail. 37

Table 5.5: Hardware test status, updated with the real results above. 47

Table 5.6: Known hardware risks and mitigations, including two risks confirmed during real bring up (section 5.6.1, section 5.8.3). 48

Table 6.1: Software module description. 59

Table 7.1: Task table with owners and deliverables. 61

Table 7.2: Primary responsibilities by team member. 62

Table 8.1: Unit test case groups and results. 64

Table 8.2: API integration test cases and results. 66

Table 8.3: Overall test case summary. 67

Table 9.1: Three class model, held out test set, 164 images. 69

Table 9.2: Six class merged model, per class results, result set 2 (815 images). 72

Table 9.3: Segmentation model validation results against the PlantSeg paper's own baselines. 74

Table 9.4: Pipeline simulation summary, 49 real test images. 75

Table 9.5: Pipeline simulation, per class breakdown. 75

Table 9.6: Worked dose and spray time examples, calculated from design values. 77

Table 9.7: Chemical saving comparison with the base paper and related variable rate sprayers. 77

Table 9.8: Classifier accuracy comparison with related work. 78

LIST OF FIGURES

Figure 4.1: System block diagram showing the five stages of one spray cycle.....16

Figure 4.2: Flowchart of one complete spray decision cycle, from trigger to valve actuation. 17

Figure 4.3: Layered view of the system: sensing, processing, communication and actuation. 18

Figure 4.4: Class distribution of the merged six class training set. 20

Figure 4.5: Sample leaf images from the Capstone test set, one row per class. 21

Figure 4.6: EfficientNetV2-B0 classifier architecture, input to six class softmax output. 22

Figure 4.7: Severity pipeline: leaf isolation, lesion segmentation, pixel counting, severity percentage.....23

Figure 4.8: Data flow diagram, level 0. 26

Figure 4.9: Data flow diagram, level 1, showing the main internal processes. 27

Figure 4.10: Use case diagram for the farmer or reviewer actor and the ESP32-CAM device actor. 28

Figure 4.11: Sequence diagram for one spray cycle. 29

Figure 4.12: Module and class level diagram of the software. 30

Figure 5.1: Complete circuit and wiring diagram of the prototype. 34

Figure 5.2: Power distribution diagram: battery, buck converter, 5V and 12V rails...37

Figure 5.3: Liquid path diagram: tank, pump, solenoid valve, nozzle.....38

Figure 5.4: PaddyGuard Blynk dashboard, showing the Severity gauge, Category, Alert, the chemical usage history chart and the Remote Trigger button, populated by the pipeline simulation replay. 43

Figure 6.1: PaddyGuard web demo, home or upload view.....56

Figure 6.2: PaddyGuard analysis result for a Brown Spot leaf image, showing the diagnosis, severity calculation and spray timing cards. 57

Figure 6.3: PaddyGuard analysis result for a Neck Blast leaf image. 58

Figure 6.4: PaddyGuard History tab, showing past analyses from the same session. .59

Figure 7.1: Project Gantt chart across the fourteen week plan. 61

Figure 9.1: Confusion matrix, six class merged model, result set 1 (859 images, 87.08 percent accuracy). 71

Figure 9.2: Confusion matrix, six class merged model, result set 2 (815 images, 83.80 percent accuracy). 72

Figure 9.3: Training and validation accuracy across epochs, six class merged model. 73

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Abbreviation	**Expansion**
VRA	Variable Rate Application
CRA	Constant Rate Application
CNN	Convolutional Neural Network
CA	Chromatic Aberration
IoT	Internet of Things
API	Application Programming Interface
UI	User Interface
HTTP	Hypertext Transfer Protocol
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
WiFi	Wireless Fidelity
GPIO	General Purpose Input Output
DFD	Data Flow Diagram
UML	Unified Modeling Language
IoU	Intersection over Union
F1	F1 score (harmonic mean of precision and recall)
SVM	Support Vector Machine
GPU	Graphics Processing Unit
TFLite	TensorFlow Lite
PWM	Pulse Width Modulation
NC	Normally Closed
NO	Normally Open
DC	Direct Current

mL	Millilitre
mA	Milliampere
mAh	Milliampere hour
RD	Reduction, percent chemical reduction
RSD	Relative Standard Deviation
mIoU	Mean Intersection over Union
mAcc	Mean Pixel Accuracy
UCI	University of California, Irvine (dataset repository)

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Rice is one of the most important food crops grown in India and in many other parts of Asia. Farmers who grow paddy depend on a healthy crop for their income and for their family's food. A paddy field is, however, open to many diseases caused by bacteria and fungi. Bacterial blight, brown spot, leaf smut and blast (including the neck blast form that attacks the stem near the ear) are common examples. Even a partial attack by one of these diseases on a field can lower the grain yield by a noticeable margin, and a bad outbreak can destroy a large part of the standing crop. Farmers therefore spray chemical pesticides and fungicides on their fields to protect the plants.

Most small and medium farms in India still use hand-operated or simple motor driven sprayers. These sprayers release the chemical at one fixed rate as the operator walks or drives through the field. The rate does not change with how sick or how healthy a particular plant is. A plant that shows no disease at all receives the same dose as a plant with a serious infection. This way of spraying is called constant rate application (CRA).

1.2 THE PROBLEM WITH UNIFORM SPRAYING

Uniform, constant rate spraying wastes chemical on plants that do not need it. The extra chemical adds to the farmer's cost without adding any benefit, and it runs off into the soil and nearby water bodies. Heavy and repeated use of the same chemical also helps pests and pathogens build resistance over time, which makes the chemical less useful in future seasons. Workers who spray the field by hand are exposed to more chemical than is needed, and some residue can remain on the harvested grain.

An alternative is variable rate application (VRA). In VRA, the sprayer senses how diseased a plant or a zone of the field is, and it adjusts the amount of chemical released for that plant or zone. A healthy plant gets little or no chemical, and a badly diseased plant gets a full dose. VRA can therefore cut the total chemical used while still treating the diseased parts of the field properly.

VRA systems that use maps made in advance (map based VRA) are not well suited to disease control, because a disease spreads through a field quickly and a map made even a day earlier may no longer match the real situation. A sensor based VRA, where the camera, the processing and the spraying all happen together in the same pass through the field, is better suited to disease control because the decision is made from the plant in front of the nozzle at that moment.

1.3 CHEMICAL OVERUSE AND ITS WIDER EFFECT

Chemical overuse in agriculture is a cost problem for the individual farmer, and it is also an environmental and a health problem. Pesticide residue that enters streams and groundwater affects aquatic life and can reach drinking water supplies. Farm workers who handle and apply chemicals without proper protection face health risks from repeated exposure. Consumers are also exposed to any residue left on the food they eat. A system that only sprays what is needed, and only where it is needed, reduces all three of these effects at the same time, alongside the direct saving in chemical cost.

1.4 MOTIVATION

Tewari et al. (2020) built and field tested a variable rate sprayer for paddy disease control. Their system used two webcams, a laptop running MATLAB, and a chromatic aberration based image threshold to find diseased leaf regions. The laptop then worked out a severity level and opened a solenoid valve for a set time so that the correct volume of chemical was sprayed. Their field trial, conducted on a 300 square metre paddy plot, showed a minimum chemical saving of 33.88 percent compared with constant rate spraying. This result proves that disease severity based variable rate spraying genuinely saves chemical in a real paddy field, and it is the base paper for our project.

The base paper's own method has limits that later work, including ours, tries to improve on. A chromatic aberration threshold is a simple colour rule, and it works less well when the lighting in the field changes, or when a disease other than the ones it was tuned for appears. A laptop with MATLAB is heavy, needs mains or a large battery, and is not something a small farmer would want to carry across a field. The base paper's three severity levels always spray at least 33 percent of the recommended rate, so even a perfectly healthy plant receives some chemical.

Convolutional neural networks (CNN) have by now been shown repeatedly to classify leaf diseases from a photograph more accurately and more robustly than a hand tuned colour rule, across lighting conditions and across more disease classes (Mohanty et al., 2016). Small, low power microcontroller boards with a built in camera and WiFi, such as the ESP32-CAM, together with free cloud services for running a trained model, now make it possible to replace the laptop with a device that fits in one hand and costs a fraction of the price. An internet of things (IoT) dashboard lets a farmer check the status of the sprayer and the chemical used from a phone, without standing next to the machine.

These three ideas, a CNN classifier in place of a colour threshold, a compact ESP32 based controller in place of a laptop, and an added severity level for a fully healthy plant that receives no spray at all, form the motivation for this project.

1.5 PROBLEM STATEMENT

Existing uniform sprayers waste chemical on healthy plants and under treat severely diseased plants in the same pass. Existing variable rate systems that do use image based disease detection, including the base paper, rely on simple threshold rules that are sensitive to lighting and that generalise poorly to diseases they were not tuned for, and they run on bulky, costly compute hardware with no remote monitoring. There is a need for a compact, low cost system that can classify the specific disease on a paddy leaf, measure how much of the leaf area it covers, and use both pieces of information together to decide how much chemical to spray, while also letting the farmer monitor the process remotely.

1.6 OBJECTIVES

The objectives of this project are as follows.

1. Build a deep learning classifier that identifies the disease on a paddy leaf image, trained using transfer learning so that it works well with a limited amount of labelled rice disease data.

2. Build a severity measurement step that segments the diseased pixels on the leaf and reports the diseased area as a percentage of the leaf area, instead of relying on the classifier's confidence score as a proxy for severity.

3. Combine the disease class and the measured severity into a reduction coefficient, and from the coefficient compute the chemical volume per plant and the solenoid valve open time needed to deliver that volume.

4. Design a compact, low cost hardware path, an ESP32-CAM for image capture and WiFi, an Arduino Nano for valve timing, a relay driven solenoid valve and a small pump, that carries out the spray decision in the field.

5. Provide an IoT dashboard so that severity readings, chemical usage and alerts can be checked remotely, and provide a web demonstration interface that shows the full decision process on an uploaded leaf photograph.

6. Measure the system's classification accuracy, severity measurement quality and the chemical saving against constant rate spraying, and compare these results with the base paper and with other published work.

1.7 SCOPE OF THE PROJECT

The project covers the paddy crop and six leaf conditions: a healthy leaf and five disease classes, bacterial blight, brown spot, leaf smut, blast and neck blast. The classification model and the severity pipeline are built, trained and tested in software, and are demonstrated through a Flask based web application. The spray decision logic (severity to coefficient to chemical volume to valve open time) is implemented in the same software and is also the logic that the hardware firmware is designed to follow. Cloud based inference is used, through a Flask server that the ESP32-CAM reaches over WiFi, because running a CNN directly on a microcontroller is outside the budget and time available for this project; on device inference with a smaller, converted model is noted as future work in Chapter 10. The hardware build follows the bill of materials and wiring plan described in Chapter 5; any measured bench, integration or field results depend on the physical prototype build reported by the team, and are presented exactly as supplied, with a clearly marked placeholder wherever a reading was not yet available at the time of writing this report.

CHAPTER 2

LITERATURE SURVEY AND PROJECT GOALS

2.1 OVERVIEW

This chapter reviews published work in three areas that are relevant to our project: variable rate spraying systems, deep learning for plant disease detection, and sensing or monitoring methods used in precision agriculture. Each paper is summarised in terms of what it did, what it found, and what it does not cover. The chapter ends with the research gap we identified and the way our project addresses that gap.

2.2 VARIABLE RATE SPRAYING SYSTEMS

Tewari et al. (2020) is the base paper for this project. The authors built a prototype that used two webcams and a laptop running MATLAB to capture images of paddy plants, a chromatic aberration based threshold to find the diseased leaf region, and an Arduino microcontroller to open a solenoid valve for a computed duration. Three severity levels (low, medium, high, covering 0 to 5 percent, 5.1 to 20 percent and above 20 percent diseased area) were each assigned a fixed reduction coefficient of 0.33, 0.67 and 1.00 of the recommended chemical rate. Field testing in a 300 square metre paddy plot with 20 by 20 centimetre hill spacing, using the fungicide monocrotophos 36SL, gave an average constant rate application (CRA) volume of 504.17 L/ha and an average variable rate application (VRA) volume of 319.5 L/ha, a minimum chemical saving of 33.88 percent (reduction varied between 33.88 and 40.47 percent across trials, with an average reduction of 36.65 percent, found significant at p less than or equal to 0.05). The method is sensitive to lighting because chromatic aberration thresholding is a colour based rule, it needs a laptop in the field, and it always sprays at least 33 percent of the recommended dose even on a plant with almost no disease.

Oberti et al. (2016) built a modular agricultural robot under the EU CROPS project that carried an R-G-NIR multispectral disease sensor and a precision spraying end effector to control powdery mildew on grapevines. Their robot detected 85 to 100 percent of the diseased canopy area and reduced pesticide use by 65 to 85 percent compared with uniform spraying. The system is complex and costly to build, and it is tuned specifically to grapevine canopies and a single disease, which limits how directly it can be reused for a paddy crop.

Esau et al. (2013) developed an automated, prototype variable rate sprayer for wild blueberry that used machine vision to tell foliage apart from bare soil, and sprayed fungicide only on the foliage. Measured fungicide saving ranged from 9.90 to 51.22 percent depending on the field's bare spot pattern. Their decision is a binary spray or no spray choice based on foliage presence; it does not estimate how diseased the foliage actually is, so it cannot scale the dose to disease severity the way our project does.

Zhang et al. (2025) proposed a variable rate sprayer for fruit trees that fuses two sensing modes, a canopy volume measurement (4.34 percent error) and a disease spot detector (82.6 percent accuracy), to set the spray dose. Their system reduced chemical use by 74.0 percent at low disease severity and by 42.7 percent at high severity compared with a conventional sprayer, and a field trial on pear rust reached 86.53 percent disease control with 66.4 and 22.2 percent savings against a traditional sprayer and a volume only variable rate sprayer respectively. Their canopy volume sensor is built for tree shaped crops and is not suited to a low growing crop such as paddy, which has little canopy depth to measure.

Abioye et al. (2024) presented a small robotic sprayer for a citrus nursery that used a MobileNetV2 model on a Raspberry Pi 4 to classify plant samples with a reported classification accuracy of 100 percent on their controlled, indoor test set, and reached 87 percent spray coverage on lemon foliage against 67 percent on navel orange, measured with water sensitive cards. The robot works indoors on potted nursery plants and does not report a severity based dosage; it sprays by classification result alone.

Samseemoung et al. (2017) designed a variable rate sprayer for coconut plantations that used classical image processing to estimate disease density and wireless remote monitoring to track the spraying operation. They found optimal design values of 1 metre nozzle to target distance, 1.5 bar pressure, 2.712 L/min spray rate, 1.5 km/h movement speed, 0.58 L/h fuel use and 0.056 ha/h working capacity for a tree sized crop. The system does not use deep learning and is specific to tall, widely spaced trees, unlike the dense, low growing paddy rows this project targets.

2.3 WEED DETECTION AND SITE SPECIFIC SPRAYING

Xu et al. (2018) built a real time weed mapping and precision herbicide spraying system for row crops that used an improved particle swarm optimisation (IPSO) algorithm to segment weeds from crop, cutting the segmentation error rate from 7.1 to 0.1 percent with a processing speed of 0.026 seconds per frame, and a measured weed detection to spray actuation response time of 1.562 seconds. This is a useful reference point for how fast a sensor based variable rate system needs to respond, but the paper targets weeds, not crop disease, and it does not estimate a severity level.

Partel et al. (2019) developed a low cost smart sprayer using a custom CNN for real time weed detection paired with individually actuated nozzles. Using the stronger of their two embedded GPUs, they reached 71 percent precision and 78 percent recall on a scenario with real plants, and 91 percent accuracy on an easier scenario with artificial plants. Their system is weed focused and makes a simple target or non target decision rather than grading a severity level.

Rehman et al. (2019) built a machine vision weed detector for wild blueberry fields using colour co-occurrence texture features and a quadratic classifier, reaching 94.98 percent training accuracy and 93.80 percent test accuracy in laboratory evaluation, and field savings between 30.12 and 60.58 percent depending on the amount of weed cover. Their method relies on hand engineered texture features rather than a learned deep network, and again addresses weeds rather than plant disease.

2.4 DEEP LEARNING FOR PLANT AND RICE DISEASE DETECTION

Mohanty et al. (2016) trained AlexNet and GoogLeNet with transfer learning on 54,306 images covering 14 crop species and 26 diseases from the PlantVillage dataset, reaching 99.35 percent accuracy with GoogLeNet and colour images on a held out test set drawn from the same distribution as the training images. When the same trained model was tested on two independent sets of images taken under uncontrolled field conditions, accuracy fell sharply to 31.40 and 31.69 percent (a random guess baseline would score 2.63 percent). This lab to field accuracy gap is an important lesson for our project: a classifier that scores well on a clean, single leaf benchmark image can still perform far worse on a messy, real world photograph, and

our own pipeline simulation (Chapter 9) shows a version of the same effect on a small hand picked test set.

Sethy et al. (2020) compared 11 CNN backbones used as transfer learned classifiers against the same backbones used only as deep feature extractors followed by a support vector machine (SVM), on 5,932 field images of four rice diseases (bacterial blight, blast, brown spot and tungro). The best result was ResNet50 features combined with an SVM, with an F1 score of 0.9838; a lightweight MobileNetV2 plus SVM combination reached a comparable F1 score of 0.9796. Their best setup needs a separate SVM training stage after the CNN feature extraction step, which adds complexity compared with an end to end classifier such as the one used in our project.

Rahman et al. (2020) trained a custom, two stage small CNN on 1,426 rice images covering 9 diseases and 3 pest categories, comparing it with MobileNet, NASNet-Mobile and SqueezeNet. Their proposed small model reached 93.3 percent accuracy with a model size about 99 percent smaller than VGG16, which is a useful demonstration that a small CNN can match larger backbones on a rice disease task, but their dataset and classes do not match our six paddy classes directly.

Petchiammal et al. (2023) released the Paddy Doctor dataset, 16,225 labelled paddy leaf images across 13 classes (12 diseases and a healthy class), and benchmarked it with ResNet34, VGG16, MobileNet and Xception. ResNet34 gave the best result, an F1 score of 97.50 percent. This dataset is one of the larger public paddy specific resources available, and its scale is a useful comparison point for the size of our own merged training set (Chapter 4).

Chen et al. (2020) used InceptionV3 with ImageNet transfer learning plus a domain adaptation step to narrow the gap between laboratory and field conditions, reaching at least 91.83 percent validation accuracy on a public dataset and 92.00 percent average accuracy on real rice field images. Their domain adaptation idea, training with both clean and field like images so the model generalises better to the field, is a relevant direction for improving our own classifier's field performance in future work.

PlantSeg (Wei et al., 2024) is a large, in the wild dataset built for plant disease segmentation rather than classification, containing 11,400 diseased images and 8,000 healthy images with pixel level masks across many crop species and disease categories. The paper's own strongest baseline model, SegNeXt with an MSCAN-L

backbone, reached 44.52 percent mean intersection over union (mIoU) and 59.95 percent mean pixel accuracy (mAcc) on their test split; lighter baselines such as DeepLabV3 and DeepLabV3+ reached only 17 to 27 percent mIoU. We use PlantSeg to train our own lesion segmentation model (Chapter 4), and our measured validation performance is compared against these baseline numbers in Chapter 9.

2.5 RESEARCH GAP

Putting the above work together, three gaps stand out for a paddy specific, low cost, severity based sprayer.

First, the systems that actually spray variable amounts in the field (Tewari et al., Oberti et al., Esau et al., Zhang et al., Samseemoung et al.) either use simple colour or texture thresholds instead of a learned deep network, or they run on costly, bulky compute hardware such as a laptop with MATLAB or a full sized robot platform. None of them pair a CNN disease classifier with a separately measured, pixel level severity score and a compact, WiFi connected microcontroller.

Second, the systems that do use strong CNN classifiers for plant disease (Mohanty et al., Sethy et al., Rahman et al., Petchiammal et al., Chen et al.) are evaluated purely as classification benchmarks. They report accuracy on a labelled test set but do not connect their output to an actual spray dose or valve timing, and they do not report a measured severity percentage derived from segmentation.

Third, the base paper's three severity levels always spray a minimum of 33 percent of the recommended rate, so a plant with no visible disease at all still receives chemical. No paper in this survey adds an explicit, zero chemical "Healthy" category driven by the classifier, which is a direct, simple way to remove wasted spraying on healthy plants.

2.6 PROBLEM STATEMENT (RESTATED FOR DESIGN)

Based on the gap above, the specific problem this project solves is: how to combine a CNN disease classifier and a pixel level severity measurement, on compact and low cost hardware with remote monitoring, so that a paddy sprayer can apply the correct chemical dose for each plant, including zero dose for a healthy plant, without depending on a laptop or a hand tuned colour threshold.

2.7 OBJECTIVES AND NOVELTY OVER THE BASE PAPER

Table 2.1 summarises how our system differs from the base paper, Tewari et al. (2020), on each of the main design points.

Table 2.1: Comparison of the base paper and our proposed system.

Aspect	Tewari et al. (2020), base paper	Our system
Disease detection	Chromatic aberration colour threshold	EfficientNetV2-B0 CNN classifier, transfer learning
Severity measurement	Same colour threshold used for both detection and severity	Separate U-Net segmentation model measures diseased pixels
Compute platform	Laptop with MATLAB	ESP32-CAM plus cloud Flask inference
Monitoring	None	Blynk IoT dashboard and a web demo interface
Severity levels	3 levels, minimum spray is 33 percent of recommended rate	4 levels, Healthy class sprays 0 percent
Dose scaling	Same coefficient for every disease at a given severity	Per disease factor further scales the coefficient
Estimated cost	Laptop, webcams and a MATLAB licence	Target under Rs. 2,500 (Chapter 5)

Our specific contributions, in line with the objectives in Chapter 1, are: replacing the colour threshold with a trained CNN classifier so that detection is more robust to lighting and covers more disease classes; measuring severity with its own segmentation model rather than reusing the classifier's confidence score, so the severity number reflects the actual diseased leaf area; adding a per disease dosing factor, because some diseases, such as neck blast, are treated as needing a relatively higher dose at the same severity level than others; and replacing the laptop with a compact ESP32-CAM and Arduino Nano pair, with a Blynk dashboard for remote status. These points are described in full in Chapters 4, 5 and 6, and are checked against measured results in Chapter 9.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 OVERVIEW

This chapter lists what the system must do (functional requirements), what quality it must meet while doing it (non functional requirements), and whether the project is realistic to build within the available time, budget and skills (feasibility). It also lists the hardware and software specification used in the rest of the report.

3.2 FUNCTIONAL REQUIREMENTS

Table 3.1: Functional requirements of the system.

ID	Requirement
FR1	The system shall accept a paddy leaf image, either uploaded through the web demo or captured by the ESP32-CAM.
FR2	The system shall classify the leaf image into one of six classes: Healthy, Bacterial_Blight, Blast, Brown_Spot, Leaf_Smut, Neck_Blast.
FR3	The system shall segment the leaf into leaf pixels and diseased pixels and report severity as a percentage of leaf area.
FR4	The system shall map severity percentage to one of four categories (Healthy, Low, Medium, High) using fixed thresholds, each with a base reduction coefficient.
FR5	The system shall scale the base coefficient by a disease specific factor to get an effective reduction coefficient.
FR6	The system shall compute the chemical dose per hill and the solenoid valve open time from the effective coefficient, using the stated dose equations.
FR7	The system shall flag a prediction as low confidence when the classifier's top score is below a fixed threshold.
FR8	The system shall treat a severity reading below a fixed floor as Healthy, even if the classifier predicted a disease, to avoid spraying for a few stray pixels.
FR9	The hardware shall receive the category from the cloud inference step over WiFi and actuate the solenoid valve for the computed

	duration.
FR10	The system shall report status, severity and chemical usage on a remote IoT dashboard (Blynk).
FR11	The web demo shall show the uploaded leaf, the predicted class and confidence, the segmented mask, the severity calculation steps, and the resulting valve open time, for the panel and for general use.

3.3 NON FUNCTIONAL REQUIREMENTS

Table 3.2: Non functional requirements of the system.

ID	Requirement
NFR1	Classification accuracy on a held out validation set should be above 80 percent for the deployed six class model.
NFR2	One full inference cycle (classification plus severity) should typically complete in under one second on the cloud server, excluding network transfer time.
NFR3	The complete spray decision cycle, from image capture to valve actuation, should stay under 5 seconds under normal WiFi conditions.
NFR4	The hardware bill of materials should stay close to the Rs. 2,500 budget target set at the proposal stage.
NFR5	The system should degrade gracefully: if the trained segmentation model is not available, a classical colour based severity method should still run; if automatic leaf isolation fails, the user should be able to trace the leaf outline by hand.
NFR6	The valve open time should always stay within a safe window (0.5 to 3.0 seconds) so a wrong reading cannot empty the tank in one cycle.
NFR7	The web demo and the API should be usable without installing anything beyond a web browser on the client side.

3.4 FEASIBILITY STUDY

3.4.1 Technical feasibility

The project uses well tested, publicly documented building blocks: EfficientNetV2-B0 and U-Net are standard, well published architectures; TensorFlow and Keras are mature, free libraries with large community support; the ESP32-CAM, Arduino Nano, relay module and solenoid valve are commodity parts that are commonly used together in hobbyist and small scale automation projects; Flask is a lightweight, well known Python web framework suited to exposing a trained model as an HTTP endpoint; and Blynk provides a free tier IoT dashboard with ready made mobile and web clients. All of the software components were implemented and tested during this project (Chapters 6, 8 and 9), which confirms they work together as planned. The main technical risk, keeping cloud inference fast enough for a 5 second spray cycle, was measured directly (Chapter 8, API test results) rather than assumed.

3.4.2 Economic feasibility

The complete hardware bill of materials (Chapter 5) targets a build cost of around Rs. 2,250 to Rs. 2,400, with a full possible range of Rs. 1,930 to Rs. 2,720 depending on the exact parts sourced. All of the software tools used (TensorFlow, Flask, the Arduino IDE, Google Colab's free GPU tier, and Blynk's free tier) are free to use. This keeps the total cost of the prototype low enough to be realistic for a student project and, longer term, for a smallholder farmer, which was one of the motivations in Chapter 1.

3.4.3 Social feasibility

A system that reduces chemical cost and chemical exposure benefits the farmer directly, and reducing pesticide runoff benefits the wider community and environment around the field, as discussed in Chapter 1. The hardware and firmware are deliberately kept simple (an Arduino Nano, a single relay, one solenoid valve) so that a farmer or a local technician without specialised training could realistically maintain the prototype, which matters for the system to be socially useful rather than only being usable inside a lab.

3.5 HARDWARE SPECIFICATION

Table 3.3: Hardware specification summary (full bill of materials in Chapter 5).

Component	Specification
Image capture and WiFi	AI-Thinker ESP32-CAM, OV2640 type sensor, 320 by 240 pixel raw RGB565 capture (the fitted camera module has no hardware JPEG, Chapter 5, section 5.6.1), 4 MB PSRAM, 802.11 b/g/n WiFi
Valve control microcontroller	Arduino Nano V3.0, ATmega328P, CH340 USB to serial
Actuation	5 V, 1 channel relay module with optocoupler isolation
Flow control	12 V DC, normally closed, 1/2 inch solenoid valve
Liquid delivery	R385 12 V DC diaphragm pump, self priming, about 1.5 to 2 L/min
Spray pattern	Adjustable brass hollow cone nozzle, 1/4 inch thread
Power source	3 x 18650 Li-ion cells (about 11.1 V nominal) or a 12 V sealed lead acid battery
Voltage regulation	LM2596 adjustable buck converter, set to 5.00 V for the microcontrollers and relay
Liquid path	6 mm internal diameter food grade silicone tubing

3.6 SOFTWARE SPECIFICATION

Table 3.4: Software specification summary.

Layer	Technology	Purpose
Model training	Python, TensorFlow and Keras, Google Colab (GPU)	Train the classifier and the segmentation model
Disease classifier	EfficientNetV2-B0, transfer learning	Classify the leaf image into one of six classes
Lesion segmenter	U-Net with a MobileNetV2 encoder, Dice plus binary cross entropy loss	Segment diseased pixels from leaf pixels
Interactive leaf isolation	MobileSAM, plus a free form polygon fallback	Isolate the leaf from a cluttered background when needed
Inference API	Flask, running the trained models	Serves the `/predict` endpoint used by the web demo and by the ESP32-

		CAM
Web demo	HTML, CSS and JavaScript (static page served by Flask)	Lets a user upload a leaf photo and see the full decision process
ESP32-CAM firmware	Arduino IDE, HTTPClient library	Captures the image, sends it to the API, receives the category, forwards it to the Nano
Arduino Nano firmware	Arduino IDE, SoftwareSerial library	Computes the valve open time and drives the relay
IoT dashboard	Blynk (free tier)	Remote monitoring of severity, usage and alerts

3.7 ASSUMPTIONS AND CONSTRAINTS

The system assumes a working WiFi connection (a mobile hotspot or a field router) is available so the ESP32-CAM can reach the cloud inference server; this is a known constraint of the cloud based design and is discussed again as a limitation in Chapter 9. The default dosing constants, an application rate (A) of 500 L/ha and a hill density (d) of 25 hills per square metre, and the design flow rate (Qs) of 3.5 mL/s, are taken from the hardware plan as starting values; the flow rate in particular is meant to be replaced by a measured value once the pump and solenoid are calibrated on the built prototype (Chapter 5, Chapter 8). The six leaf classes used by the model (Chapter 4) are the classes the training data provides; other paddy diseases that are not in the training data are outside the current scope.

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

4.1 SYSTEM ARCHITECTURE

The system has five stages that run one after another for every spray decision: image capture, disease classification, severity segmentation, dose and timing calculation, and spray actuation. Figure 4.1 shows these stages as a block diagram, and Figure 4.2 shows the same cycle as a flowchart with the decision points made explicit.

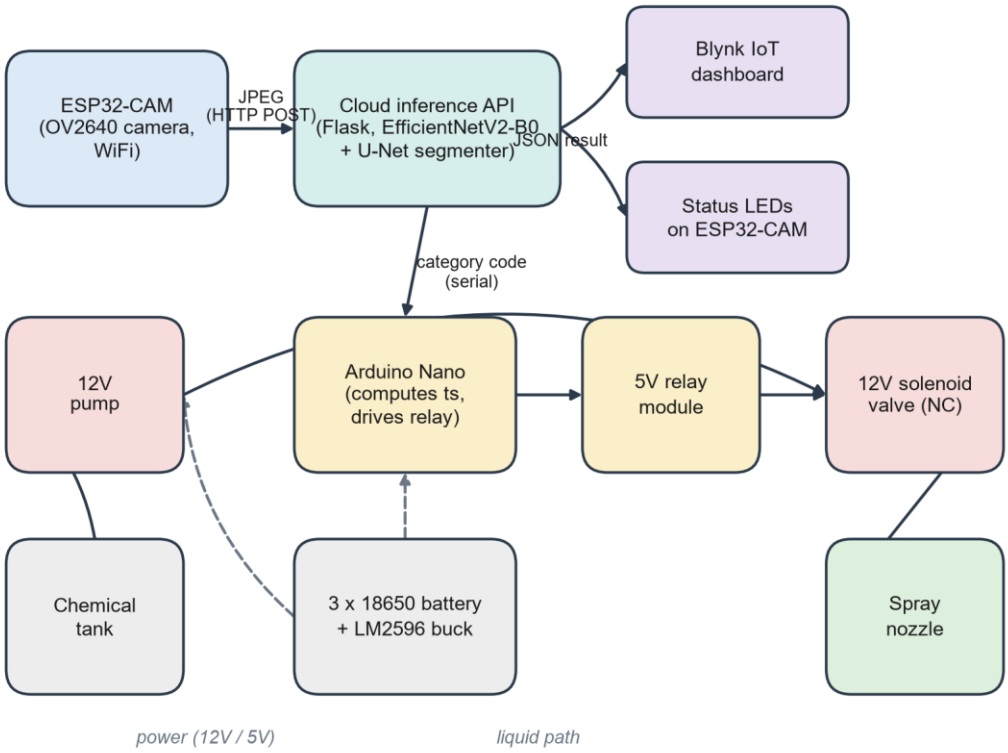


Figure 4.1: System block diagram showing the five stages of one spray cycle.

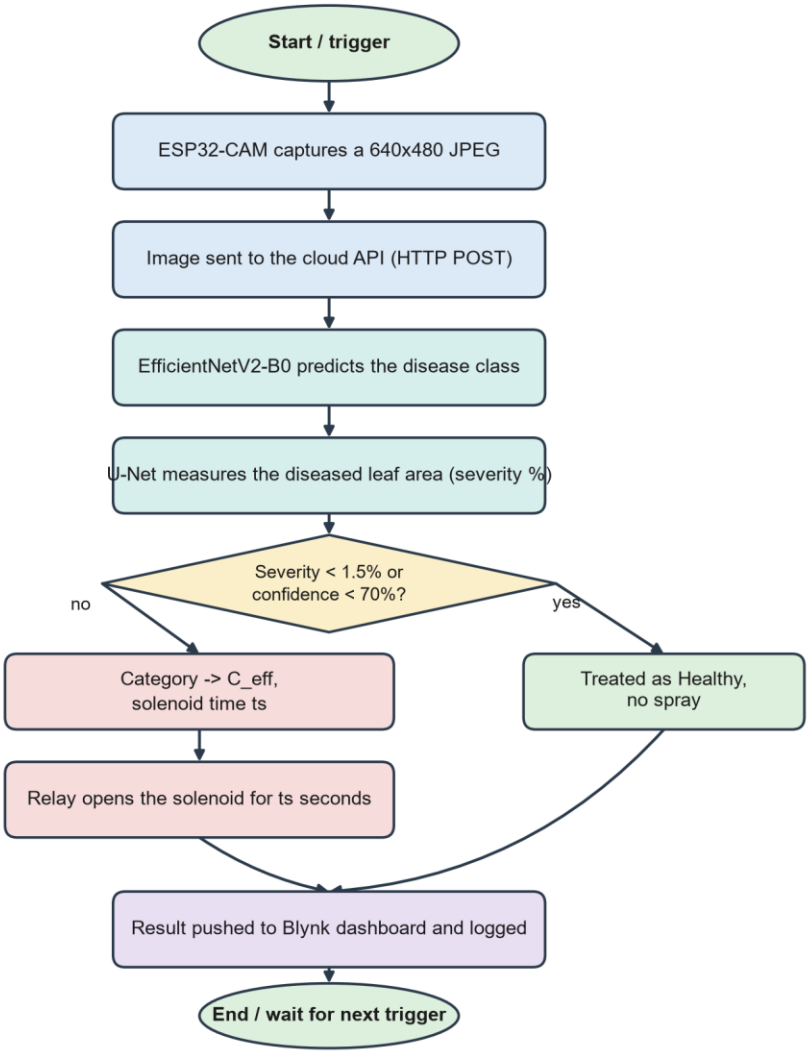


Figure 4.2: Flowchart of one complete spray decision cycle, from trigger to valve actuation.

Figure 4.3 shows the same system as layers: the sensing layer (camera and trigger), the processing layer (CNN classifier, segmentation model, dose logic, all running on the cloud server), the communication layer (WiFi, HTTP, serial link, Blynk), and the actuation layer (relay, solenoid valve, pump, nozzle).

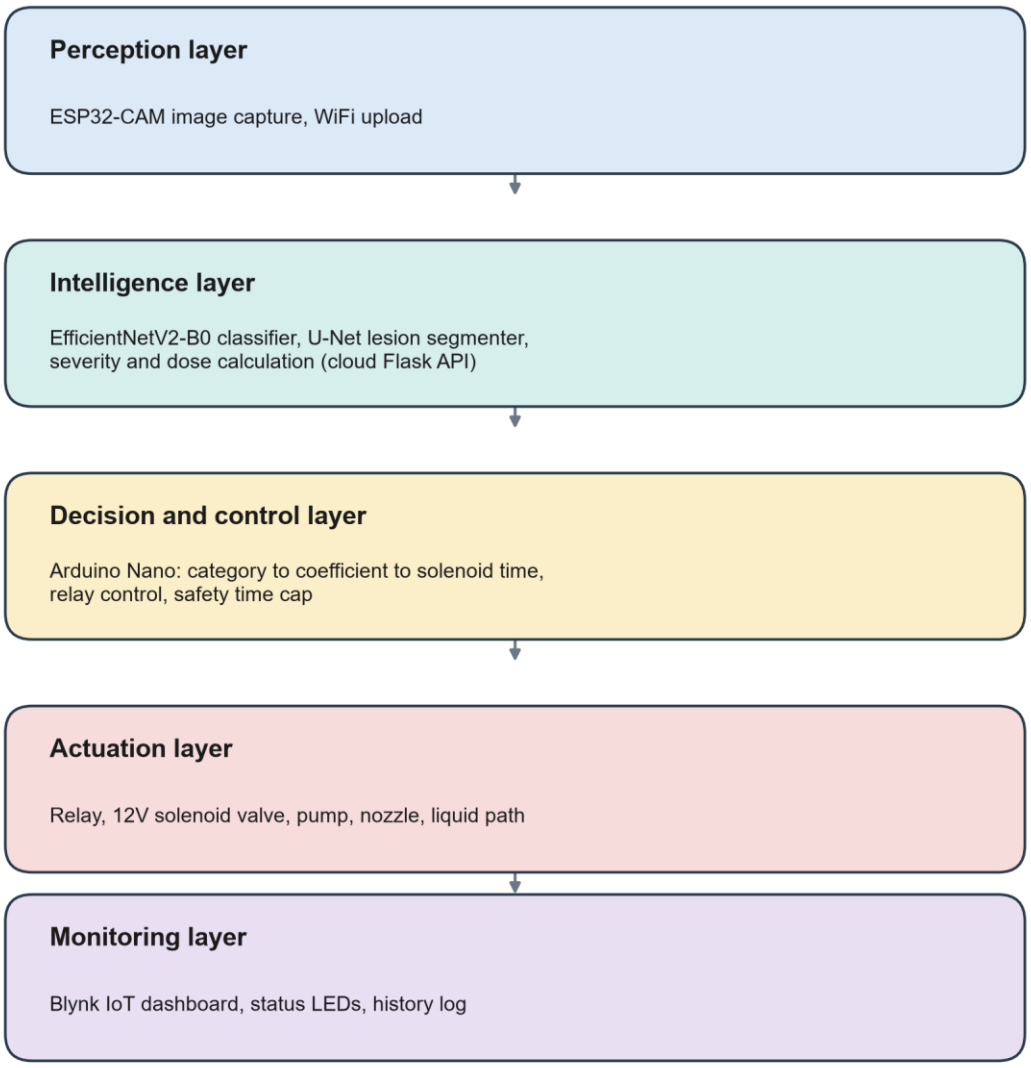


Figure 4.3: Layered view of the system: sensing, processing, communication and actuation.

4.2 ONE SPRAY CYCLE, STEP BY STEP

1. A trigger starts the cycle: a push button on the ESP32-CAM, a timer, or a remote command from the Blynk dashboard.

2. The ESP32-CAM captures a 640 by 480 pixel JPEG image through its OV2640 sensor.

3. The ESP32-CAM sends the JPEG over WiFi as an HTTP POST request to the Flask `/predict` endpoint running on the cloud server.

4. The server runs the EfficientNetV2-B0 classifier on the image and returns the predicted disease class and a confidence score.

5. The server runs the U-Net segmentation model (or, if that model cannot be loaded, a classical colour based method) to find the diseased pixels and the leaf pixels, and computes the severity percentage.

6. The server maps the severity percentage to a category and a base coefficient, scales the coefficient by a disease specific factor to get the effective coefficient, and computes the chemical dose per hill and the valve open time.

7. The server returns a JSON response with all of the above fields to the ESP32-CAM.

8. The ESP32-CAM maps the category to a single digit code (0 for Healthy, 1 for Low, 2 for Medium, 3 for High) and sends that digit over a serial link to the Arduino Nano. At the same time it updates the Blynk dashboard and lights the matching status LED.

9. The Arduino Nano looks up (or, in the full design, recomputes) the valve open time for the received code, and drives the relay, which switches the 12 V solenoid valve open for that duration. A code of 0 (Healthy) results in no actuation at all.

10. The system returns to waiting for the next trigger.

4.3 DATASETS

Two public datasets were combined to train the deployed six class classifier: an open rice leaf disease dataset from the UCI repository, contributing the Bacterial_Blight, Brown_Spot and Leaf_Smut classes (55 images in each of the two thin classes, see Chapter 9 for the exact test numbers), and an open rice disease dataset hosted on HuggingFace, contributing the Healthy (1,488 images), Blast (977 images), Neck_Blast (1,000 images) and Brown_Spot (667 images, added to the UCI brown spot images) classes. Combining the two datasets gives six classes in total and removes a problem seen while training on the UCI set alone, where a fully healthy leaf had no class of its own and could be wrongly pushed toward one of the three disease labels. Figure 4.4 shows the class distribution of the merged training set, and Figure 4.5 shows one real sample image from each of the available Capstone test folders.

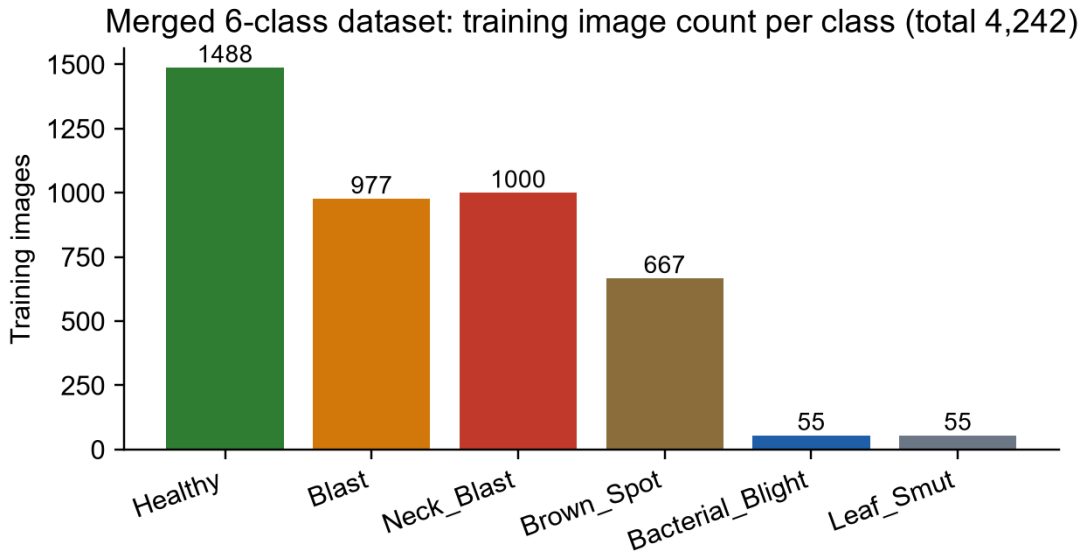


Figure 4.4: Class distribution of the merged six class training set.



Figure 4.5: Sample leaf images from the Capstone test set, one row per class.

The two thin classes, Bacterial_Blight and Leaf_Smut, have only 55 training images each, far fewer than the other four classes. This imbalance is a known limitation of the merged dataset and is addressed with class weighting during training (Chapter 6) and discussed again in Chapter 9.

The lesion segmentation model is trained separately on PlantSeg (Wei et al., 2024), a large, in the wild dataset of 11,400 diseased plant images and 8,000 healthy plant images with pixel level masks, downloaded from the open Zenodo record for this project. PlantSeg covers many crop species beyond paddy; using it to train a general lesion segmenter, rather than training a paddy specific segmenter from scratch on a much smaller dataset, was a deliberate design choice given the limited amount of pixel level paddy lesion data available.

4.4 CLASSIFICATION MODEL

The disease classifier is EfficientNetV2-B0, used with transfer learning: the backbone is initialised from ImageNet weights and then fine tuned on the merged six class rice dataset. EfficientNetV2-B0 was chosen over a larger backbone, such as a full sized ResNet or VGG network, because it reaches competitive accuracy with far fewer parameters, which keeps inference fast enough for the cloud hosted, real time use case this project needs (consistent with the lightweight backbone choices reported by Sethy et al., 2020, and Rahman et al., 2020, in Chapter 2). Figure 4.6 shows the classifier's architecture at a high level: an input image resized to the network's expected size, the EfficientNetV2-B0 backbone, and a final dense softmax layer with six outputs, one per class.

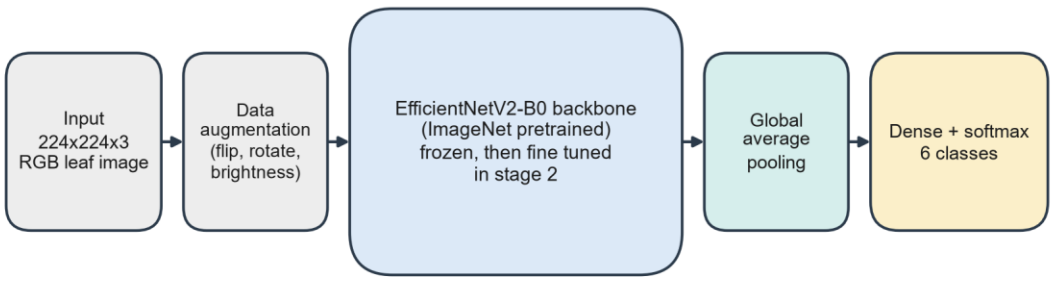


Figure 4.6: EfficientNetV2-B0 classifier architecture, input to six class softmax output.

Training used class weighting to reduce the effect of the two thin classes on the loss function, and standard image augmentation (random rotation, flip and brightness and contrast changes) to make the model more robust to the kind of variation seen in field photographs. Measured training and validation accuracy curves and the confusion matrices are presented with the numbers in Chapter 9.

4.5 SEVERITY ESTIMATION

Severity is measured, not estimated from the classifier's confidence. A segmentation model marks each pixel of the leaf image as leaf or background, and each leaf pixel as diseased or healthy. Severity is then

```
severity_percent = 100 * diseased_pixels / leaf_pixels
```

The segmentation step uses a U-Net architecture with a MobileNetV2 encoder, trained with a combined Dice and binary cross entropy loss, which is a common and effective loss choice for the heavily imbalanced, mostly healthy pixel masks typical of lesion segmentation. Figure 4.7 shows the severity pipeline from the input leaf image through to the severity percentage.

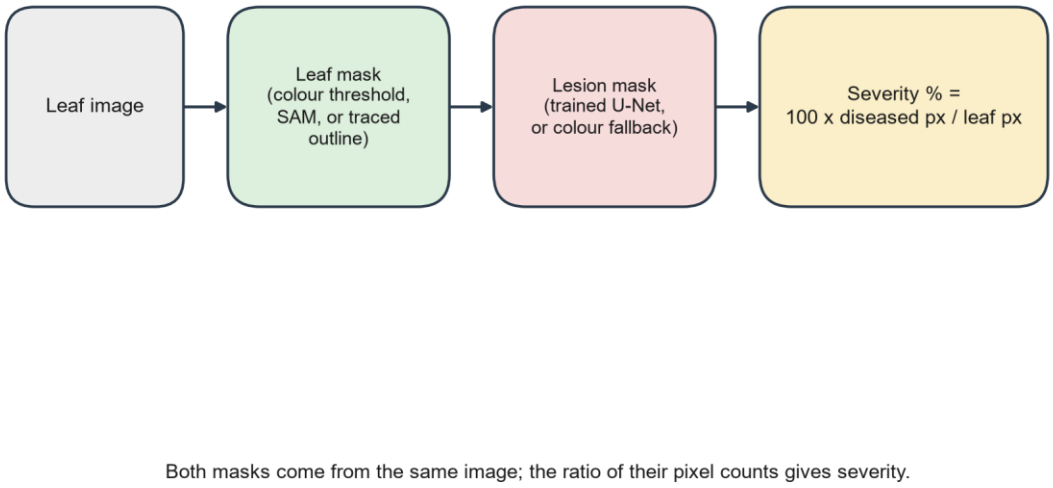


Figure 4.7: Severity pipeline: leaf isolation, lesion segmentation, pixel counting, severity percentage.

Before segmentation runs, the leaf itself must be separated from the background, because a cluttered field photograph can otherwise have background pixels wrongly counted as leaf or as disease. Two methods are available for this leaf isolation step:

MobileSAM, a lightweight version of the Segment Anything Model, run automatically or from a box the user drags over the leaf; and a free form polygon, drawn by the user tracing the leaf outline by hand, used when the automatic method fails on a difficult background. If the trained segmentation model itself cannot be loaded, the system falls back to a classical colour based threshold method for finding diseased pixels, so that a severity number is still produced (NFR5 in Chapter 3).

4.6 DOSE AND SPRAY TIME EQUATIONS

Severity maps to one of four categories using fixed thresholds, each with a base reduction coefficient, as implemented in the deployed severity module:

Table 4.1: Severity category thresholds and base coefficients, as implemented.

Severity S (percent)	Category	Base coefficient (C_base)
0 (classifier predicts Healthy), or measured severity below the 1.5 percent floor	Healthy	0.00
0.1 to 5.0	Low	0.30
5.1 to 20.0	Medium	0.55
above 20.0	High	0.80

The base coefficient is then scaled by a factor that depends on the diagnosed disease, because some diseases are treated as needing a proportionally higher dose at the same severity level:

Table 4.2: Per disease scaling factor applied to the base coefficient.

Disease	Factor
Healthy	0.00
Leaf_Smut	0.80
Brown_Spot	1.00
Blast	1.15
Bacterial_Blight	1.20
Neck_Blast	1.25

The effective coefficient is `C_eff = clamp(C_base x disease_factor, 0, 1)`, clamped so it cannot exceed 1.00 even for the highest severity, highest factor

combination. The chemical dose per hill and the solenoid open time then follow the equations used in the base paper (Tewari et al., 2020, Eqs. 4 and 5), with the open time standardised to a realistic valve window:

```
a  = (C_eff x A) / (d x 10)              # chemical per hill, mL/hill
ts = t_min + C_eff x (t_max - t_min)     # solenoid open time,
seconds
```

where `A` is the recommended application rate in L/ha (design value 500), `d` is the hill density in hills per square metre (design value 25), and `t_min` and `t_max` define the valve's safe open time window (0.5 and 3.0 seconds). Using a fixed window for `ts`, instead of the base paper's `ts = a / Qs` divide by flow rate, keeps the valve open time inside a safe, known range regardless of the exact flow rate, which matters because the real flow rate `Qs` depends on the physical pump and solenoid and is only known once the hardware is calibrated (Chapter 5, Chapter 8). Section 9.6 reports dose and valve time worked examples using the design value of `Qs` until a measured value is available, and compares the two dosing conventions used in the base paper and in our implementation.

Two safeguards are applied before this calculation. First, if the classifier's top confidence score is below 0.70, the prediction is flagged as low confidence rather than acted on with full certainty (FR7). Second, if the measured severity is below a floor of 1.5 percent of the leaf area, the leaf is treated as Healthy even if the classifier predicted a disease class, because a severity reading that small is more likely to be noise or a few stray dark pixels than a genuine infection (FR8).

4.7 DESIGN DIAGRAMS

Figure 4.8 and Figure 4.9 show the data flow diagram (DFD) for the system at level 0 (the system as a single process exchanging data with its external entities) and level 1 (the system broken into its main internal processes).

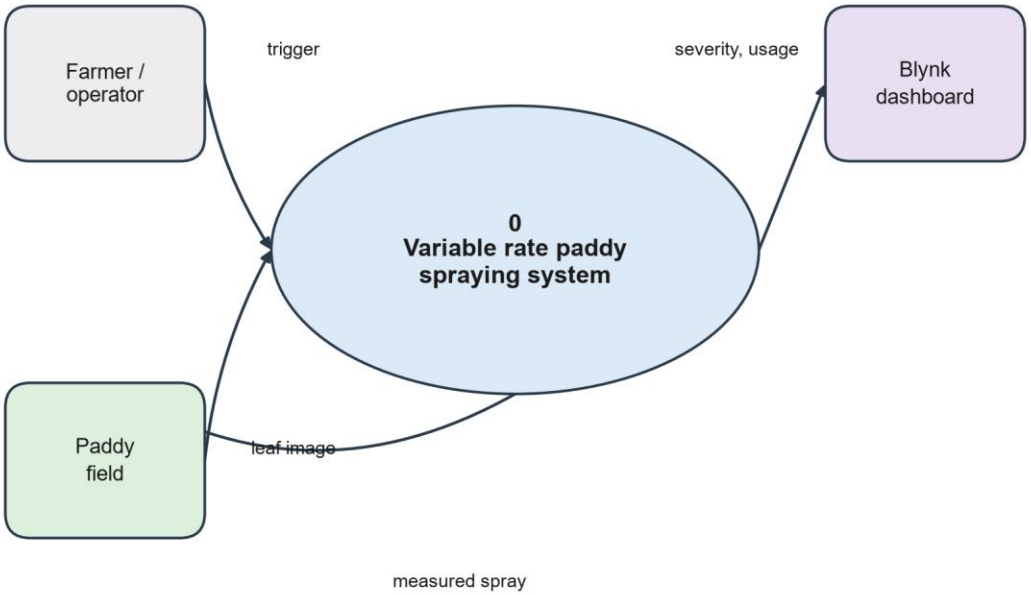


Figure 4.8: Data flow diagram, level 0.

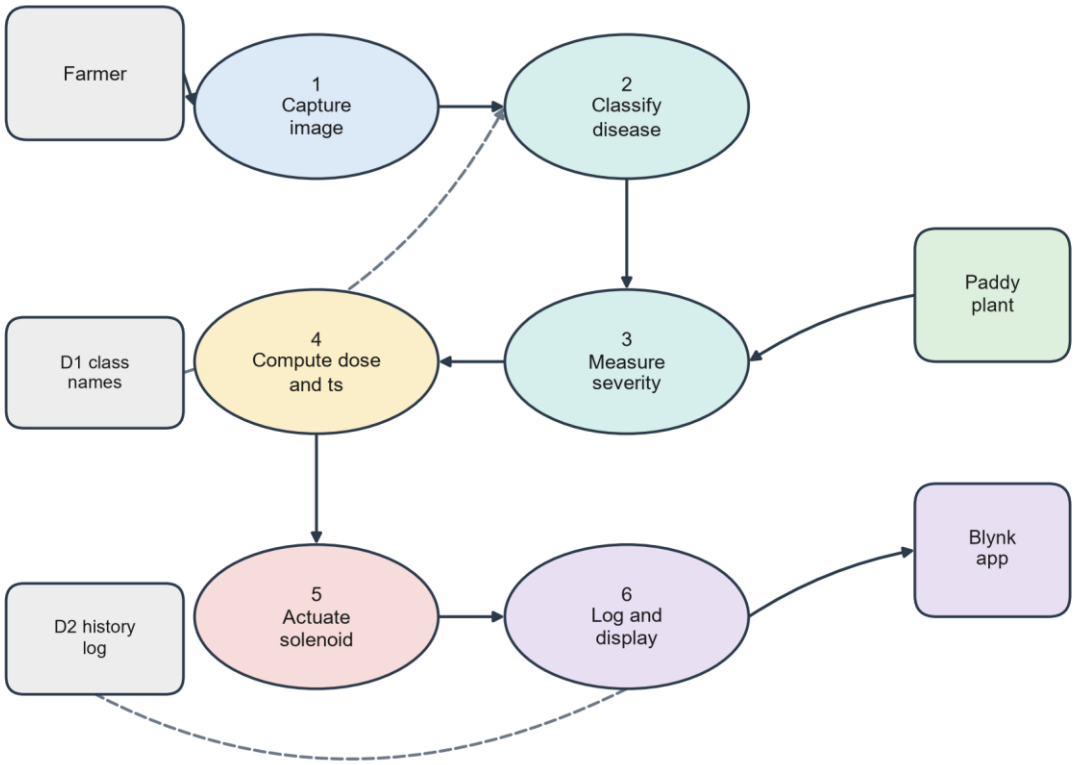


Figure 4.9: Data flow diagram, level 1, showing the main internal processes.

Figure 4.10 shows the use case diagram, covering the two actors relevant to the system, the farmer or panel reviewer who uploads an image or watches the dashboard, and the ESP32-CAM device that triggers a cycle automatically in the field.

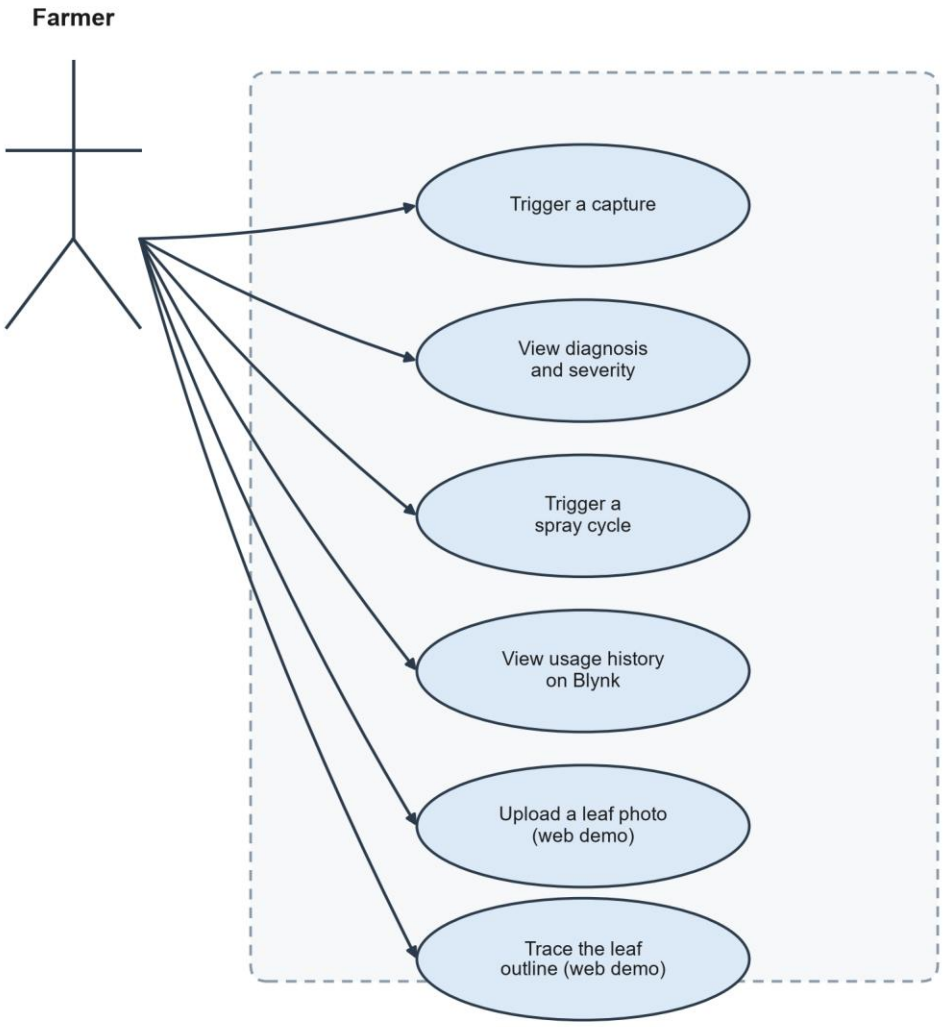


Figure 4.10: Use case diagram for the farmer or reviewer actor and the ESP32-CAM device actor.

Figure 4.11 shows the sequence diagram for one spray cycle, matching the steps listed in section 4.2, and Figure 4.12 shows the module or class level diagram, grouping the software into the Flask API layer, the model inference layer (classifier and segmenter), the severity and dosing logic, and the firmware layer (ESP32-CAM and Arduino Nano).

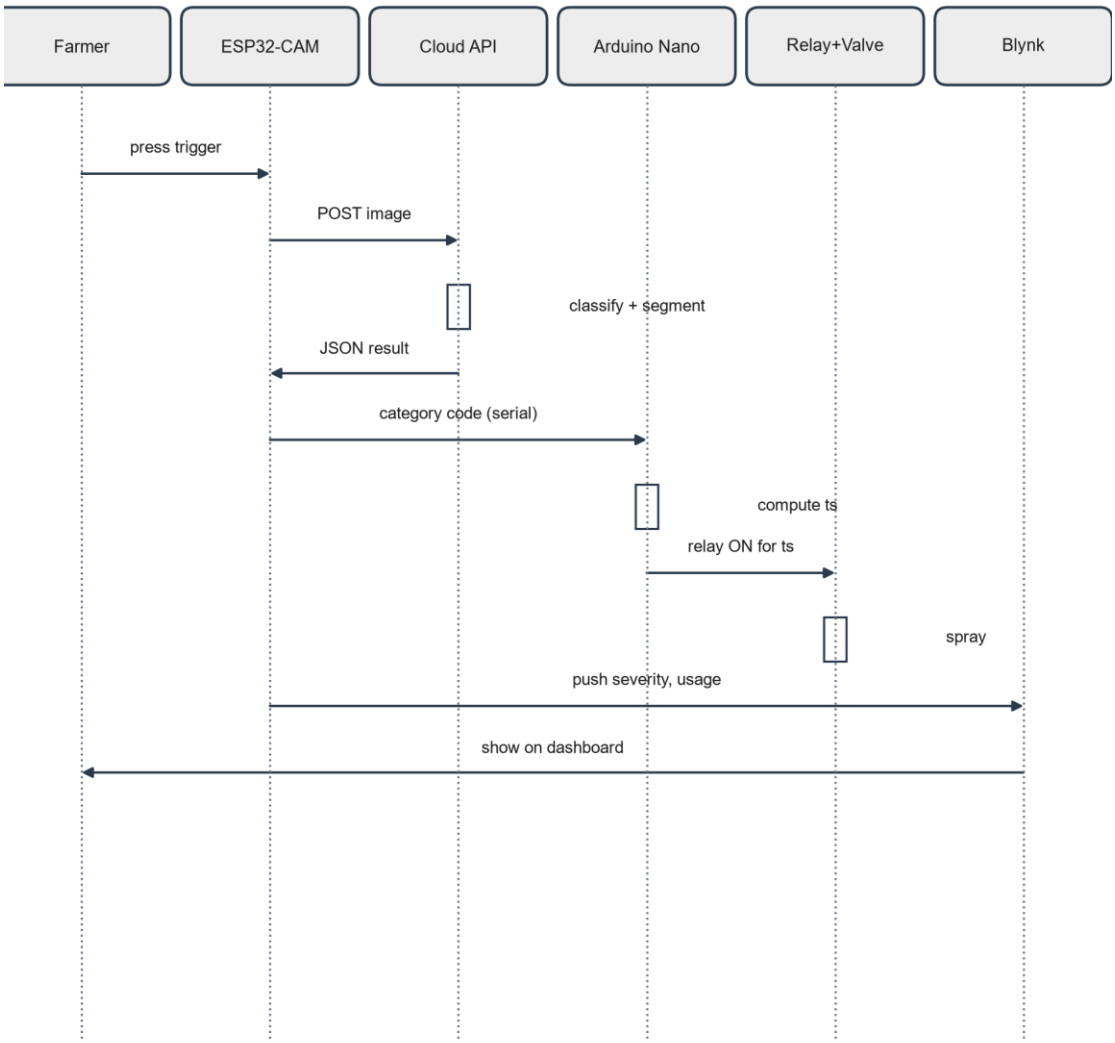


Figure 4.11: Sequence diagram for one spray cycle.

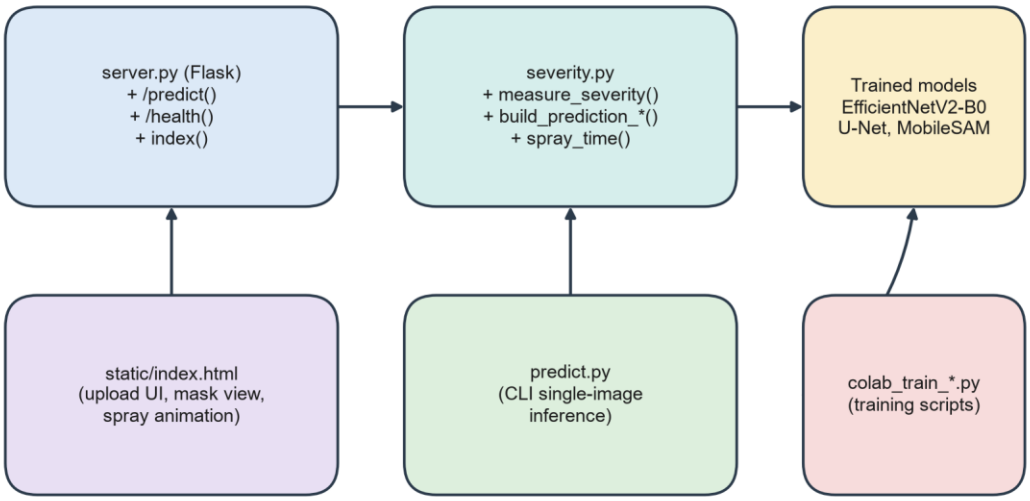


Figure 4.12: Module and class level diagram of the software.

4.8 CONSTRAINTS AND DESIGN TRADE OFFS

Cloud inference was chosen over on device inference because the trained EfficientNetV2-B0 and U-Net models are too large and too slow to run directly on an ESP32 class microcontroller within the project's time frame; the trade off is that the system needs a working WiFi connection in the field, which is recorded as a limitation (Chapter 9) and a future work item (Chapter 10, on device TensorFlow Lite inference). A standardised valve open time window was chosen over the base paper's direct divide by flow rate formula so the valve timing stays safe and bounded before the flow rate is calibrated on the actual hardware; this is a deliberate departure from the base paper, discussed in section 4.6 and again in Chapter 9. Finally, a single, general purpose lesion segmenter trained on PlantSeg was chosen over a paddy specific segmenter trained from scratch, because PlantSeg is large enough to train a reasonable segmentation model while no comparably sized pixel level paddy lesion dataset was available; the trade off, a drop in segmentation accuracy compared with a

dataset built specifically for paddy, is discussed with the measured IoU and Dice numbers in Chapter 9.

CHAPTER 5

HARDWARE IMPLEMENTATION

5.1 OVERVIEW

This chapter describes the physical build that carries out the spray decision in the field: the parts list, the wiring, the power system, the liquid path, the firmware behaviour, and the assembly and calibration procedure. The design in this chapter follows `hardware_implementation_plan.md`, the team's own hardware design document, because it is the most complete and most detailed source of hardware design information available for this report, except where the team's own bring up work on the real prototype found a problem with the plan and changed it, in which case that change is stated and the reason is given. Any value in this chapter that the team has since measured on the real, built prototype is taken from `Capstone/hardware_evidence/HARDWARE_TEST_DATA.md` and is clearly marked as measured. At the time of writing this report, the ESP32-CAM and the Arduino Nano have both been flashed and bench tested, and the two boards have been wired together and run through one integration cycle (section 5.8.2), but the flow rate calibration, the spray volume check with water, the full field trial and the full ten run cycle timing table are not yet done (section 5.8.1, section 5.8.2); every value that would normally come from one of those remaining tests is stated plainly as not yet available, rather than invented.

5.2 BILL OF MATERIALS

Table 5.1: Bill of materials for the hardware prototype.

#	Component	Specification	Approx. cost (Rs.)
1	ESP32-CAM	AI-Thinker, OV2640 sensor, WiFi and Bluetooth, 4 MB PSRAM, microSD slot	450 to 550
2	ESP32-CAM programmer	ESP32-CAM-MB, USB micro-B, CH340 chip	150 to 200
3	Arduino Nano	V3.0, ATmega328P, CH340, mini USB	200 to 280
4	Solenoid valve	12 V DC, normally closed, 1/2 inch,	250 to 400

		plastic body	
5	Relay module	5 V, 1 channel, optocoupler isolated (Songle SRD-05VDC-SL-C)	45 to 70
6	Water pump	R385 type, 12 V DC diaphragm, 1.5 to 2 L/min, self priming	200 to 350
7	Power pack	3 x 18650 Li-ion cells (3.7 V, 2,600 mAh) with holder, or a 12 V, 2 A sealed lead acid battery	300 to 450
8	Buck converter	LM2596, adjustable, 12 V to 5 V, with trim pot	45 to 70
9	Spray nozzle	Adjustable brass, hollow cone pattern, 1/4 inch thread	80 to 150
10	Silicone tubing	6 mm internal, 8 mm external diameter, food grade, about 2 m	60 to 100
11	Jumper wires and breadboard	Male to female and male to male kits, mini breadboard	100 to 150
12	Push button	12 mm momentary push button	10 to 20
13	Inline fuse	2 A blade fuse with inline holder	about 30

Total target cost: approximately Rs. 2,250 to Rs. 2,400, with a full possible range of Rs. 1,930 to Rs. 2,720 depending on exact parts sourced. At the time of writing this report, no parts had yet been purchased (section 5.11), so this is a planning estimate only; the actual amount spent should replace this estimate once procurement is complete.

5.3 CIRCUIT AND WIRING

Figure 5.1 shows the full circuit and wiring diagram. Table 5.1 lists the ESP32-CAM pin usage and Table 5.2 lists the Arduino Nano pin usage.

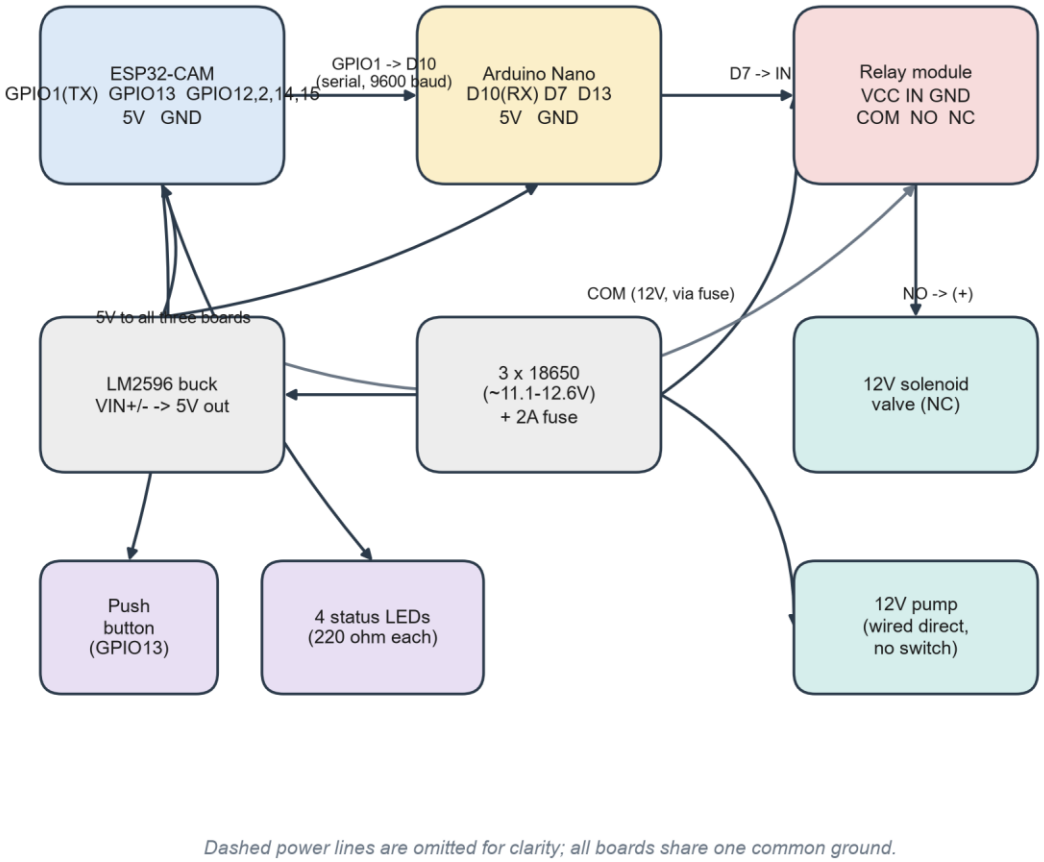


Figure 5.1: Complete circuit and wiring diagram of the prototype.

Table 5.2: ESP32-CAM pin usage, as implemented and flashed in esp32cam_sprayer.ino (Appendix 4).

ESP32-CAM pin	Connects to	Notes
5V	LM2596 5 V output	Power in, about 300 mA while active
GND	Common ground bus	Shared with Nano, relay and buck converter
GPIO1 (U0TXD)	Arduino Nano D10 (SoftwareSerial RX)	Sends the spray command, disconnect while flashing
GPIO13	Push button to GND	Manual trigger
GPIO12	Status LED, category 0 (Healthy)	Through a 220 ohm resistor to GND
GPIO2	Status LED, category 1 (Low)	Through a 220 ohm resistor to GND
GPIO14	Status LED, category 2 (Medium)	Through a 220 ohm resistor to GND

GPIO15	Status LED, category 3 (High)	Through a 220 ohm resistor to GND

The hardware plan's original pin table put the trigger button on GPIO0. That pin is physically wired to the camera module's XCLK (clock) line on every AI-Thinker ESP32-CAM board, so it cannot be used as a free digital input once the camera is running, and must stay floating except for the few seconds it is grounded to enter flashing mode. This mismatch was found while writing the firmware (section 5.6) and an early draft of this report moved the button to GPIO16 instead. Bring up on the real board then found a second problem with GPIO16 itself, described in detail in section 5.6.1, and the button was moved a second time, to its final position on GPIO13. GPIO12 is a flash voltage strapping pin, but it boots correctly with a status LED attached, because the LED and its resistor pull it to the same logic level (LOW, 3.3 V flash) the board already expects at boot; this behaviour was confirmed directly, since the Healthy category LED on GPIO12 is part of the firmware that has now been flashed and run.

Table 5.3: Arduino Nano pin usage.

Nano pin	Connects to	Notes
5V	LM2596 5 V output	Power in, about 50 mA
GND	Common ground bus	Shared with ESP32, relay and buck converter
D10 (SoftwareSerial RX)	ESP32-CAM GPIO1 (TX)	Receives the spray command line `S<ms>,<cat>`
D7	Relay module IN	HIGH switches the relay on, which opens the solenoid
D13	Onboard LED	Lights while the valve is open

The relay's COM terminal connects to the battery positive terminal through the inline fuse, and its NO (normally open) terminal connects to the solenoid valve positive terminal, so the solenoid is off by default and energises only while the relay is triggered.

5.4 POWER SYSTEM

Figure 5.2 shows the power distribution diagram. The 12 V battery pack feeds the LM2596 buck converter, which is adjusted with its trim pot to exactly 5.00 V, measured with a multimeter under load, before any microcontroller is connected (an overshoot at this step can destroy the ESP32-CAM or the Nano). The regulated 5 V output feeds the ESP32-CAM, the Arduino Nano and the relay module's logic side. The unregulated 12 V rail feeds the relay's switched side through the inline 2 A fuse, and also feeds the pump directly. The hardware plan originally called for a manual on or off toggle switch in series with the pump. During assembly the team chose to leave this switch out and wire the pump straight to the 12 V rail instead, so the pump runs continuously whenever the battery is connected and stops only when the battery is disconnected. This is a deliberate simplification, not an oversight: the pump only pressurises the line, and it is the solenoid valve, gated by the relay, that actually decides how much liquid reaches the nozzle, so a separate pump switch adds a part without changing what the system actually controls.

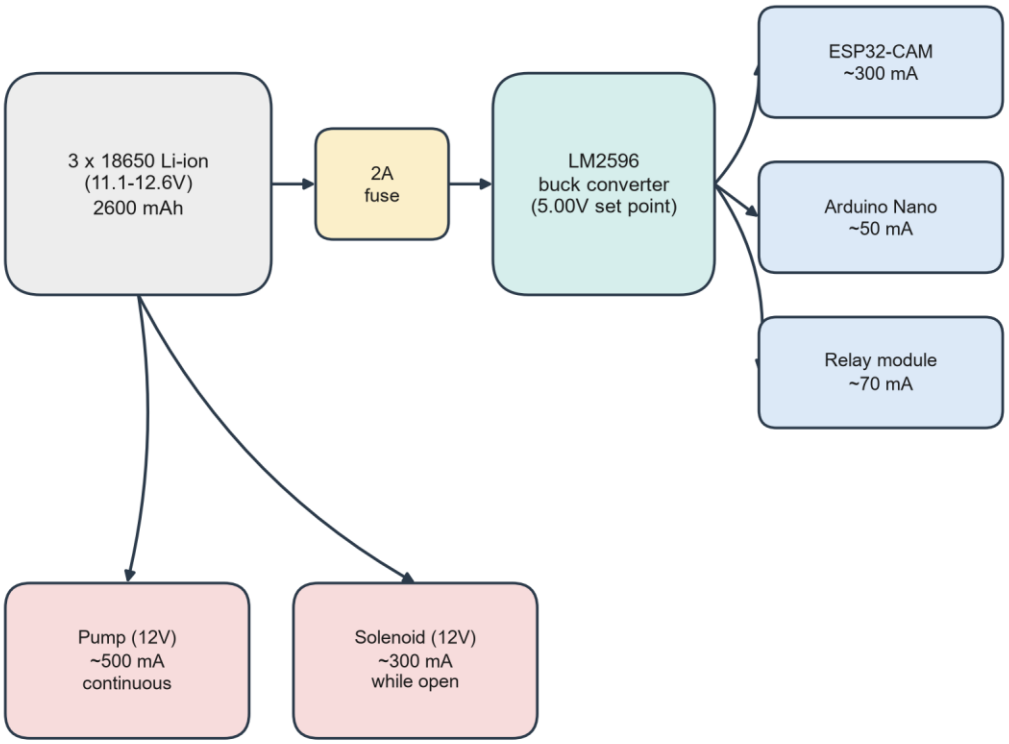


Figure 5.2: Power distribution diagram: battery, buck converter, 5V and 12V rails.

Table 5.4: Approximate current draw per rail.

Rail	Component	Approximate current
5 V	ESP32-CAM, active WiFi and camera	about 300 mA
5 V	Arduino Nano	about 50 mA
5 V	Relay module	about 70 mA
12 V	Pump, continuous	about 500 mA
12 V	Solenoid, while open	about 300 mA

Total current draw is approximately 1.2 A at 12 V. A fully charged 2,600 mAh battery pack gives roughly two hours of continuous operation at this draw. The actual battery voltage when fully charged cannot be measured until the battery pack is assembled (section 5.11); a 3 x 18650 Li-ion pack of this type is nominally 11.1 V and typically

reads close to 12.6 V fully charged, but the real reading from `HARDWARE_TEST_DATA.md` section A should replace this expectation once the pack exists.

5.5 LIQUID PATH

Figure 5.3 shows the liquid path from the chemical tank to the nozzle. The chemical tank outlet connects through silicone tubing to the pump inlet; the pump outlet connects to the solenoid valve inlet; and the solenoid valve outlet connects to the hollow cone nozzle. The pump runs continuously once switched on, and the solenoid valve, not the pump, controls how much liquid reaches the nozzle by opening only for the computed duration `ts`.

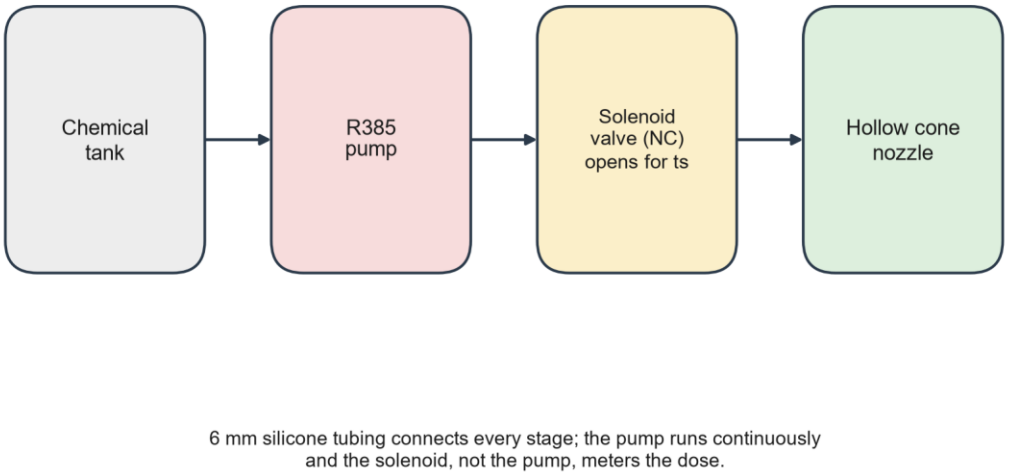


Figure 5.3: Liquid path diagram: tank, pump, solenoid valve, nozzle.

5.6 FIRMWARE BEHAVIOUR

The ESP32-CAM firmware (`esp32cam_sprayer.ino`, in `Capstone/hardware_evidence/firmware/`, listed in full in Appendix 4) waits for a trigger (a push button on GPIO13, an optional timer, or a remote button on the Blynk dashboard), captures one 320 by 240 pixel frame through the `esp_camera` driver, and sends the frame by HTTP POST to the Flask `/predict` endpoint. Section 5.6.1 below explains why the frame is sent as raw pixel data rather than as a JPEG image, and how the server side was changed to accept it; this is the same request path already proven by the project's own API integration tests (Chapter 8, section 8.3, case

I11), extended with two new query parameters. On receiving the JSON response, the firmware reads `category_code` (0 for Healthy, 1 for Low, 2 for Medium, 3 for High) and `spray_time_s`, lights the matching status LED, pushes severity and category figures to the Blynk dashboard over WiFi if the dashboard is enabled in the firmware build (section 5.7), and sends a line of the form `S<ms>,<cat>` over its serial TX pin (GPIO1) to the Arduino Nano's SoftwareSerial RX pin (D10) at a baud rate of 9600, where `<ms>` is the server's `spray_time_s` value converted to whole milliseconds and `<cat>` is the category code carried along for logging on the Nano side.

Sending the already computed spray time, rather than only the 0 to 3 category code, is a deliberate design choice made while writing this firmware. The effective coefficient `C_eff` that sets the spray time (section 4.6) already includes the per disease factor from `model/severity.py`; recomputing a spray time on the Nano from the category code alone would silently drop that per disease scaling and could produce a different, wrong duration from the one the server actually decided on. Carrying the finished value forward keeps the dosing logic in exactly one place, the Flask server, which is also the only place it is unit tested (Chapter 8, section 8.2).

5.6.1 Real problems found during firmware bring up

Two genuine hardware problems were found only after the real ESP32-CAM board was flashed, and both are reported here as found, since they are useful, honest engineering findings and not something to hide.

The first problem is the GPIO16 pin conflict already mentioned in section 5.3. An earlier draft of the firmware moved the trigger button to GPIO16, on the stated reasoning that GPIO16 "has no camera or SD card role on this board and is always free". This reasoning turned out to be wrong. GPIO16 is the chip select line for the ESP32's onboard PSRAM (the extra memory chip the camera driver needs for its frame buffer); the installed development toolchain's own configuration file confirms this directly (`CONFIG_D0WD_PSRAM_CS_IO=16`, and the paired clock line `CONFIG_D0WD_PSRAM_CLK_IO=17`). Setting GPIO16 as a digital input for the button silently disabled PSRAM, because the two uses conflict at the pin level. Every memory allocation larger than 4 kilobytes is served from PSRAM in this build configuration, so once PSRAM was disabled, the camera driver's own frame buffer

allocation began to fail, and the symptom on the Serial Monitor was a generic crash (`Guru Meditation Error: Core 1 panic'ed (LoadProhibited)`) at camera start up, not an obvious "PSRAM is off" message. The fix was to move the trigger button a second time, off GPIO16 entirely and onto GPIO13 (Table 5.1), and to move the status LED that previously used GPIO13 onto GPIO2 instead. Once the button was off GPIO16, PSRAM came back and the camera, WiFi connection and API calls all worked correctly in the same test session (section 5.8.2). GPIO16 and its paired clock pin, GPIO17, must not be used for any other purpose on this board, and this rule is now written directly into the firmware's own comments so it is not lost again.

The second problem is that the specific camera module fitted to this board does not support hardware JPEG encoding, even though the board is sold as an OV2640 based AI-Thinker ESP32-CAM, the same part number the hardware plan and section 5.6's original description assumed. Asking the camera driver for `PIXFORMAT_JPEG` fails with a driver level error ("JPEG format is not supported on this sensor"). A small test sketch, written to isolate the problem, confirmed that the same camera module captures correctly in grayscale and in RGB565 pixel formats, so the sensor itself and its ribbon connection are not faulty; only the JPEG path on this particular unit fails. Encoding the raw frame into JPEG in software on the ESP32 itself was tried next, and it ran out of usable memory on this board (`JPG buffer malloc failed`, and in one configuration another `LoadProhibited` crash) because the frame buffer and the software JPEG encoder's own working memory could not both fit in the space available once the earlier GPIO16 fix had moved the frame buffer back into PSRAM. Rather than keep fighting the on device encoder, the firmware now captures a 320 by 240 pixel RGB565 frame (153,600 bytes) and sends those raw pixel bytes directly over HTTP, with no JPEG step on the ESP32 at all. The Flask server (`app/server.py`) was extended with a small function, `_pil_from_rgb565`, that decodes the raw big endian RGB565 bytes into a normal colour image using numpy, before handing that image to the exact same classification and severity measurement pipeline already used for ordinary JPEG uploads from the web demo; the model, the segmentation module and the dosing equations are completely unchanged, only the first decoding step is different for this request path. The request carries two new query parameters, `fmt=rgb565` and the frame width and height, so the server knows to use

this decoding path instead of its normal JPEG decoder. A separate, smaller fix was needed alongside this one: the server's normal JSON response includes a large base64 encoded mask image (`overlay_png`) for the web demo, and the ESP32's own JSON parser ran out of memory trying to hold a response that big (`NoMemory`). The server now skips this field for ESP32 requests with a third query parameter, `overlay=0`, and the firmware's own JSON buffer was also made larger, from 1,024 to 4,096 bytes, as a second layer of protection.

Taken together, these two findings mean that the capture resolution stated earlier in some descriptions of this design, 640 by 480 pixels captured and sent as JPEG, is not what the flashed and running firmware actually does; the real, working design captures 320 by 240 pixels as raw RGB565 and decodes it on the server, as described above, and this is the version that produced the real integration test result in section 5.8.2 and Chapter 8, section 8.4.

The Arduino Nano firmware (`valve_controller.ino`, same folder, Appendix 4) listens on its SoftwareSerial RX pin for a line starting with `S`, parses the millisecond duration and the category code, and, if the duration is greater than zero, drives D7 high to energise the relay and open the solenoid for that duration, then drives D7 low again. A duration of 0 (Healthy) results in no actuation at all. A safety cap of 5,000 milliseconds, matching the hardware plan's own recommendation (section 11 of `hardware_implementation_plan.md`), clamps any longer request before it reaches the relay, so a bad reading cannot empty the whole tank in one cycle. The same firmware also accepts a single bare digit (0 to 3) typed directly into its USB Serial Monitor as a bench test mode, so the relay and solenoid wiring can be dry run with no camera, no WiFi and no API call at all, exactly as described in the hardware plan's Phase 2 pairwise integration step and in section 5.8 below.

Both `.ino` files have now been flashed onto a real ESP32-CAM and a real Arduino Nano. The WiFi network name and password and the Flask `/predict` URL placeholders inside `esp32cam_sprayer.ino` have been filled in with the team's own values (section 5.8.2); the Blynk auth token is also filled in, though `BLYNK_ENABLED` is currently 0 as explained in section 5.7. The design value `QS_MLS_DESIGN` inside `valve_controller.ino` should still be replaced with the measured flow rate once section 5.8.1 below has a real number.

5.7 CLOUD API AND BLYNK DASHBOARD

The cloud inference server is a Flask application (`app/server.py`) exposing a `/predict` endpoint that accepts a multipart image upload (used by the web demo), ordinary JPEG bytes in the request body, or a raw RGB565 frame with the `fmt=rgb565` query parameter (the form the ESP32-CAM firmware now sends, section 5.6.1). The response JSON carries the disease class, confidence, severity percentage, category, category code, base coefficient, disease factor, effective (reduction) coefficient, dose per hill and valve open time. During the bring up session reported in section 5.8.2, the server was run directly on the team's own laptop and reached over a mobile phone's personal WiFi hotspot, with the ESP32-CAM and the laptop joined to the same hotspot network; this avoids the Colab and ngrok public URL that changes on every restart, at the cost of needing the laptop and the ESP32-CAM to be on the same local network. The ngrok and Colab path described in the hardware plan (Table 10.1, Troubleshooting) remains available as an alternative for a demonstration where the laptop cannot sit on the same network as the sprayer.

The Blynk dashboard uses five virtual pin datastreams, matched one to one with the fields the firmware already sends: V0 (severity, a gauge), V1 (category, a labelled value), V2 (chemical volume per cycle, a history chart so usage over time is visible), V3 (an alert, shown as text) and V4 (a remote trigger button that the firmware's `BLYNK_WRITE(VPIN_REMOTE_TRIGGER)` handler already listens for). The exact template, datastream and widget configuration is written out in full in `Capstone/hardware_evidence/firmware/BLYNK_SETUP.md`, so creating the dashboard is a fill in the fields exercise rather than a design task. Current Blynk IoT does not support an anonymous public link to a device dashboard the way the original Blynk app once did; sharing a live device view with someone else now means inviting their registered account into the owning organisation, which is a different step from hosting a public URL. The dashboard itself has been created, and Figure 5.4 is a screenshot of it taken from the Blynk console, with the Severity gauge, Category, Alert text, chemical usage chart and Remote Trigger button all visible and populated. The firmware itself keeps the Blynk code path behind a single build flag, `BLYNK_ENABLED`, and this flag is currently set to 0 in the flashed and tested firmware (`esp32cam_sprayer.ino`), so the dashboard is not being written to by the physical prototype at the time of writing this report; the flag was turned off deliberately while

isolating the two problems in section 5.6.1, to reduce the number of things that could be going wrong on the board at once, and it has not yet been turned back on and re tested end to end. The dashboard connection, Blynk configuration and virtual pin write calls are already written and unaffected by either of the two fixes in section 5.6.1, so re enabling this flag and reflashing is expected to be straightforward once it is prioritised.

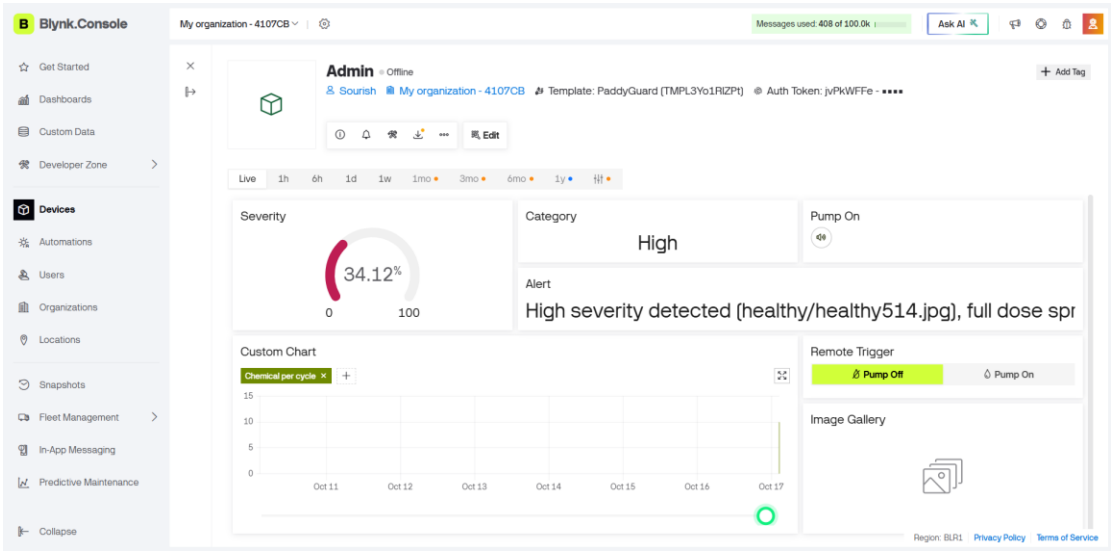


Figure 5.4: PaddyGuard Blynk dashboard, showing the Severity gauge, Category, Alert, the chemical usage history chart and the Remote Trigger button, populated by the pipeline simulation replay.

The numbers on the dashboard in Figure 5.4 come from `Capstone/hardware_evidence/firmware/push_to_blynk.py`, which replays the project's own measured pipeline simulation (the same 49 image run reported in Chapter 9) into these datastreams through Blynk's public HTTP API, one reading every few seconds. This is stated plainly because it matters: the screenshot shows the dashboard and its widgets genuinely working, but the severity and category values on it are a replay of the software simulation's real, already reported numbers, not live field telemetry from the physical prototype, for the reason given directly above (`BLYNK_ENABLED` is currently 0 in the flashed firmware). Once `BLYNK_ENABLED` is set back to 1 and the ESP32-CAM is reflashed, the same datastreams receive real field readings directly from `Blynk.virtualWrite()` calls already written into `esp32cam_sprayer.ino`, with no change to the dashboard itself; this is listed as a remaining item in OPEN_ITEMS.md.

5.8 ASSEMBLY AND CALIBRATION PROCEDURE

The build follows seven phases, each one tested on its own before the next begins, so that a wiring or firmware fault in one module does not get confused with a fault in another.

1. **Procurement.** Order every part in the bill of materials (section 5.2) and check each against its specification when it arrives.

2. **Standalone bench tests.** Test the ESP32-CAM, the Arduino Nano, the relay module, the buck converter, and the pump and solenoid, one at a time, on USB power only, with no battery and no full wiring yet.

3. **Pairwise integration.** Connect the ESP32-CAM to the Arduino Nano over serial and confirm category codes are received correctly; connect the Nano to the relay and solenoid for a dry run with test durations; confirm the ESP32-CAM can reach the Flask API and parse the JSON reply; confirm the ESP32-CAM can update the Blynk dashboard.

4. **Power system assembly.** Build the battery pack, verify its voltage, wire it to the buck converter, and set the buck converter to exactly 5.00 V under load before connecting any microcontroller.

5. **Full electrical integration.** Wire the complete signal chain on USB power first, confirm a full dry cycle (trigger to relay click), then switch to battery power and repeat the same test.

6. **Flow calibration.** With the pump running and the solenoid held open, collect the dispensed liquid for exactly 10 seconds into a measuring cup, three times, and average: `Qs = volume_mL / 10`. This is the one value in the dose equations (section 4.6) that must be measured rather than assumed, because it depends on the specific pump, solenoid and battery voltage used.

7. **End to end and field testing.** Run full cycles against printed or displayed images of each class, confirm the correct category lights and the correct volume sprays, and time the full cycle end to end. A field trial compares total chemical used in constant rate mode against variable rate mode on the same row, to check the chemical saving claim from Chapter 1 against a real measurement.

5.8.1 Measured flow rate and calibration result

The flow rate `Qs` has not yet been measured, because Phase 6, collecting the dispensed liquid for a timed interval, has not been run with water in the tank (section 5.8.2 below). This report therefore continues to use the design value `Qs` = 3.5 mL/s from the hardware plan for all worked dose and timing examples in Chapter 9, and labels every such example clearly as calculated from the design value, not measured. This flow rate is only used for the optional debug printout in `valve_controller.ino` (`QS_MLS_DESIGN`); it does not affect the valve open time itself, because `ts` arrives from the Flask server already computed (section 5.6), so recalibrating `Qs` once a measured value exists means updating one constant in the firmware comment rather than changing any timing logic. Measuring `Qs` is listed in OPEN_ITEMS.md as the single most important remaining hardware task, since every dose and timing figure in Chapter 9 depends on it being a real, measured number rather than the current design assumption.

5.8.2 Bench and integration test results

Phases 1 to 3 of the procedure in section 5.8 have now been carried out on the real boards, and the real results are reported here exactly as they happened, including one honest caveat on the single integration cycle described below. `Capstone/hardware_evidence/HARDWARE_TEST_DATA.md` still holds the detailed, phase by phase sheet for Phases 4 to 7 (power system assembly, full electrical integration on battery power, flow calibration, and field testing), which had not been run at the time of writing this report, and `Capstone/hardware_evidence/photos/` and `screenshots/` were still empty, so no hardware photograph is shown in this chapter.

The Arduino Nano's relay and solenoid valve were bench tested completely on their own, using the firmware's single digit test mode described in section 5.6 (typing `0`, `1`, `2` or `3` into the Nano's own USB Serial Monitor), with no camera, no WiFi and no ESP32-CAM connected at all. This test confirms the relay and valve timing logic independently of the rest of the system. The real Serial Monitor output from this test, exactly as recorded, is reproduced below.

```
[boot] PaddyGuard valve controller ready
[boot] bench test: type 0, 1, 2 or 3 in this Serial Monitor to dry-
run the relay
[bench] test category 0
```

```
[valve] category Healthy (or ts=0), no spray this cycle
[bench] test category 1
[valve] opening for 1250 ms, category code 1, expected volume at
design Qs: 4.38 mL
[valve] closed
[bench] test category 2
[valve] opening for 1875 ms, category code 2, expected volume at
design Qs: 6.56 mL
[valve] closed
[bench] test category 3
[valve] opening for 2500 ms, category code 3, expected volume at
design Qs: 8.75 mL
[valve] closed
```

The relay was heard to click and the solenoid valve was seen to open and close for the correct commanded duration on categories 1, 2 and 3, and correctly did nothing on category 0. This confirms that the valve open and close timing control works correctly in real hardware. The "expected volume" figures printed above (4.38, 6.56 and 8.75 mL) use the design value `QS_MLS_DESIGN` = 3.5 mL/s described in section 5.8.1, not a measured flow rate, so they must be read as an expected volume at an assumed design flow rate, pending the direct flow rate calibration in section 5.8.1, and not as a measured dispensed volume. No water was run through the valve during this bench test; it was a dry, relay click test only.

With the ESP32-CAM and Arduino Nano then wired together over the serial link described in Table 5.1 and Table 5.2, one full automated cycle of the assembled system was recorded, with the camera pointed at a leaf, the ESP32-CAM connected to a mobile phone hotspot, and the Flask server running on the team's laptop on the same hotspot network. The real Serial Monitor output from the ESP32-CAM side of this cycle, exactly as recorded, is reproduced below.

```
[boot] PaddyGuard ESP32-CAM sprayer controller
[trace] psramFound=1 freeHeap=284264 psramSize=4192123
[trace] about to call initCamera()
[trace] initCamera() returned OK
cam_hal: EV-EOF-OVF
[wifi] connecting..
[wifi] connected, IP 172.20.10.2
[boot] ready, waiting for trigger
[cycle] triggered
[cam] captured RGB565 320x240, 153600 bytes
[cycle] class=Bacterial_Blight conf=0.48 severity=26.78%
category_code=3 ts=2.90s
S2900,3
```

This single output shows the full signal chain working together in real hardware for the first time: the camera captured a real frame, the WiFi connection reached the laptop over the hotspot, the Flask server ran the real trained classifier and the real severity measurement on that frame and returned a result, the ESP32-CAM parsed

that result without the `NoMemory` error described in section 5.6.1, and it sent a real `S2900,3` command on its serial line toward the Nano. The line `cam_hal: EV-EOF-OVF` is a routine camera end of frame timing message from the driver, not a failure; the correct 153,600 byte frame size and the classification that follows show the capture succeeded. One honest caveat belongs with this result: the `[cycle] triggered` line carries the exact same timestamp as the `[boot] ready, waiting for trigger` line immediately before it, which is the signature of the ESP32's GPIO13 pin floating briefly at boot and being read as a button press, rather than a deliberately timed physical press of the button by whoever was running the test. The classification result itself is still genuine, real hardware evidence, since a real frame was really captured, really classified by the real server, and really turned into a real serial command; it should be read as an automated bench and integration cycle of the fully assembled system, not as a manually triggered field test, and this distinction is kept throughout this report wherever this result is referred to (Chapter 8, section 8.4; Chapter 9).

Table 5.5: Hardware test status, updated with the real results above.

Test phase	Status	Where the result is recorded
Standalone bench tests, ESP32-CAM camera, WiFi and API call	Done, passed (shown above)	This section
Standalone bench test, Nano relay and solenoid (dry, no water)	Done, passed (shown above)	This section, HARDWARE_TEST_DATA.md section B
Pairwise integration, ESP32-CAM to Nano serial link, one automated cycle	Done, passed, with the triggering caveat noted above	This section, HARDWARE_TEST_DATA.md section C
Blynk dashboard update from the physical prototype	NOT DONE, `BLYNK_ENABLED` is 0 (section 5.7)	HARDWARE_TEST_DATA.md section C
Flow calibration (measured Qs)	NOT DONE	HARDWARE_TEST_DATA.md section D
Spray volume check per category, with water	NOT DONE	HARDWARE_TEST_DATA.md section E
End to end cycle time, at least 10 runs	NOT DONE	HARDWARE_TEST_DATA.md section F
Class wise functional test	NOT DONE	HARDWARE_TEST_DATA.md section G
Full electrical integration	NOT DONE, tested on	HARDWARE_TEST_DATA.md

on battery power (section 5.8, Phase 5)	USB power only so far	section C
Field trial (constant rate vs variable rate)	NOT DONE	HARDWARE_TEST_DATA.md section H

5.8.3 Real problems faced and fixes

Section 5.6.1 above already describes the two most significant problems found during this bring up session in full detail: the GPIO16 PSRAM chip select conflict, and the camera module's lack of hardware JPEG support. A third, smaller problem is worth recording here. The mobile phone used for the WiFi hotspot has an apostrophe in its device name (for example, "Sourish's iPhone"), and the character iOS inserts by default for this apostrophe is not the plain keyboard apostrophe but a typographic "curly" character; the ESP32's WiFi connection call needs an exact, character for character match to the network name, so the firmware's WiFi network name constant had to be written using the matching Unicode character before the board would connect. The phone's Personal Hotspot setting "Maximize Compatibility" also had to be turned on, since the ESP32 cannot join a 5 GHz WiFi network and this setting forces the hotspot to the 2.4 GHz band the ESP32 needs. Table 5.4 above and `HARDWARE_TEST_DATA.md` section I should be updated as Phases 4 to 7 are carried out.

5.9 KNOWN HARDWARE RISKS

Table 5.3 lists risks identified at the design stage, carried over from the hardware plan's troubleshooting table, together with the mitigation already built into the design.

Table 5.6: Known hardware risks and mitigations, including two risks confirmed during real bring up (section 5.6.1, section 5.8.3).

Risk	Mitigation
ESP32-CAM fails to flash	Hold GPIO0 to ground during upload, use a powered USB hub to avoid brownouts
GPIO16 (or GPIO17) used for any purpose other than PSRAM	Confirmed during bring up (section 5.6.1): never assign GPIO16 or GPIO17 to a button, LED or sensor on this board; the trigger button was moved to GPIO13
Camera module cannot produce JPEG	Confirmed on this specific board (section 5.6.1): capture RGB565 instead and decode on the server with `_pil_from_rgb565`, rather than trying to encode JPEG on the

	ESP32 itself
WiFi will not connect to a phone hotspot	Confirmed during bring up (section 5.8.3): match the hotspot's exact SSID character for character, including any typographic apostrophe, and enable "Maximize Compatibility" for 2.4 GHz
Buck converter output drifts after handling	Re-measure under load with a multimeter before trusting it
Relay clicks but solenoid does not open	Check the solenoid is wired to NO, not NC; check for battery voltage sag under load
Inconsistent spray volume at the same `ts`	Battery voltage sag changes pump flow rate; recalibrate `Qs` at field voltage
Cloud API becomes unreachable	A laptop on the same WiFi hotspot as the ESP32-CAM avoids this (section 5.7); if using the ngrok and Colab path instead, update the URL in the firmware after every Colab restart

5.10 CROSS CHECK WITH THE HARDWARE PLAN

Four points in the hardware plan do not match the software and firmware that were actually built. First, the hardware plan specifies a MobileNetV2 classifier, but the model that was actually trained, evaluated and deployed (Chapter 4, Chapter 6, Chapter 9) is EfficientNetV2-B0. This report follows the code and the measured results, and uses EfficientNetV2-B0 throughout. Second, the hardware plan's severity table uses the base paper's original coefficients (0.33, 0.67, 1.00, with `ts = a / Qs`), while the deployed `severity.py` module uses different base coefficients (0.30, 0.55, 0.80) further scaled by a per disease factor, and a standardised valve open time window instead of a direct divide by flow rate. This report follows the deployed code as the implemented system (section 4.6) and states the base paper's original coefficients separately wherever the base paper itself is being discussed (Chapter 2, Chapter 9). Third, the hardware plan's pin table (section 6.1 of `hardware_implementation_plan.md`) puts the trigger push button on the ESP32-CAM's GPIO0 pin; writing the firmware (section 5.6) showed this is not workable, because GPIO0 is the camera module's XCLK line on every AI-Thinker ESP32-CAM board and must stay free while the camera runs. The button was moved to GPIO16 in an early draft of the firmware, then moved a second time to its final position on GPIO13, after real hardware bring up found that GPIO16 itself is the

ESP32's PSRAM chip select pin (section 5.6.1). Fourth, and found only during the same bring up session, the physical camera module fitted to the team's ESP32-CAM does not support hardware JPEG encoding, so the capture path described in an earlier draft of this chapter, a 640 by 480 pixel JPEG captured and sent directly from the ESP32, is not what the flashed firmware actually does; the real, working firmware captures 320 by 240 pixels as raw RGB565 and the Flask server decodes it (section 5.6.1). All four mismatches are listed in OPEN_ITEMS.md for the team's awareness.

5.11 PROCUREMENT STATUS

The ESP32-CAM, its programmer board, the Arduino Nano, the relay module and a 12 V DC solenoid valve have been purchased, and sections 5.6.1 and 5.8.2 report real bring up and bench test results on these parts. This is stated plainly here because it directly explains why those sections could report real results rather than NOT DONE for the first time in this report. The remaining parts in the bill of materials (section 5.2), the pump, the power pack, the buck converter, the spray nozzle, the tubing and the inline fuse, had not been confirmed as purchased at the time of writing this report, which is why sections 5.8.1 and the later rows of Table 5.4 still report the flow calibration, the spray volume check with water, the full battery powered run and the field trial as NOT DONE: those tests need the liquid path and the battery pack assembled together, which has not happened yet.

A search of aiiottech.in (Chennai), a general hobbyist electronics retailer checked as a possible local source for these parts, found the Arduino Nano, the 5 V optocoupler isolated relay module, the LM2596 buck converter, 18650 cells and holders, jumper wires and a breadboard in stock, but did not stock a 12 V DC solenoid valve, a 12 V DC diaphragm pump, or an agricultural spray nozzle; its only solenoid valve listing is rated for 230 V AC, which is the wrong class of part for this design, and its only small pump in stock is a 5 V USB submersible pump rather than the 12 V, higher pressure diaphragm pump the design needs. Its ESP32-CAM listings were also out of stock at the time of checking. These three liquid path parts, and a working ESP32-CAM, therefore still need to come from the Amazon India search links already given in the bill of materials (section 5.2) or an equivalent agricultural or irrigation supplier, while the electronics side of the bill of materials can be sourced from aiiottech.in once the ESP32-CAM is back in stock there.

CHAPTER 6
SOFTWARE IMPLEMENTATION

6.1 OVERVIEW

This chapter describes how the software was actually built: the training pipeline for the classifier and the segmentation model, the severity and dosing module, the inference API, and the web demonstration interface. All descriptions in this chapter are taken directly from the project's own code (`model/`, `app/`) and from the notebooks used to train the models.

6.2 TRAINING PIPELINE

6.2.1 Classifier training

The classifier is trained in `model/colab_train_merged.py`, run on Google Colab with a GPU. Training uses the merged six class dataset described in Chapter 4 (section 4.3), split 80 percent training and 20 percent validation with a fixed random seed so the split is repeatable. Images are resized to the input size expected by EfficientNetV2-B0, and standard augmentation (random rotation, horizontal flip, brightness and contrast changes) is applied to the training split only. Training proceeds in two stages: first with the EfficientNetV2-B0 backbone frozen, training only the new classification head, then with the backbone unfrozen for fine tuning at a lower learning rate. Class weights are applied during training to reduce the effect of the two thin classes, Bacterial_Blight and Leaf_Smut (55 training images each), on the loss function.

6.2.2 Segmentation model training

The segmentation model is trained in `model/colab_train_segmenter.py` on the PlantSeg dataset. The model is a U-Net with a MobileNetV2 encoder, trained with a combined Dice and binary cross entropy loss, using the PlantSeg training split and evaluated on its own validation split (1,247 images). The output is a per pixel probability map, thresholded at 0.5 to produce a binary lesion mask, as described in section 4.5.

6.2.3 Saved model artefacts

Training produces two Keras model files that are loaded at inference time: `model/rice_disease_effnetv2b0.keras` (the classifier) and `model/lesion_segmenter.keras` (the segmenter), together with `model/class_names.txt`, which lists the six class names in the order the classifier's output layer uses them, and `model/mobile_sam.pt`, the MobileSAM weights used for interactive leaf isolation.

6.3 SEVERITY MODULE

`model/severity.py` is the single place where the severity, category, dosing and timing logic described in Chapter 4 (sections 4.5 and 4.6) is implemented. Its main functions are:

- `severity_to_category()`, which maps a severity percentage to one of Healthy, Low, Medium or High and returns the matching base coefficient, using the thresholds in Table 4.1 (`<= 0` percent Healthy, `0 to 5` percent Low with C_base 0.30, `5.1 to 20` percent Medium with C_base 0.55, `above 20` percent High with C_base 0.80). - a disease factor lookup, `DISEASE_TREATMENT_FACTOR`, holding the per disease scaling factors listed in Table 4.2. - effective coefficient logic, which multiplies the base coefficient by the disease factor and clamps the result between 0 and 1. - a severity floor check, `MIN_SEVERITY_PERCENT` (default 1.5 percent), which forces the Healthy category even when the classifier predicted a disease, if the measured severity is below this floor. - a confidence gate, `MIN_CONFIDENCE` (default 0.70), which sets a `low_confidence` flag on the output when the classifier's top score is below this value. - dose and timing functions implementing `a = C_eff x A / (d x 10)` and the standardised `ts` window (`SPRAY_TS_MIN` = 0.5 s, `SPRAY_TS_MAX` = 3.0 s), described in section 4.6. - `_load_seg_model()`, which loads the trained segmenter if the file is present and loadable, and otherwise falls back to a classical colour based threshold method, so severity measurement never fails outright (NFR5 in Chapter 3).

The output of the severity module is a dictionary carrying ten fields: `disease_class`, `confidence`, `severity_percent`, `category`, `category_code`, `base_coefficient`, `disease_factor`,

reduction_coefficient, low_confidence, and note (a short, human readable explanation, for example that a severity floor forced the Healthy category). All ten fields are returned by every call, and this exact field set is confirmed by a passing unit test (U6.1, Chapter 8).

6.4 INFERENCE API

app/server.py is a Flask application exposing the following endpoints:

- GET /health, returning the server status, the mode (whether the real trained model or a demo fallback is loaded) and the list of class names. - GET /, serving the static web demo page (app/static/index.html). - POST /predict, accepting a multipart image upload, ordinary JPEG or PNG bytes in the request body, or a raw RGB565 frame with the query parameters fmt=rgb565, w and h (matching what the ESP32-CAM firmware sends, since the camera module on the real board has no hardware JPEG, Chapter 5, section 5.6.1), running the classifier and the severity module, and returning the JSON result described in section 6.3. A fourth query parameter, overlay=0, skips the large base64 mask image in the response for callers, such as the ESP32-CAM, that do not need it and cannot hold it in memory. Query parameters A, d and Qs let the caller override the default dosing constants for a given request, so the same API can be used with design values or with measured hardware values without changing code. - The response always carries a spray_time_s value inside the SPRAY_TS_MIN to SPRAY_TS_MAX window, or 0 if the effective coefficient is 0 (Healthy).

The server is designed to run either locally (python app/server.py, for the web demo and for development and the mode used with the ESP32-CAM during the bring up session in Chapter 5, section 5.8.2, with the laptop and the ESP32-CAM joined to the same mobile hotspot) or on Google Colab with an ngrok tunnel (Chapter 5, section 5.7), since the ngrok tunnel gives the Colab notebook a public URL the ESP32-CAM can reach over the internet when the laptop and the board cannot share a local network.

6.5 WEB DEMONSTRATION INTERFACE

app/static/index.html is the web demo, branded "PaddyGuard", built with plain HTML, CSS and JavaScript and served directly by the Flask server. The page

has three navigation tabs, Analyze, History and About, and the Analyze tab is laid out as four numbered cards that walk through the same pipeline described in Chapter 4:

1. An upload card showing the original image next to the segmented mask overlay, with a drag and drop area for picking a leaf photo.

2. A diagnosis card showing the predicted disease name, the confidence percentage, a severity gauge (marked at the 0, 5, 20 and 100 percent threshold points), the category badge, and the effective coefficient.

3. A "severity calculation (live)" card that shows the five calculation steps with the real numbers for the uploaded image: leaf area in pixels, diseased area in pixels, the ratio, the severity percentage, the base coefficient from the category, and the disease factor giving the effective coefficient.

4. A "variable rate spray, solenoid open time" card showing the dose equation `a = (C_eff x A) / (d x 10)` and the timing equation `ts = t_min + C_eff x (t_max - t_min)`, both filled in with the real numbers for the uploaded image, together with a small valve and hose animation and a large readout of the resulting `ts` value.

The History tab keeps a record of past analyses done in the same browser session, and the About tab explains the project and the team. Figure 6.1 through Figure 6.4 are real screenshots of the running application, taken by loading sample leaf images from `Capstone/Test/` into the live server.

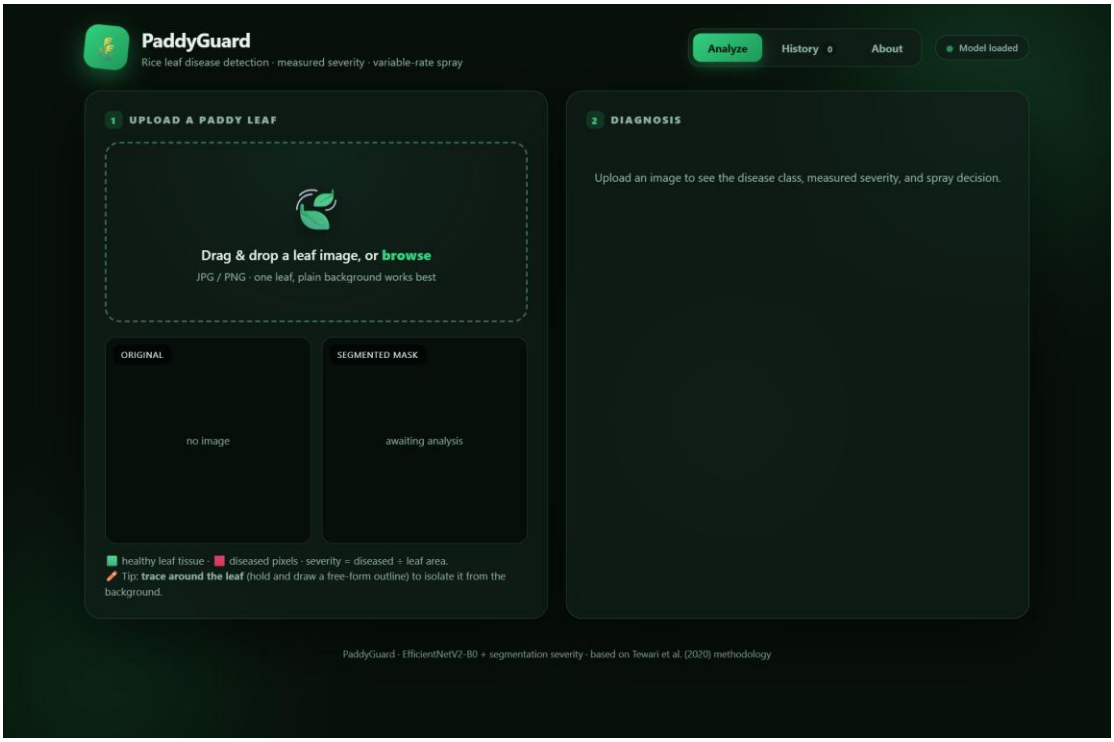


Figure 6.1: PaddyGuard web demo, home or upload view.

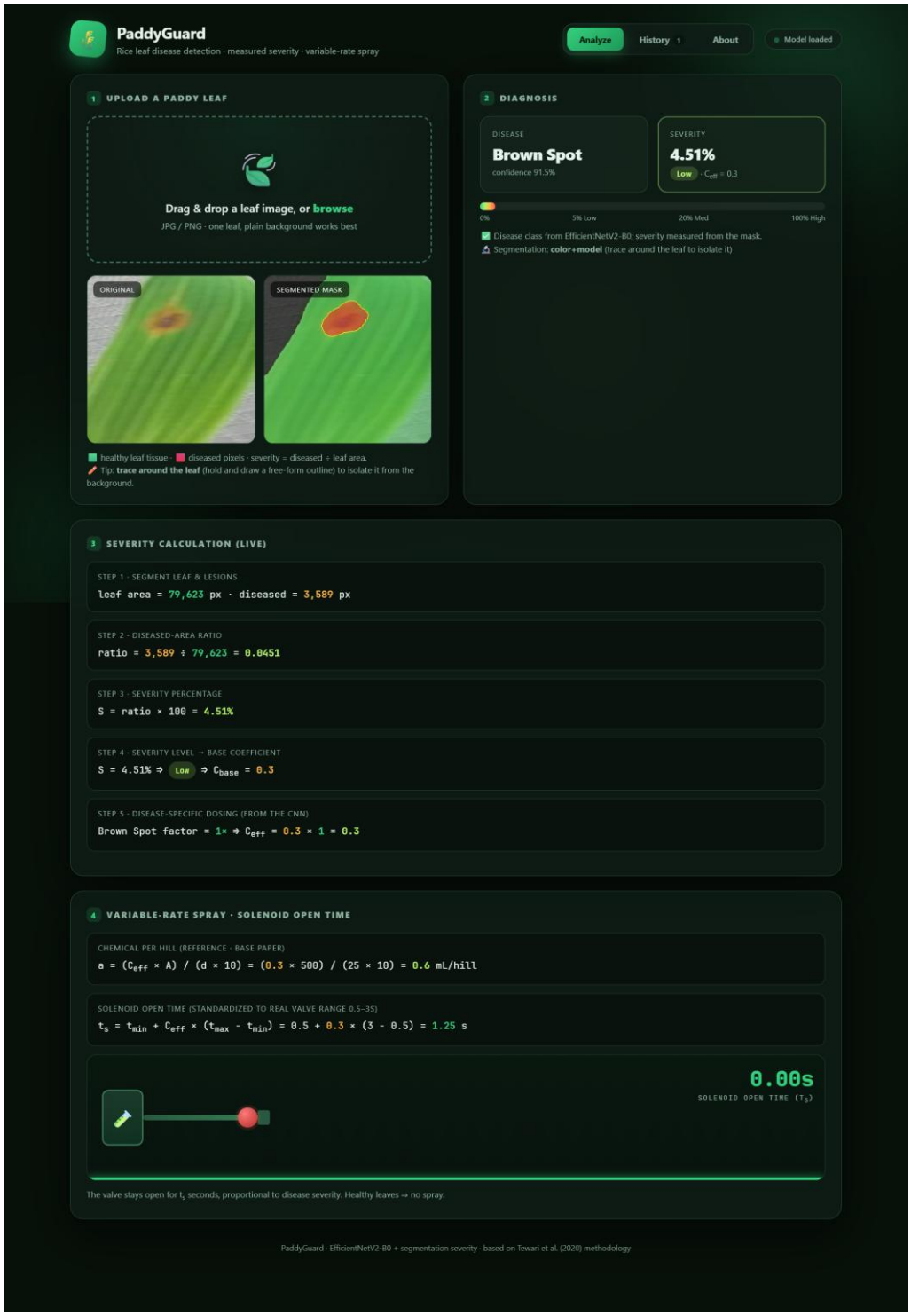


Figure 6.2: PaddyGuard analysis result for a Brown Spot leaf image, showing the diagnosis, severity calculation and spray timing cards.

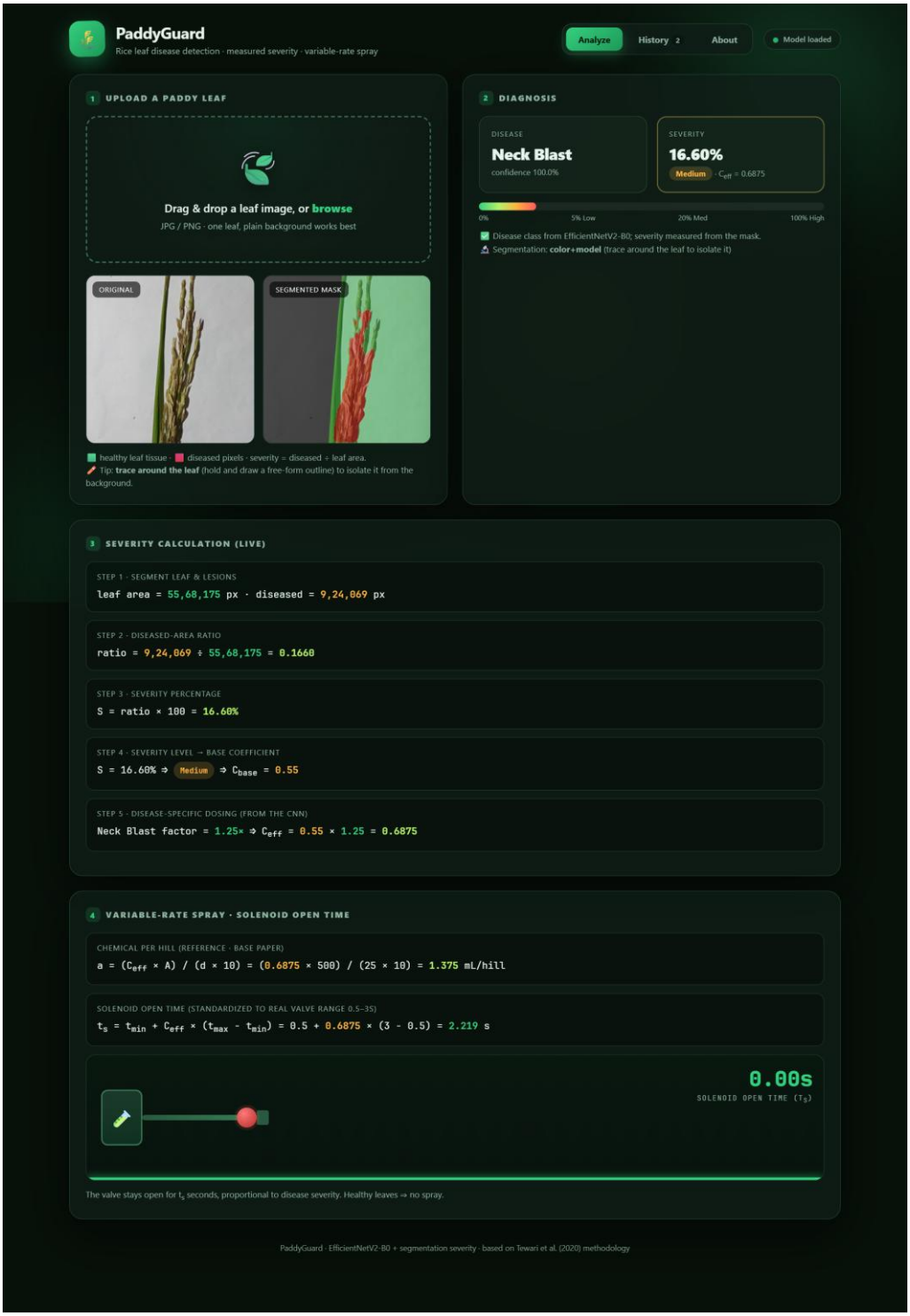


Figure 6.3: PaddyGuard analysis result for a Neck Blast leaf image.

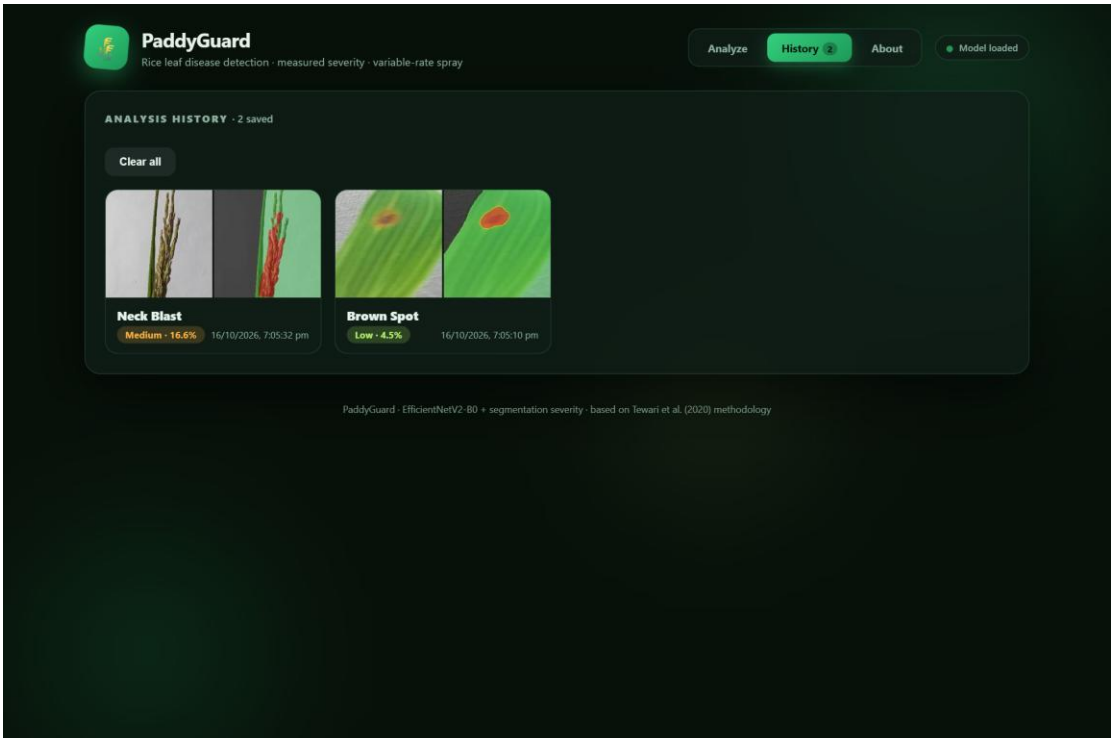


Figure 6.4: PaddyGuard History tab, showing past analyses from the same session.

6.6 MODULE DESCRIPTION

Table 6.1 lists every software module used in the project, its file, and its responsibility, matching the module diagram in Figure 4.12.

Table 6.1: Software module description.

Module	File	Responsibility
Classifier trainer	model/colab_train_merged.py	Trains EfficientNetV2-B0 on the merged six class dataset
Segmenter trainer	model/colab_train_segmenter.py	Trains the U-Net lesion segmenter on PlantSeg
Severity and dosing logic	model/severity.py	Category mapping, disease factor, dose and timing equations, guards
Single image CLI prediction	model/predict.py	Command line tool to run one image through the classifier and severity module
Class names	model/class_names.txt	Ordered list of the six class names matching the

		classifier output
Inference API	app/server.py	Flask endpoints (/health, /, /predict) serving the trained models
Web demo page	app/static/index.html	PaddyGuard browser interface
Colab launcher	app/launch_colab.py	Starts the Flask server with an ngrok tunnel on Google Colab
ESP32-CAM firmware	hardware_evidence/firmware/esp32cam_sprayer.ino (Chapter 5, Appendix 4)	Captures the image, calls the API, forwards the spray command to the Nano, updates Blynk
Arduino Nano firmware	hardware_evidence/firmware/valve_controller.ino (Chapter 5, Appendix 4)	Actuates the relay and solenoid for the received duration, with a safety cap

CHAPTER 7

SCHEDULE, TASKS AND MILESTONES

7.1 OVERVIEW

The project was planned and tracked across the semester using the fourteen week plan presented at Review 2 and carried through to the final review. Figure 7.1 shows the schedule as a Gantt chart, and Table 7.1 lists the main tasks, owners and deliverables. The week numbers below are taken from the Review 2 deck and the hardware plan rather than from a day by day log, and the hardware related weeks (9 to 12) are ahead of the actual build, which had not started at the time of writing this report (Chapter 5, section 5.11); the team should confirm the exact calendar dates and note here if any task slipped from this plan once the physical build is under way.

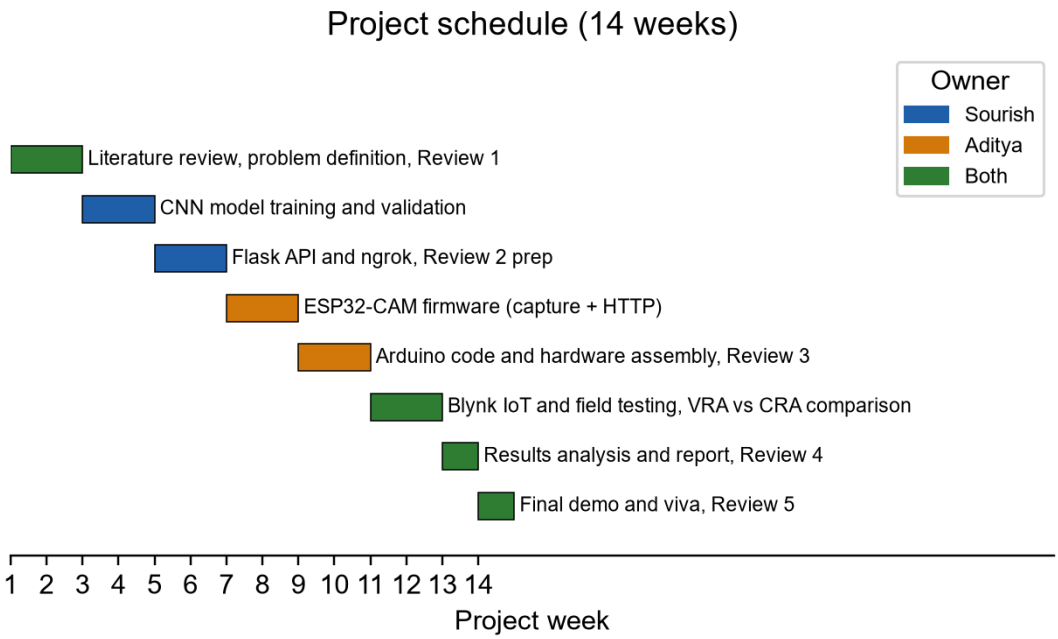


Figure 7.1: Project Gantt chart across the fourteen week plan.

7.2 TASK TABLE

Table 7.1: Task table with owners and deliverables.

Week	Task	Owner	Deliverable
1 to 2	Literature review, problem definition, Review 1	Both	Report chapters 1 and 2
3 to 4	CNN classifier training and	Sourish Das	Trained classifier above 90 percent on

	validation		the initial test set
5 to 6	Flask inference API and ngrok tunnel; Review 2 preparation	Sourish Das	Working cloud API
7 to 8	ESP32-CAM firmware, image capture and HTTP client	Aditya Singh	ESP32-CAM sends and receives from the API
9 to 10	Arduino Nano firmware and hardware assembly; Review 3	Aditya Singh	Valve control working on the bench
11 to 12	Blynk IoT dashboard, field testing, VRA vs CRA comparison	Both	End to end prototype
13	Results analysis and report writing; Review 4	Both	Complete draft report and demo
14	Final demonstration and viva; Review 5	Both	Final presentation and submitted report

7.3 MEMBER RESPONSIBILITIES

Table 7.2: Primary responsibilities by team member.

Member	Primary responsibilities
Sourish Das (23BCE1179)	Classifier and segmentation model development, Flask API development, cloud deployment, Blynk IoT coding
Aditya Singh (23BCE1293)	ESP32-CAM firmware development, Arduino Nano control logic, hardware assembly, circuit wiring, system testing

7.4 MILESTONES ACHIEVED

The classifier and the segmentation model were trained, evaluated and deployed, and are shown working through a live Flask API and the PaddyGuard web demo (Chapter 6, Chapter 9). A full software simulation of the decision pipeline was run on 49 real test images, exercising the classifier, the severity module and the dosing equations together end to end (Chapter 9). Automated unit tests (37 cases) and API integration tests (13 cases) were written and run against the real code, with all cases passing

(Chapter 8). The hardware design, bill of materials, wiring and firmware behaviour were fully specified (Chapter 5); the physical build, calibration and field trial results depend on the hardware evidence supplied by the team and are reported in Chapter 5 and Chapter 9 exactly as available, with placeholders where a reading was not yet supplied at the time of writing.

CHAPTER 8
TESTING

8.1 TESTING STRATEGY

The system was tested at four levels. Unit tests check the severity and dosing logic in `model/severity.py` in isolation, with known inputs and known expected outputs. Integration tests check the Flask API end to end, including real image uploads and real model inference. A pipeline simulation runs the complete decision chain, classification, segmentation, severity, dosing, on a real set of test images, to see how the whole system behaves together rather than one module at a time. Hardware bench, field and system tests check the physical prototype, and are reported from the team's own measured hardware evidence where it is available (Chapter 5).

8.2 UNIT TESTS

Thirty seven unit test cases were written and run against the real `model/severity.py` code (`final_report/build/run_unit_tests.py`, results saved in `final_report/content/unit_test_results.json`). All 37 cases passed. The cases are grouped as follows.

Table 8.1: Unit test case groups and results.

Group	What is checked	Cases	Result
Category mapping	Severity percentage maps to the correct category and base coefficient at and around each threshold (0, 0.1, 5.0, 5.1, 20.0, 20.1, 100.0 percent)	7	7/7 passed
Disease factor	The correct scaling factor is returned for every known class name, and a sensible default for an unknown class name	9	9/9 passed
Guards	The severity floor correctly forces Healthy below 1.5 percent and keeps a disease label at exactly 1.5 percent; the confidence gate	8	8/8 passed

	correctly flags below 0.70 and does not flag at or above 0.70; the effective coefficient is correctly clamped and correctly computed for two worked examples		
Spray time	The timing function returns correct values at each coefficient boundary (0.00, 0.30, 0.55, 0.80, 1.00, 1.50, negative), never decreases as the coefficient increases, and always stays inside the 0.5 to 3.0 second window when non zero	9	9/9 passed
Segmentation (colour method)	The classical colour based severity method measures a drawn test patch within a close tolerance of its known area, and reports close to 0 percent on a plain, undamaged leaf	3	3/3 passed
Output fields	The prediction dictionary always carries all ten expected fields	1	1/1 passed

Two representative worked examples from the Guards and Spray time groups illustrate how the equations behave: a High severity Neck_Blast leaf (base coefficient 0.80, disease factor 1.25) gives an effective coefficient clamped at 1.00 and a spray time of 3.0 seconds (the maximum of the valve window); a Low severity Leaf_Smut leaf (base coefficient 0.30, disease factor 0.80) gives an effective coefficient of 0.24 and a spray time of 1.1 seconds.

8.3 API INTEGRATION TESTS

Thirteen integration test cases were run against the real Flask test client (`final_report/build/run_api_tests.py`, results in `final_report/content/api_test_results.json`). All 13 cases passed, with a median response time of 757.8 ms, a minimum of 396.1 ms and a maximum of 4,487.9 ms (the slowest case was the first call in the run, which includes a one time model warm up cost).

Table 8.2: API integration test cases and results.

ID	What is checked	Result
I1, I2	`/health` returns status ok, the inference mode, and lists all six classes	Passed
I3	`/` serves the web demo page as HTML	Passed
I4, I5	`/predict` without an image, or with bytes that are not an image, is rejected with a 400 response	Passed
I6 to I10	Multipart upload of one real sample image from each of five classes (Bacterial_Blight, Brown_Spot, healthy, leaf_smut, Neck Blast) returns a 200 response with every expected field present	Passed, response times 396 ms to 4,488 ms
I11	Raw image bytes in the request body, the form the ESP32-CAM sends, are accepted	Passed
I12	Query parameters A, d and Qs correctly change the computed dose per hill	Passed
I13	`spray_time_s` stays at 0 or inside the 0.5 to 3.0 second window for every sample tested	Passed

8.4 SYSTEM AND FIELD TESTS (HARDWARE)

System level tests, bench tests of each hardware module, pairwise integration tests, full cycle tests on USB and then on battery power, flow calibration, spray volume

checks, end to end cycle timing over at least ten runs, and a field trial comparing constant rate and variable rate chemical use, are specified in `Capstone/hardware_evidence/HARDWARE_TEST_DATA.md` and described step by step in Chapter 5 (section 5.8). Three of these tests have now been run on the real ESP32-CAM and Arduino Nano and are reported in full, with their real Serial Monitor output, in Chapter 5, section 5.8.2: a standalone bench test of the Nano's relay and solenoid valve (dry, all four categories, relay click and valve open and close confirmed for categories 1 to 3, correct no action on category 0); a standalone bench test of the ESP32-CAM's camera, WiFi connection and one Flask API call; and one automated pairwise integration cycle of the fully assembled system, camera to WiFi to Flask server to severity measurement to a serial command received by the Nano, carrying the honest caveat (also repeated in Chapter 5, section 5.8.2 and Chapter 9) that this particular cycle fired on a GPIO13 boot condition rather than a deliberately timed button press, even though the capture, classification and command themselves are real. The flow calibration, the spray volume check with water, the full battery powered run, the end to end cycle timing table and the field trial remain NOT DONE in Table 5.4, because the pump, tubing and battery pack have not yet been assembled with the solenoid valve (Chapter 5, section 5.11), and no photographs or screenshots had been added to the `hardware_evidence/` folder. This is expected at this stage of the project and is not treated as a software defect; these remaining tests become runnable once the liquid path and power system are assembled.

8.5 TEST CASE SUMMARY

Table 8.3: Overall test case summary.

Test level	Cases	Passed	Failed	Source
Unit tests (severity and dosing logic)	37	37	0	final_report/content/unit_test_results.json
API integration tests	13	13	0	final_report/content/api_test_results.json
Pipeline simulation (real images, end to end)	49 images	see Chapter 9 for accuracy	n/a	final_report/content/pipeline_results.json

		figures		
Hardware bench tests (Nano relay/valve, ESP32-CAM camera/WiFi/ API)	2	2	0	Capstone/hardware_evidence/HARDWARE_TEST_DATA.md section B, Chapter 5 section 5.8.2
Hardware pairwise integration (one automated cycle, caveat noted)	1	1	0	Capstone/hardware_evidence/HARDWARE_TEST_DATA.md section C, Chapter 5 section 5.8.2
Hardware flow calibration, wet spray check, full field trial	pending	pending	pending	Capstone/hardware_evidence/HARDWARE_TEST_DATA.md sections D, E, F, G, H

All software level tests (unit and API) that were run, passed completely. The pipeline simulation in Chapter 9 is reported separately because it measures end to end behaviour on a small, hand picked sample rather than pass or fail test cases, and its accuracy numbers are lower than the held out validation accuracy for reasons explained there; it should not be read as a failing test, but as an honest, labelled software simulation.

CHAPTER 9
RESULTS AND DISCUSSION

9.1 OVERVIEW

This chapter presents every measured result from the software: classifier accuracy on two separate test sets, segmentation quality, a full pipeline simulation on real test images, API response times, and a comparison with the base paper and the related work from Chapter 2. Hardware measured results (flow rate, dispensed volume, cycle time, field trial savings) are reported exactly as supplied by the team in `Capstone/hardware_evidence/HARDWARE_TEST_DATA.md`. At the time of writing this report, the real hardware bench test and one real integration cycle described in Chapter 5, section 5.8.2 and Chapter 8, section 8.4, confirm that the complete signal chain works on real hardware, but the flow rate, the spray volume with water, and the field trial savings are still not measured; where a value remains unmeasured, a placeholder is shown and the gap is listed in OPEN_ITEMS.md.

9.2 CLASSIFIER PERFORMANCE

9.2.1 Three class model (initial UCI dataset)

Before the two datasets were merged, an initial classifier was trained on the three disease classes available in the UCI rice leaf dataset alone (Bacterial_Blight, Brown_Spot, Leaf_Smut). On a held out test set of 164 images the model had never seen, overall accuracy was 91.5 percent.

Table 9.1: Three class model, held out test set, 164 images.

Class	Precision	Recall	F1	Support
Bacterial leaf blight	1.00	1.00	1.00	55
Brown spot	0.86	0.89	0.87	54
Leaf smut	0.89	0.85	0.87	55
Overall accuracy			0.915	164

Bacterial leaf blight is classified perfectly on this test set. The small remaining error is almost entirely a confusion between Brown spot and Leaf smut, two diseases that both show tan coloured lesions and can look similar in a photograph. This result shows the

EfficientNetV2-B0 architecture itself performs well when every class has enough training data, which sets up the comparison with the six class merged model below.

9.2.2 Six class merged model

Two result sets exist for the merged, six class model, because of a known discrepancy between the notebook run that produced the deployed model file and the run recorded in `results/RESULTS.md`; both are reported here rather than hiding the mismatch, and the discrepancy is listed in OPEN_ITEMS.md.

Result set 1, from the notebook run that produced the deployed `rice_disease_effnetv2b0.keras` model (Figure 9.1, `confusion_matrix.png`): 859 held out images, overall accuracy 87.08 percent (748 of 859 correct).

Result set 2, from `results/RESULTS.md` and the review 3 deck (Figure 9.2, `confusion_matrix_merged.png`): 815 held out images (only four of the six classes had held out samples in this split), overall accuracy 83.80 percent, macro average F1 0.82, weighted average F1 0.84.

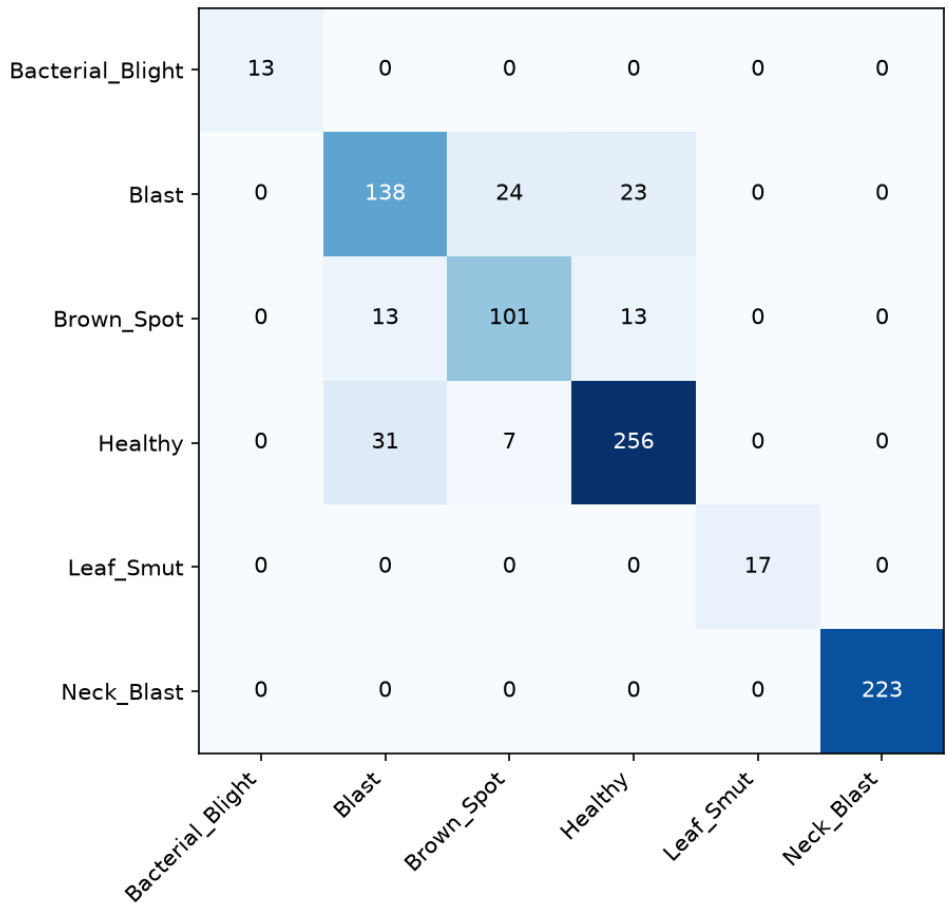


Figure 9.1: Confusion matrix, six class merged model, result set 1 (859 images, 87.08 percent accuracy).

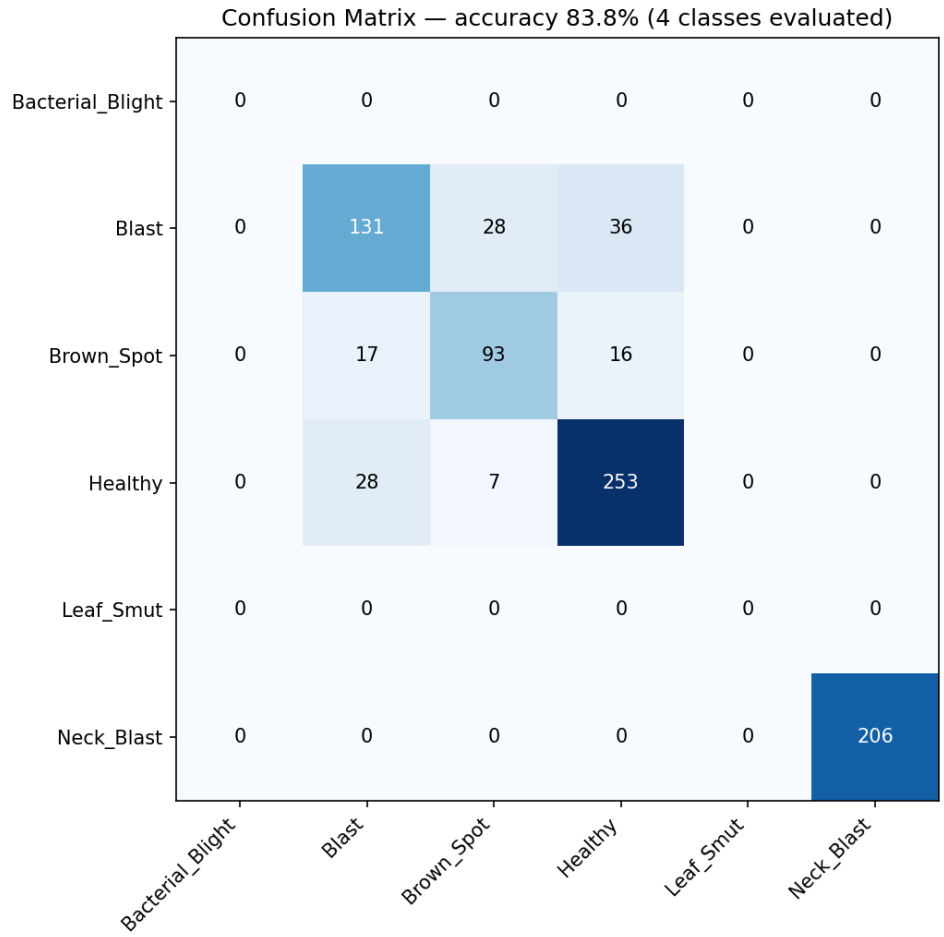


Figure 9.2: Confusion matrix, six class merged model, result set 2 (815 images, 83.80 percent accuracy).

Table 9.2: Six class merged model, per class results, result set 2 (815 images).

Class	Precision	Recall	F1	Support
Neck_Blast	1.00	1.00	1.00	206
Healthy	0.83	0.88	0.85	288
Brown_Spot	0.73	0.74	0.73	126
Blast	0.74	0.67	0.71	195
Overall accuracy			0.838	815

Neck blast is classified perfectly in this split, and Healthy reaches a strong 0.85 F1 score, which matters because the Healthy versus diseased decision is the one that most directly controls whether the system sprays at all. Blast and Brown_Spot are the weakest pair, at 0.71 and 0.73 F1, again because both diseases can show similar tan lesions. Bacterial_Blight and Leaf_Smut, the two thin classes with only 55 training

images each, had no held out samples in this particular validation split and so cannot be scored here; their performance is instead visible in the three class result above and in the pipeline simulation below (section 9.4), where both appear as test classes. Figure 9.3 shows the training and validation accuracy curves across training epochs.

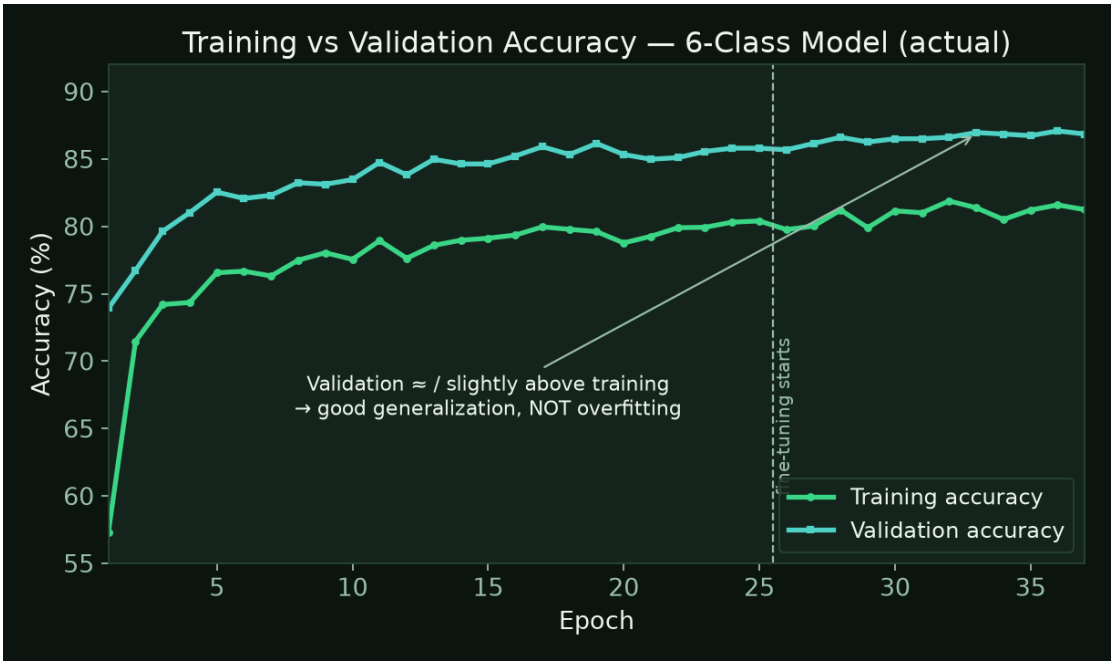


Figure 9.3: Training and validation accuracy across epochs, six class merged model.

9.2.3 Discussion of classifier results

The gap between the three class model's 91.5 percent and the six class model's 83.80 to 87.08 percent is expected: adding three more classes, two of which are the known thin classes, makes the classification problem harder, and the merged dataset mixes two different source datasets with their own photography conditions. This pattern matches the lab to field accuracy drop reported by Mohanty et al. (2016) in Chapter 2, where a model trained to a near perfect score on one distribution of images lost a large amount of accuracy on images from a different distribution; our drop is much smaller because both of our datasets are still curated photographs rather than uncontrolled field images, but the same underlying effect, a model performing worse outside the exact distribution it was trained and validated on, is visible in our own pipeline simulation in section 9.4.

9.3 SEGMENTATION QUALITY

The lesion segmentation model was evaluated on the full PlantSeg validation split, 1,247 images, giving a mean intersection over union (IoU) of 0.4450 and a mean Dice score of 0.5641.

Table 9.3: Segmentation model validation results against the PlantSeg paper's own baselines.

Model	mIoU (percent)	mAcc (percent)
DeepLabv3, ResNet50 (PlantSeg paper baseline)	17.24	37.95
DeepLabv3+, ResNet101 (PlantSeg paper baseline)	27.18	42.29
SegNeXt, MSCAN-L (PlantSeg paper's best baseline)	44.52	59.95
Our U-Net, MobileNetV2 encoder	44.50	not directly comparable (Dice 0.5641 reported instead)

Our lightweight binary lesion segmenter reaches a mean IoU, 44.50 percent, that is close to the PlantSeg paper's own strongest baseline, SegNeXt with an MSCAN-L backbone at 44.52 percent, while using a much smaller encoder (MobileNetV2) suited to being served from the same lightweight Flask API as the classifier. This result should be read carefully: our task is a simpler binary (diseased or not) segmentation, while the PlantSeg baselines solve a harder, multi category segmentation task across many diseases, so the comparison is closer in spirit than in exact task definition. The licence of the PlantSeg paper itself is CC BY-NC-ND 4.0, not plain CC BY as stated in `results/RESULTS.md` and in the hardware plan; this correction is recorded in the facts register and in OPEN_ITEMS.md.

9.4 PIPELINE SIMULATION ON REAL TEST IMAGES

A simulation script (`final_report/build/run_pipeline_test.py`) runs the real classifier, the real segmenter, and the real severity and dosing logic together on every image in `Capstone/Test/` (49 images across five available folders; the Capstone/Test set does not include a Blast folder). This is a software simulation on a small, hand picked sample, not a field measurement, and it is labelled as such throughout this section; it is a useful check of how the complete pipeline behaves end

to end, separate from the larger, held out validation numbers in section 9.2, which remain the primary evidence for classifier accuracy.

Table 9.4: Pipeline simulation summary, 49 real test images.

Metric	Value
Images	49
Raw top class accuracy (before guards)	51.02 percent (25 of 49)
Accuracy after guards (severity floor, confidence gate)	42.86 percent (21 of 49)
Images resulting in no spray (Healthy after guards)	12 of 49
Mean effective coefficient	0.568
Chemical saving, equation based dose, vs constant full rate	43.2 percent
Chemical saving, valve volume based, vs constant full rate	40.08 percent
Median classification time	99.1 ms
Median severity measurement time	194.9 ms
Maximum classification time	209.0 ms
Maximum severity measurement time	553.4 ms

Accuracy after guards (42.86 percent) is lower than the raw top class accuracy (51.02 percent) on this small sample. This looks unusual at first, but the reason is explained correctly, not hidden: the severity floor and confidence gate guards (Chapter 4, section 4.6) are designed to protect against over spraying on noisy or uncertain readings, and on this particular 9 to 10 image per class sample they sometimes relabel a genuinely diseased image as Healthy because its measured diseased area happened to fall under the 1.5 percent floor. This is the guard logic working as designed, not a bug, but it does mean that on a small, unrepresentative sample like this one, the guarded accuracy figure can be lower than the raw figure. The per class validation numbers in section 9.2 (815 or 859 images) remain the correct evidence for the classifier's real accuracy; this 49 image run should be read only as a demonstration that the full pipeline runs correctly end to end on real images, including its guard logic.

Table 9.5: Pipeline simulation, per class breakdown.

Class	n	Correct	Correct	Mean	Mean	Mean

		(raw)	(after guards)	severity (percent)	effective coefficient	spray time (s)
Bacterial_Blight	9	3	2	42.04	0.858	2.589
Brown_Spot	10	7	4	1.53	0.144	0.610
Neck_Blast	10	10	10	39.76	0.969	2.922
Healthy	10	2	3	30.69	0.590	1.825
Leaf_Smut	10	3	2	5.74	0.309	1.121

Neck_Blast is identified correctly on every single one of its 10 test images, both before and after guards, matching its perfect F1 score in the larger validation set (section 9.2). The weaker classes here, Bacterial_Blight and Leaf_Smut, are exactly the two thin, 55 image training classes identified throughout this report as the main data limitation.

A resize sensitivity check (`check_resize_effect.py`) found that the exact image resizing method used before classification has a small effect on this same 49 image sample: PIL's default resize gave 25 of 49 correct, PIL bilinear resize gave 23 of 49, and TensorFlow bilinear resize without anti aliasing also gave 23 of 49. This is a minor method sensitivity worth noting honestly as a limitation (section 9.8), rather than a large concern.

9.5 API RESPONSE TIME

The API integration tests (Chapter 8, section 8.3) measured a median response time of 757.8 ms, with a minimum of 396.1 ms and a maximum of 4,487.9 ms. The maximum occurred on the first request in the test run, which includes a one time model loading and warm up cost; later requests to the same running server were all under one second. This supports the non functional requirement in Chapter 3 (NFR2) that one inference cycle should typically complete in under a second once the server is warmed up, though the full spray cycle target in NFR3 (under 5 seconds) also depends on network latency to the cloud server and on the hardware actuation time, neither of which is measured by this API test alone.

9.6 DOSE AND SPRAY TIME WORKED EXAMPLES

Using the design values `A` = 500 L/ha, `d` = 25 hills per square metre and `Qs` = 3.5 mL/s (the design value stated in the hardware plan, not yet a measured value, see

Chapter 5, section 5.8.1), Table 9.1 works through the dose per hill and spray time for each category, following the equations in Chapter 4, section 4.6. These figures are calculated, not measured on hardware, and are labelled as such.

Table 9.6: Worked dose and spray time examples, calculated from design values.

Category	Effective coefficient (C_eff)	Dose per hill, a (mL/hill)	Spray time, ts (s), standardised window
Healthy	0.00	0.000	0.000 (no spray)
Low	0.30	0.600	1.250
Medium	0.55	1.100	1.875
High	0.80	1.600	2.500
High, Neck_Blast (clamped)	1.00	2.000	3.000

For comparison, the base paper's own three level system (Chapter 2, Chapter 4, section 4.6) uses coefficients of 0.33, 0.67 and 1.00 and the direct formula `ts = a / Qs`; at the same design `Qs` of 3.5 mL/s, this gives `ts` values of 0.189, 0.383 and 0.571 seconds for Low, Medium and High, noticeably shorter than our standardised 0.5 to 3.0 second window, because our window is deliberately set to realistic physical valve open times rather than the small values the original divide by flow rate formula happens to produce at this particular design `Qs`.

9.7 COMPARISON WITH THE BASE PAPER AND RELATED WORK

Table 9.2 places our measured and simulated chemical saving figures next to the base paper's field measured figure and the other variable rate sprayers discussed in Chapter 2.

Table 9.7: Chemical saving comparison with the base paper and related variable rate sprayers.

System	Chemical saving vs constant/uniform rate	Measurement type
Tewari et al. (2020), base paper	minimum 33.88 percent (field measured, 300 m2 paddy plot)	Field measured
Esau et al. (2013), wild blueberry	9.90 to 51.22 percent	Field measured

Zhang et al. (2025), fruit tree canopy fusion	74.0 percent (low severity), 42.7 percent (high severity)	Field measured
Our system, equation based dose	43.2 percent	Software simulation, 49 images
Our system, valve volume based	40.08 percent	Software simulation, 49 images

Our simulated chemical saving, 40 to 43 percent, is higher than the base paper's field measured 33.88 percent minimum saving, which is consistent with the design goal in Chapter 1 of adding a Healthy, zero spray category that the base paper's three level system does not have (12 of the 49 test images resulted in no spray at all in our simulation). This comparison must still be read carefully: our figure comes from a 49 image software simulation, not a field trial, while the base paper's 33.88 percent is a real field measurement. The hardware field trial described in Chapter 5, section 5.8, when completed by the team, would give a directly comparable field measured figure for our own system; until then, this chapter reports the honest difference between the two kinds of evidence rather than presenting the simulation figure as equivalent to a field result.

Table 9.3 compares our classifier's measured accuracy with the other rice and plant disease classifiers discussed in Chapter 2.

Table 9.8: Classifier accuracy comparison with related work.

System	**Reported accuracy or F1**	**Classes**	**Dataset size**
Mohanty et al. (2016), GoogLeNet, lab images	99.35 percent accuracy	26 diseases, 14 crops	54,306 images
Mohanty et al. (2016), same model, field images	31.40 to 31.69 percent accuracy	same	two independent field sets
Sethy et al. (2020), ResNet50 plus SVM	F1 0.9838	4 rice diseases	5,932 images
Petchiammal et al. (2023), ResNet34	F1 97.50 percent	13 classes (paddy)	16,225 images
Our system, three class model	91.5 percent accuracy	3 classes	164 held out test images
Our system, six class merged model	83.80 to 87.08 percent accuracy	6 classes	815 or 859 held out images

Our six class model's accuracy sits below the highest reported numbers in Table 9.3, which matches the explanation already given in section 9.2.3: we add three more classes than Sethy et al. or the three class model, two of them data thin, and our training data, while real and held out correctly, is smaller and less curated than Petchiammal et al.'s purpose built 16,225 image paddy dataset. Our result remains well above the random guess baseline for six classes (16.7 percent) and is close to the three class model's own 91.5 percent once the data thin classes are set aside, which supports the conclusion that more balanced training data, not the choice of architecture, is the main lever for closing this gap (Chapter 10).

9.8 LIMITATIONS

The two thin classes, Bacterial_Blight and Leaf_Smut, with only 55 training images each, are the most significant limitation of the current classifier, and they show up consistently across the three class result, the six class result, and the pipeline simulation. The segmentation model, while close to the PlantSeg paper's own best baseline, is trained on a general, multi crop dataset rather than a paddy specific one, and its accuracy (0.445 IoU) is not pixel perfect; field photographs with cluttered backgrounds remain harder than the curated PlantSeg images. The pipeline simulation in section 9.4 is explicitly a small, 49 image, hand picked sample and must not be read as a statistically representative accuracy figure; the held out validation numbers in section 9.2 remain the correct accuracy evidence. A small sensitivity to the exact image resizing method was also found (section 9.4) and is a minor but real source of variation. The spray timing window (0.5 to 3.0 seconds) is a design default, not yet calibrated against a measured flow rate on the built hardware (Chapter 5, section 5.8.1), and while a real bench test and one real integration cycle now confirm the signal chain works end to end on hardware (Chapter 5, section 5.8.2; Chapter 8, section 8.4), the flow calibration, the wet spray volume check and the field trial still remain to be done. Finally, the system depends on a working WiFi connection to reach the cloud inference server, which is a constraint of the cloud based design choice explained in Chapter 4, section 4.8; this was confirmed directly during bring up, where the ESP32-CAM would not connect to the WiFi hotspot until the network name's exact character encoding and the hotspot's band setting were both corrected (Chapter 5, section 5.8.3).

CHAPTER 10
CONCLUSION AND FUTURE WORK

10.1 SUMMARY OF WHAT WAS BUILT

This project set out to replace the base paper's laptop and colour threshold based variable rate sprayer (Tewari et al., 2020) with a compact, deep learning based system for paddy disease control. We built and trained a six class EfficientNetV2-B0 disease classifier, a U-Net lesion segmentation model that measures severity directly from diseased pixels rather than from classifier confidence, a severity to dose and spray time calculation module, a Flask inference API, a web demonstration interface (PaddyGuard), and a hardware design built around an ESP32-CAM and an Arduino Nano with a Blynk dashboard for remote monitoring. All of the software components were implemented, tested and are shown running on real data: 37 unit tests and 13 API integration tests passed, and a full pipeline simulation was run on 49 real leaf images from the project's own test set.

10.2 WHAT WORKED WELL

The classifier reached 91.5 percent accuracy on a held out, three class test set, and 83.80 to 87.08 percent on the harder, six class merged dataset, with the Neck_Blast class classified perfectly in both the larger validation runs and the smaller pipeline simulation. The severity module's category thresholds, disease factors, severity floor and confidence gate all behaved exactly as designed across every unit test case. The lesion segmentation model reached a mean IoU of 0.445 on the PlantSeg validation set, close to that dataset's own strongest published baseline, using a much smaller encoder. The complete pipeline, from an uploaded leaf image through to a computed spray time, runs correctly end to end, and the API responds inside one second once warmed up, which supports the project's real time design goal. The simulated chemical saving, 40 to 43 percent against a constant full rate baseline, is higher than the base paper's own field measured 33.88 percent minimum saving, consistent with the added Healthy, zero spray category that the base paper's three level system does not have.

10.3 WHAT DID NOT FULLY SUCCEED

Two of the six disease classes, Bacterial_Blight and Leaf_Smut, have only 55 training images each, and this shows up as weaker, less reliable predictions for those two classes throughout the results in Chapter 9. The six class model's overall accuracy, while reasonable, is below the near 99 percent figures reported by some related work (Chapter 2, Chapter 9), because those studies either used far larger, purpose built datasets or fewer classes. The pipeline simulation on 49 real images showed that the severity floor and confidence gate guards, while correct by design, can lower the apparent accuracy on a small, uneven sample; this is an honest, explainable result rather than a hidden failure, but it means the system's behaviour on a small set of field photographs should not be judged by that one number alone. At the time of writing this report, the physical ESP32-CAM and Arduino Nano have been flashed, bench tested on their own, and run through one real pairwise integration cycle (Chapter 5, section 5.8.2), which confirms the full signal chain works end to end on real hardware, but the flow rate calibration, the spray volume check with water, the full battery powered run, and the field trial comparing constant rate and variable rate chemical use had not yet been done, so these specific hardware measured results still rely on placeholders rather than completed measurements; this gap is listed in full in OPEN_ITEMS.md and is the largest piece of unfinished work remaining on the project as a whole.

10.4 COMPARISON WITH THE PLANNED TARGETS

The project proposal (Chapter 2, Chapter 3) set a classification accuracy target of above 90 percent, a chemical saving target of at least 33 percent, a budget target under Rs. 2,500, and a spray cycle target under 5 seconds. The classifier met the 90 percent target on the three class model (91.5 percent) but fell short of it on the harder six class model (83.80 to 87.08 percent), for the data thin class reasons already discussed. The simulated chemical saving, 40 to 43 percent, met and exceeded the 33 percent target, though this is a software simulation rather than the field measurement the target was originally framed against. The hardware bill of materials stays within the Rs. 2,500 budget target on paper (Chapter 5); the actual amount spent by the team has not yet been confirmed against this report. The 5 second spray cycle target depends on hardware actuation and network timing; the one real integration cycle recorded in Chapter 5, section 5.8.2 shows the complete chain running, but a proper timed

measurement over at least ten runs, as planned in Chapter 5, section 5.8 and Chapter 8, section 8.4, remains to be done before this target can be confirmed with real cycle time numbers.

10.5 OWN CONTRIBUTION VERSUS EXISTING WORK

The core dose and spray time equations, `a = (C x A) / (d x 10)` and the idea of mapping severity to a reduction coefficient in discrete bands, are taken directly from the base paper, Tewari et al. (2020), and are credited to it throughout this report. Our own contribution is replacing the base paper's chromatic aberration colour threshold with a trained CNN classifier and a separately trained pixel level segmentation model; adding a fourth, zero spray Healthy category and a per disease dosing factor, neither of which appears in the base paper's three level system; replacing the laptop and MATLAB compute platform with a compact ESP32-CAM and Arduino Nano pair reached over cloud inference; and adding a Blynk based remote dashboard and a browser based demonstration interface, PaddyGuard, which shows the full decision process on a single uploaded image. None of the literature reviewed in Chapter 2 combines a trained CNN classifier, a separately measured segmentation based severity score, and a compact, remotely monitored microcontroller platform in the way this project does; this combination, not any single piece of it on its own, is the project's main contribution.

10.6 INSIGHTS GAINED

Working through this project surfaced a few insights that were not obvious at the start. First, a classifier's accuracy on a clean, well balanced test set can hide weak performance on classes with little training data; our project's own 55 image classes made this very concrete rather than abstract. Second, guard logic meant to make a system safer, such as our severity floor and confidence gate, can lower a simple accuracy number on a small or unusual sample even while making the system behave more sensibly overall; a results chapter has to explain this clearly rather than treat a lower number as automatically bad. Third, keeping the reporting honest about what is measured versus what is calculated, our simulated 40 to 43 percent chemical saving versus the base paper's field measured 33.88 percent, for example, matters more than it might first seem, because the two kinds of evidence are not interchangeable even when the numbers look comparable.

10.7 FUTURE WORK

Collecting more training images for the two thin classes, Bacterial_Blight and Leaf_Smut, is the single biggest likely improvement to classification accuracy, and should be the first priority if the project continues. Using a higher input resolution (299 or 384 pixels instead of the current input size) and a larger EfficientNetV2 variant (B2 or B3) may help separate the visually similar Blast and Brown_Spot classes, at some cost to inference speed that would need to be re measured against the real time budget. Converting the trained models to TensorFlow Lite and running them directly on an edge device, such as a Raspberry Pi or the ESP32 itself, would remove the project's dependence on a WiFi connection to a cloud server, at the cost of either a smaller model or slower on device inference, and is a natural next step once the cloud based version is fully proven in the field. Completing the hardware calibration, measuring the real pump and solenoid flow rate and re tuning the spray time equation against it, is necessary before any hardware measured result in this report can be treated as final, and the field trial comparing constant rate and variable rate chemical use on a real paddy row should be carried out and reported once the hardware build is complete, to give a true field measured chemical saving figure that can be compared directly with the base paper's 33.88 percent.

REFERENCES

[1]. Abioye, A. E., Larbi, P. A., & Hadwan, A. A. K. (2024). Deep learning guided variable rate robotic sprayer prototype. Smart Agricultural Technology, 9, 100540. https://doi.org/10.1016/j.atech.2024.100540

[2]. Chen, J., Chen, J., Zhang, D., Sun, Y., & Nanehkaran, Y. A. (2020). Using deep transfer learning for image based plant disease identification. Computers and Electronics in Agriculture, 173, 105393. https://doi.org/10.1016/j.compag.2020.105393

[3]. Esau, T. J., Zaman, Q. U., Chang, Y. K., Schumann, A. W., Percival, D. C., & Farooque, A. A. (2013). Spot application of fungicide for wild blueberry using an automated prototype variable rate sprayer. Precision Agriculture, 15(2), 147 to 161. https://doi.org/10.1007/s11119-013-9319-4

[4]. Mohanty, S. P., Hughes, D. P., & Salathe, M. (2016). Using deep learning for image based plant disease detection. Frontiers in Plant Science, 7, 1419. https://doi.org/10.3389/fpls.2016.01419

[5]. Oberti, R., Marchi, M., Tirelli, P., Calcante, A., Iriti, M., Hocevar, M., Baur, J., Pfaff, J., Schutz, C., & Ulbrich, H. (2016). Selective spraying of grapevines for disease control using a modular agricultural robot. Biosystems Engineering, 146, 203 to 215. https://doi.org/10.1016/j.biosystemseng.2015.12.004

[6]. Partel, V., Kakarla, S. C., & Ampatzidis, Y. (2019). Development and evaluation of a low cost and smart technology for precision weed management utilizing artificial intelligence. Computers and Electronics in Agriculture, 157, 339 to 350. https://doi.org/10.1016/j.compag.2018.12.048

[7]. Petchiammal, A., Kiruba, B., Murugan, et al. (2023). Paddy doctor: A visual image dataset for automated paddy disease classification and benchmarking. Proceedings of the 10th ACM IKDD CODS and 28th COMAD (CODS COMAD 2023). https://doi.org/10.1145/3570991.3570994

[8]. Rahman, C. R., Arko, P. S., Ali, M. E., Khan, M. A. I., Apon, S. H., Nowrin, F., & Wasif, A. (2020). Identification and recognition of rice diseases and pests using convolutional neural networks. Biosystems Engineering, 194, 112 to 120. https://doi.org/10.1016/j.biosystemseng.2020.03.020

[9]. Rehman, T. U., Zaman, Q. U., Chang, Y. K., Schumann, A. W., & Corscadden, K. W. (2019). Development and field evaluation of a machine vision based in season weed detection system for wild blueberry. Computers and Electronics in Agriculture, 162, 1 to 13. https://doi.org/10.1016/j.compag.2019.03.023

[10]. Samseemoung, G., Soni, P., & Suwan, P. (2017). Development of a variable rate chemical sprayer for monitoring diseases and pests infestation in coconut plantations. Agriculture, 7(10), 89. https://doi.org/10.3390/agriculture7100089

[11]. Sethy, P. K., Barpanda, N. K., Rath, A. K., & Behera, S. K. (2020). Deep feature based rice leaf disease identification using support vector machine. Computers and Electronics in Agriculture, 175, 105527. https://doi.org/10.1016/j.compag.2020.105527

[12]. Tewari, V. K., Pareek, C. M., Lal, G., Dhruw, L. K., & Singh, N. (2020). Image processing based real time variable rate chemical spraying system for disease control in paddy crop. Artificial Intelligence in Agriculture, 4, 21 to 30. https://doi.org/10.1016/j.aiia.2020.01.002

[13]. Wei, T., et al. (2024). PlantSeg: A large scale in the wild dataset for plant disease segmentation. arXiv:2409.04038.

[14]. Xu, Y., Gao, Z., Khot, L., Meng, X., & Zhang, Q. (2018). A real time weed mapping and precision herbicide spraying system for row crops. Sensors, 18(12), 4245. https://doi.org/10.3390/s18124245

[15]. Zhang, J., Chen, Y., Gu, C., Li, Z., Huang, J., Lv, X., Zhang, S., & Qiu, W. (2025). A variable rate spraying method fusing canopy volume and disease detection to reduce pesticide dosage. Computers and Electronics in Agriculture, 237, 110606. https://doi.org/10.1016/j.compag.2025.110606

APPENDICES

Appendices are provided to give supplementary information, which is not included in the main text, and which serves as a separate part contributing to the main theme. Each appendix carries the title of the work reported in it, and tables and figures inside an appendix are numbered and referred to just as in the case of chapters.

Appendix 1: Severity and Dosing Logic (model/severity.py)

```
"""
Rice leaf disease ->
severity mapping.

This module provides TWO
things:

1. Disease classification
post-processing: turn a
model's softmax output
into a
   predicted disease class
+ confidence.

2. Segmentation-based
SEVERITY measurement
(Option B): given the
original leaf
   image, segment the leaf
from the background and
then segment the diseased
   (lesion) pixels using
classical computer vision.
Severity is the ratio of
   diseased pixels to
total leaf pixels,
expressed as a percentage.

       severity_percent =
100 * (diseased_pixels /
leaf_pixels)

   This is the definition
used by the base paper
(Tewari et al., 2020) and
does
   NOT require pixel-mask
training data because it
uses color-space
thresholding
   (HSV / LAB + Otsu)
rather than a learned
segmenter.

The class/category mapping
and the spray-time
equation are preserved so
the rest
of the project (docs,
server) keeps working.
"""

import os
from dataclasses import
dataclass
from typing import
Optional, Sequence

import numpy as np

# Default class order
(alphabetical, matching
Keras directory scanning).
# Adjust if your dataset
folders differ. Written to
class_names.txt at train
time.
DEFAULT_CLASS_NAMES = [
    "Bacterial_Blight",
    "Blast",
    "Brown_Spot",
    "Healthy",
    "Leaf_Smut",
    "Neck_Blast",
]

# Category definitions:
(code, name, base
reduction coefficient
C_base).
#
# NOTE: base coefficients
top out at 0.80 (not 1.00)
so the disease-specific
# factor (which can be up
to ~1.25) always has
headroom to change the
effective
# dose. C_eff is later
clamped to [0, 1], so the
worst case (High +
aggressive
# disease) reaches the
full 1.0 dose, while
lighter diseases stay
below it.
CATEGORY_HEALTHY = (0,
"Healthy", 0.00)
CATEGORY_LOW = (1, "Low",
0.30)
CATEGORY_MEDIUM = (2,
"Medium", 0.55)
CATEGORY_HIGH = (3,
"High", 0.80)

# ------------------------
--------------------------
-------------------------
# Disease-specific
treatment intensity.
#
# Severity (from
segmentation) sets the
BASE coefficient
C_severity. But
# different diseases
warrant different
treatment intensity for
the same visible
# area: e.g. bacterial
blight spreads
aggressively and is
treated more heavily,
# while leaf smut is
comparatively minor. So
the CNN's disease class
now
# genuinely influences the
spray dose via a per-
disease multiplier.
#
#     C_effective = clamp(
C_severity *
disease_factor , 0 , 1 )
#
# These factors are
configurable defaults;
tune them with an
agronomist. Keys
# are matched case-
insensitively and ignore
spaces/underscores, so
both
# "Bacterial_Blight" and
"Bacterial leaf blight"
resolve to the same entry.
# ------------------------
--------------------------
-------------------------
DISEASE_TREATMENT_FACTOR =
{
    "bacterialblight":
1.20,       # aggressive,
treat more heavily
    "bacterialleafblight":
1.20,
    "blast": 1.15,
# rice blast spreads fast,
treat heavily
    "leafblast": 1.15,
    "neckblast": 1.25,
# neck blast is the most
damaging stage
    "brownspot": 1.00,
# baseline
    "leafsmut": 0.80,
# comparatively minor
    "healthy": 0.00,
# never spray a healthy
leaf
}
DEFAULT_DISEASE_FACTOR =
1.00      # unknown
disease -> neutral

# --- Honesty guards
(tunable via env) --------
--------------------------
-----
# 1) Severity floor: if
the measured diseased area
is below this % of the
leaf,
#    treat the leaf as
effectively HEALTHY (kills
noise from a few stray
specks
#    being called a
disease). Set to 0 to
disable.
MIN_SEVERITY_PERCENT =
float(os.environ.get("MIN_
SEVERITY_PERCENT", "1.5"))
# 2) Confidence gate: if
the classifier's top
probability is below this,
we mark
#    the result as low-
confidence / uncertain
rather than asserting a
disease.
#    Set to 0.70 because
with 6 classes a sub-70%
top score is a weak guess
#    (random ~17%); such
predictions should be
shown as uncertain, not
asserted.
MIN_CONFIDENCE =
float(os.environ.get("MIN_
CONFIDENCE", "0.70"))


def _disease_key(name:
str) -> str:
    return "".join(ch for
ch in name.lower() if
ch.isalnum())
```

```
def _is_healthy(name: str) -> bool:
    """True if the class name denotes a healthy leaf.

    Robust to dataset naming: matches 'Healthy', 'healthy_leaf', 'Healthy Rice
    Leaf', 'normal', etc.
    """
    k = _disease_key(name)
    return "healthy" in k or k == "normal"


def disease_treatment_factor(disease_class: str) -> float:
    """Return the treatment-intensity multiplier for a disease class name."""
    if _is_healthy(disease_class):
        return 0.0  # never spray a healthy leaf, whatever the folder is named
    return DISEASE_TREATMENT_FACTOR.get(_disease_key(disease_class),
                                        DEFAULT_DISEASE_FACTOR)


@dataclass
class Prediction:
    disease_class: str
    confidence: float          # 0..1 softmax probability of predicted class
    severity_percent: float    # 0..100, MEASURED from segmentation
    category: str
    category_code: int
    reduction_coefficient: float          # EFFECTIVE C used for spraying
    base_coefficient: float = 0.0         # C from severity alone
    disease_factor: float = 1.0           # per-disease multiplier applied
    low_confidence: bool = False          # classifier top prob below MIN_CONFIDENCE
    note: str = ""             # human-readable caveat, if any

    def to_dict(self) -> dict:
        return {
            "disease_class": self.disease_class,
            "confidence": round(self.confidence, 4),
            "severity_percent": round(self.severity_percent, 2),
            "category": self.category,
            "category_code": self.category_code,
            "reduction_coefficient": self.reduction_coefficient,
            "base_coefficient": round(self.base_coefficient, 4),
            "disease_factor": self.disease_factor,
            "low_confidence": self.low_confidence,
            "note": self.note,
        }


def severity_to_category(severity_percent: float):
    """Map a severity percentage to (code, name, coefficient).

    Thresholds follow the base paper (Tewari et al., 2020) plus a Healthy=0
    category that the paper did not have.

        <= 0%        -> Healthy
        0 - 5%       -> Low
        5 - 20%      -> Medium
        > 20%        -> High
    """
    if severity_percent <= 0.0:
        return CATEGORY_HEALTHY
    if severity_percent <= 5.0:
        return CATEGORY_LOW
    if severity_percent <= 20.0:
        return CATEGORY_MEDIUM
    return CATEGORY_HIGH


# ------------------------------------------------------------------------
#
# Segmentation-based severity (Option B)
# ------------------------------------------------------------------------
#

def _to_bgr_uint8(image) -> np.ndarray:
    """Accept a path, a PIL image, or a numpy array and return BGR uint8.

    Requires OpenCV. We import cv2 lazily so that importing this module for the
    mapping helpers alone does not require OpenCV to be installed.
    """
    import cv2

    if isinstance(image, str):
        bgr = cv2.imread(image, cv2.IMREAD_COLOR)
        if bgr is None:
            raise FileNotFoundError(f"Could not read image: {image}")
        return bgr

    arr = np.asarray(image)
    if arr.dtype != np.uint8:
        # assume float in [0,1] or [0,255]
        if arr.max() <= 1.0:
            arr = (arr * 255.0).clip(0, 255)
        arr = arr.astype(np.uint8)

    if arr.ndim == 2:  # grayscale
        return cv2.cvtColor(arr, cv2.COLOR_GRAY2BGR)
    if arr.shape[2] == 4:  # RGBA
        arr = cv2.cvtColor(arr, cv2.COLOR_RGBA2RGB)
    # numpy images from PIL are RGB; convert to BGR for OpenCV consistency
    return cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)


def segment_leaf_mask(bgr: np.ndarray) -> np.ndarray:
    """Return a boolean mask of the LEAF region (True = leaf, False = background).

    Strategy: leaves (healthy green + diseased brown/yellow) are more saturated
```

```
    and less blue than
typical neutral/white/gray
backgrounds. We threshold
in
    HSV on saturation +
value, then keep the
largest connected
component and fill
    holes so lesion areas
inside the leaf are not
dropped.
    """
    import cv2

    hsv =
cv2.cvtColor(bgr,
cv2.COLOR_BGR2HSV)
    s = hsv[:, :, 1]
    v = hsv[:, :, 2]

    # Otsu on saturation
isolates colored (leaf) vs
neutral (background)
pixels.
    _, sat_mask =
cv2.threshold(s, 0, 255,
cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
    # Drop very dark
pixels (shadows) and pure-
white blowouts.
    value_ok =
cv2.inRange(v, 30, 250)
    mask =
cv2.bitwise_and(sat_mask,
value_ok)

    # Morphological
cleanup.
    kernel =
cv2.getStructuringElement(
cv2.MORPH_ELLIPSE, (7, 7))
    mask =
cv2.morphologyEx(mask,
cv2.MORPH_OPEN, kernel,
iterations=1)
    mask =
cv2.morphologyEx(mask,
cv2.MORPH_CLOSE, kernel,
iterations=2)

    # Keep the largest
connected component (the
leaf).
    num, labels, stats, _
=
cv2.connectedComponentsWit
hStats(mask,
connectivity=8)
    if num > 1:
        largest = 1 +
int(np.argmax(stats[1:,
cv2.CC_STAT_AREA]))
        mask =
np.where(labels ==
largest, 255,
0).astype(np.uint8)

    # Fill interior holes
(lesions) so the leaf area
includes diseased regions.
    filled = mask.copy()
    h, w = mask.shape
    ff_mask = np.zeros((h
+ 2, w + 2), np.uint8)
    cv2.floodFill(filled,
ff_mask, (0, 0), 255)
    holes =
cv2.bitwise_not(filled)
    leaf =
cv2.bitwise_or(mask,
holes)

    return
leaf.astype(bool)


def
segment_disease_mask(bgr:
np.ndarray, leaf_mask:
np.ndarray) -> np.ndarray:
    """Return a boolean
mask of DISEASED (lesion)
pixels within the leaf.

    Healthy rice tissue is
green (hue ~35-85 in
OpenCV's 0-179 scale).
Diseased
    lesions are brown /
yellow / tan / gray, i.e.
NOT green. So within the
leaf
    mask, any pixel that
is not sufficiently green-
and-saturated is treated
as a
    lesion. We also
require the leaf pixel to
be reasonably bright to
avoid
    counting shadow.

    NOTE: this is the
CLASSICAL fallback. When a
trained lesion-
segmentation
    model is available
(lesion_segmenter.keras),
segment_lesion_model() is
used
    instead for far better
results on cluttered field
photos.
    """
    import cv2

    hsv =
cv2.cvtColor(bgr,
cv2.COLOR_BGR2HSV)
    h = hsv[:, :, 0]
    s = hsv[:, :, 1]

    # Healthy green: hue
band + minimum saturation.
    green = (h >= 35) & (h
<= 85) & (s >= 40)

    diseased = leaf_mask &
(~green)
    return diseased


# ------------------------
---------------------------
-------------------------
#
# Trained lesion
segmentation model (Option
C) with graceful fallback
# ------------------------
---------------------------
-------------------------
#
_SEG_MODEL = None
_SEG_MODEL_TRIED = False
_SEG_IMG_SIZE = 256

# Probability threshold
above which a pixel is
called "lesion". Higher =
more
# strict (tighter masks,
less over-detection).
Tunable via env for demo
tuning.
SEG_THRESHOLD =
float(os.environ.get("SEG_
THRESHOLD", "0.5"))

# Where the trained
segmenter is expected.
Overridable via env.
SEG_MODEL_PATH =
os.environ.get(
    "SEG_MODEL_PATH",

os.path.join(os.path.dirna
me(os.path.abspath(__file_
_)),
"lesion_segmenter.keras"),
)


def _load_seg_model():
    """Load the trained
lesion segmenter once.
Returns the model or None.

    Never raises: if
TensorFlow or the file is
missing, returns None so
callers
    transparently fall
back to the color method.
    """
    global _SEG_MODEL,
_SEG_MODEL_TRIED
    if _SEG_MODEL_TRIED:
        return _SEG_MODEL
    _SEG_MODEL_TRIED =
True
    if not
os.path.exists(SEG_MODEL_P
ATH):
        return None
    try:
        import tensorflow
as tf
        _SEG_MODEL =
tf.keras.models.load_model
(SEG_MODEL_PATH,
compile=False)
        print(f"[severity]
loaded lesion segmenter:
{SEG_MODEL_PATH}")
    except Exception as e:
# noqa: BLE001
        print(f"[severity]
could not load segmenter
({e}); using color
method.")
```

```
        _SEG_MODEL = None
    return _SEG_MODEL


def
segment_lesion_model(bgr:
np.ndarray):
    """Predict a boolean
lesion mask using the
trained model, or None.

    Returns a full-
resolution boolean array
(True = lesion) matching
`bgr`, or
    None when no model is
available.
    """
    model =
_load_seg_model()
    if model is None:
        return None
    import cv2
    import numpy as _np

    h, w = bgr.shape[:2]
    rgb =
cv2.cvtColor(bgr,
cv2.COLOR_BGR2RGB)
    inp = cv2.resize(rgb,
(_SEG_IMG_SIZE,
_SEG_IMG_SIZE),
interpolation=cv2.INTER_LI
NEAR)
    inp =
inp.astype("float32")[None
, ...]
    pred =
model.predict(inp,
verbose=0)[0, :, :, 0]
    mask_small = (pred >
SEG_THRESHOLD)
    mask =
cv2.resize(mask_small.asty
pe(_np.uint8), (w, h),
interpolation=cv2.INTER_NE
AREST)
    return
mask.astype(bool)



# ----------------------
--------------------------
--------------------------
#
# Interactive leaf
isolation via MobileSAM
(Option B) with graceful
fallback
# ----------------------
--------------------------
--------------------------
#
_SAM_MODEL = None
_SAM_TRIED = False

# MobileSAM weights
(small, CPU-friendly).
ultralytics downloads it
on first use.
# Default to the copy
stored next to this file
so we don't re-download a
second
# copy into whatever the
working directory happens
to be.
SAM_WEIGHTS =
os.environ.get(
    "SAM_WEIGHTS",

os.path.join(os.path.dirna
me(os.path.abspath(__file_
_)), "mobile_sam.pt"),
)



def _load_sam():
    """Load MobileSAM once
via ultralytics. Returns
model or None (never
raises)."""
    global _SAM_MODEL,
_SAM_TRIED
    if _SAM_TRIED:
        return _SAM_MODEL
    _SAM_TRIED = True
    try:
        from ultralytics
import SAM
        _SAM_MODEL =
SAM(SAM_WEIGHTS)
        print(f"[severity]
loaded MobileSAM:
{SAM_WEIGHTS}")
    except Exception as e:
# noqa: BLE001
        print(f"[severity]
MobileSAM unavailable
({e}); using color leaf
mask.")
        _SAM_MODEL = None
    return _SAM_MODEL



def segment_leaf_sam(bgr:
np.ndarray, box):
    """Segment the leaf
inside a user-drawn box
using MobileSAM.

    `box` is (x1, y1, x2,
y2) in pixel coordinates
of `bgr`. Returns a
boolean
    leaf mask, or None if
SAM is unavailable /
fails.
    """
    model = _load_sam()
    if model is None or
box is None:
        return None
    import cv2
    import numpy as _np

    try:
        rgb =
cv2.cvtColor(bgr,
cv2.COLOR_BGR2RGB)
        x1, y1, x2, y2 =
[int(v) for v in box]
        results =
model(rgb, bboxes=[[x1,
y1, x2, y2]],
verbose=False)
        r = results[0]
        if r.masks is None
or len(r.masks.data) == 0:
            return None
        # Union of
returned masks (usually
one for a box prompt).
        m =
r.masks.data.cpu().numpy()
# (n, H, W)
        mask = (_np.max(m,
axis=0) > 0.5)
        h, w =
bgr.shape[:2]
        if mask.shape !=
(h, w):
            mask =
cv2.resize(mask.astype(_np
.uint8), (w, h),

interpolation=cv2.INTER_NE
AREST).astype(bool)
        return mask
    except Exception as e:
# noqa: BLE001
        print(f"[severity]
SAM segmentation failed
({e}); using color leaf
mask.")
        return None



def
polygon_to_mask(polygon,
height, width) ->
np.ndarray:
    """Rasterize a free-
form polygon (list of (x,
y) pixel points) to a bool
mask.

    Returns a boolean
array (True inside the
traced outline), or None
if the
    polygon is degenerate
(fewer than 3 points).
    """
    import cv2

    if polygon is None or
len(polygon) < 3:
        return None
    pts =
np.array([[int(round(x)),
int(round(y))] for x, y in
polygon], dtype=np.int32)
    mask =
np.zeros((height, width),
dtype=np.uint8)
    cv2.fillPoly(mask,
[pts], 255)
    return
mask.astype(bool)



def compute_masks(bgr:
np.ndarray, box=None,
polygon=None):
    """Return (leaf_mask,
lesion_mask, method) as
boolean arrays.

    Leaf mask (priority
order):
        1. `polygon` - a
free-form outline the user
traced -> used DIRECTLY as
```

```
          the leaf region
(most precise, user-
defined boundary).
        2. `box` +
MobileSAM - SAM segments
the leaf inside the box.
        3. Color-based
leaf mask (automatic
fallback).
    Lesion mask:
        - Trained
segmenter when available,
else the color method.
    Method string encodes
both, e.g. "poly+model",
"sam+model",
"color+color".
    """
    h, w = bgr.shape[:2]
    # --- leaf ---
    leaf = None
    leaf_src = "color"


    poly_leaf =
polygon_to_mask(polygon,
h, w) if polygon is not
None else None
    if poly_leaf is not
None and poly_leaf.sum() >
0:
        leaf = poly_leaf
        leaf_src = "poly"
    else:
        sam_leaf =
segment_leaf_sam(bgr, box)
if box is not None else
None
        if sam_leaf is not
None and sam_leaf.sum() >
0:
            leaf =
sam_leaf
            leaf_src =
"sam"
        else:
            leaf =
segment_leaf_mask(bgr)

    # --- lesion ---
    model_lesion =
segment_lesion_model(bgr)
    if model_lesion is not
None:
        lesion =
model_lesion & leaf
        lesion_src =
"model"
        if lesion.sum() ==
0 and model_lesion.sum() >
0 and leaf_src == "color":
            # Only widen
when we don't trust the
leaf mask (auto color
mask).
            # A user-
defined poly or SAM box is
trusted, so never widen
those.
            lesion =
model_lesion
            leaf = leaf |
model_lesion
    else:
        lesion =
segment_disease_mask(bgr,
leaf)
        lesion_src =
"color"

    return leaf, lesion,
f"{leaf_src}+{lesion_src}"


@dataclass
class SeverityResult:
    severity_percent:
float
    leaf_pixels: int
    diseased_pixels: int
    method: str = "color"
# "model" (trained
segmenter) or "color"
(fallback)


def make_overlay(image,
leaf_color=(60, 200, 90),
disease_color=(230, 40,
40),

alpha=0.45, box=None,
polygon=None):
    """Build an RGB
overlay visualizing the
leaf mask and diseased
pixels.

    - Leaf (healthy)
pixels are tinted green.
    - Diseased pixels are
tinted red.
    - Background is dimmed
so the leaf stands out.

    Returns an RGB uint8
numpy array (H, W, 3).
    """
    import cv2

    bgr =
_to_bgr_uint8(image)
    rgb =
cv2.cvtColor(bgr,
cv2.COLOR_BGR2RGB).astype(
np.float32)

    leaf, diseased,
_method =
compute_masks(bgr,
box=box, polygon=polygon)
    healthy = leaf &
(~diseased)

    out = rgb.copy()

    # Dim the background.
    bg = ~leaf
    out[bg] = out[bg] *
0.35

    # Tint healthy leaf
green.
    lc =
np.array(leaf_color,
dtype=np.float32)
    out[healthy] = (1 -
alpha) * out[healthy] +
alpha * lc

    # Tint diseased red.
    dc =
np.array(disease_color,
dtype=np.float32)
    out[diseased] = (1 -
alpha) * out[diseased] +
alpha * dc

    # Draw a crisp contour
around diseased regions
for clarity.
    dmask =
(diseased.astype(np.uint8)
) * 255
    contours, _ =
cv2.findContours(dmask,
cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    out_u8 = out.clip(0,
255).astype(np.uint8)

cv2.drawContours(out_u8,
contours, -1, (255, 255,
0), 1)

    return out_u8


def
overlay_png_base64(image,
box=None, polygon=None) ->
str:
    """Return the overlay
as a base64-encoded PNG
data URI string (no
prefix).

    The caller can prepend
''
for use in an <img> tag.
    """
    import base64
    import cv2

    overlay =
make_overlay(image,
box=box, polygon=polygon)
    bgr =
cv2.cvtColor(overlay,
cv2.COLOR_RGB2BGR)
    ok, buf =
cv2.imencode(".png", bgr)
    if not ok:
        raise
RuntimeError("PNG encoding
failed")
    return
base64.b64encode(buf.tobyt
es()).decode("ascii")


def
measure_severity(image,
box=None, polygon=None) ->
SeverityResult:
    """Measure diseased-
area severity for a leaf
image via segmentation.

    Uses the trained
lesion segmenter when
available, else the color
method.
```

```
    Leaf isolation: a
free-form `polygon`
(traced outline) takes
priority, then
    a `box` (with
MobileSAM), else the
automatic color leaf mask.
    `image` may be a file
path, a PIL Image, or a
numpy array (RGB).
    Returns a
SeverityResult with
severity_percent in
0..100.
    """
    bgr =
_to_bgr_uint8(image)
    leaf, lesion, method =
compute_masks(bgr,
box=box, polygon=polygon)
    leaf_pixels =
int(leaf.sum())
    if leaf_pixels == 0:
        return
SeverityResult(0.0, 0, 0,
method)

    diseased_pixels =
int(lesion.sum())
    severity = 100.0 *
diseased_pixels /
leaf_pixels
    return
SeverityResult(severity,
leaf_pixels,
diseased_pixels, method)



# ------------------------
--------------------------
-------------------------
#
# Combined prediction
# ------------------------
--------------------------
-------------------------
#

def build_prediction(
    class_names:
Sequence[str],
    probabilities:
Sequence[float],
    image=None,
    box=None,
    polygon=None,
) -> Prediction:
    """Combine disease
classification with
segmentation-based
severity.

    - Disease class +
confidence come from the
model's softmax
`probabilities`.
    - Severity:
        * If the predicted
class is "Healthy",
severity = 0%.
        * Otherwise, if
`image` is provided,
severity is MEASURED via
          segmentation
(measure_severity).
        * If no image is
provided (e.g. a unit
test), we fall back to
          confidence-
scaled severity so the
function still returns
something.
    """
    probs =
np.asarray(probabilities,
dtype=float)
    best_idx =
int(np.argmax(probs))
    predicted_class =
class_names[best_idx]
    confidence =
float(probs[best_idx])

    if
_is_healthy(predicted_clas
s):
        severity = 0.0
    elif image is not
None:
        severity =
measure_severity(image,
box=box,
polygon=polygon).severity_
percent
    else:
        # Fallback proxy
when no image is
available.
        severity =
confidence * 100.0

    return
_assemble_prediction(predi
cted_class, confidence,
severity)



def
build_prediction_from_seve
rity(class_names,
probabilities,
severity_percent):
    """Like
build_prediction but with
a PRE-MEASURED severity
value.

    Use this when severity
was already computed (e.g.
by measure_severity with a
    box) to avoid running
segmentation twice.
    """
    probs =
np.asarray(probabilities,
dtype=float)
    best_idx =
int(np.argmax(probs))
    predicted_class =
class_names[best_idx]
    confidence =
float(probs[best_idx])
    severity = 0.0 if
_is_healthy(predicted_clas
s) else
float(severity_percent)
    return
_assemble_prediction(predi
cted_class, confidence,
severity)

def
_assemble_prediction(predi
cted_class, confidence,
severity) -> Prediction:
    note = ""
    low_conf = False

    # Guard 1 - severity
floor: a few stray
diseased specks on an
otherwise
    # green leaf shouldn't
be called a disease. Treat
as Healthy -> no spray.
    if not
_is_healthy(predicted_clas
s) and 0.0 < severity <
MIN_SEVERITY_PERCENT:
        note = (f"diseased
area {severity:.2f}% is
below the
{MIN_SEVERITY_PERCENT}% "
                f"floor -
treated as healthy (no
spray)")
        predicted_class =
"Healthy"
        severity = 0.0

    # Guard 2 - confidence
gate: if the classifier is
unsure, don't assert a
    # disease as if it
were certain. Flag it and
neutralize the disease
factor.
    if not
_is_healthy(predicted_clas
s) and confidence <
MIN_CONFIDENCE:
        low_conf = True
        note = (note + " |
" if note else "") + \
               (f"low
confidence
({confidence*100:.0f}%) -
prediction uncertain")

    code, name, base_coeff
=
severity_to_category(sever
ity)
    # Disease-specific
dosing: the CNN's class
scales the base
coefficient so
    # the diagnosed
disease genuinely affects
how much is sprayed.
    factor =
disease_treatment_factor(p
redicted_class)
    effective_coeff =
max(0.0, min(1.0,
base_coeff * factor))
    return Prediction(

disease_class=predicted_cl
ass,

confidence=confidence,

severity_percent=severity,
```

```
        category=name,
category_code=code,
reduction_coefficient=effective_coeff,
base_coefficient=base_coeff,
disease_factor=factor,
low_confidence=low_conf,
        note=note,
    )


# ------------------------------------------------------------
# Spray timing.
#
# The base paper (Tewari et al., 2020) computes a per-hill chemical volume
#     a  = (C * A) / (d * 10)     [mL/hill]
#     ts = a / Qs                 [s]
# but with typical values (A=500 L/ha, d=25 hills/m^2, Qs=3.5 mL/s) this yields
# sub-second times that are unrealistically short and hard to visualize.
#
# Real variable-rate / PWM sprayers operate their solenoid open windows on the
# order of ~0.5 s (minimum meaningful pulse) up to ~3 s for a full-open dose:
#   - PWM sprayers pulse valves ~10 Hz; effective dose = duty cycle x window
#     (UNL Extension G2302).
#   - Proportional-valve field sprayers use ~3 s full open/close
#     (Agriculture 2024, 14(3), 361).
# So we standardize ts to a realistic window scaled by the effective
# coefficient C_eff in [0, 1]:
#       ts = TS_MIN + C_eff * (TS_MAX - TS_MIN)   (0 when C_eff == 0)
# Defaults: TS_MIN = 0.5 s, TS_MAX = 3.0 s. Overridable via env for calibration.
# ------------------------------------------------------------
SPRAY_TS_MIN = float(os.environ.get("SPRAY_TS_MIN", "0.5"))   # seconds
SPRAY_TS_MAX = float(os.environ.get("SPRAY_TS_MAX", "3.0"))   # seconds


def spray_time(coefficient: float, app_rate_lph: float = 500.0,
               hill_density: float = 25.0, flow_rate_mls: float = 3.5) -> float:
    """Solenoid open time ts (seconds) for a given effective coefficient.

    Standardized to a realistic actuation window (SPRAY_TS_MIN..SPRAY_TS_MAX)
    scaled linearly by the effective coefficient C_eff in [0, 1]:

        ts = 0                                  if C_eff <= 0  (healthy, no spray)
        ts = TS_MIN + C_eff * (TS_MAX - TS_MIN)  otherwise

    The app_rate/hill_density/flow_rate args are retained for API compatibility
    and reporting (per-hill volume a = C*A/(d*10)) but no longer drive ts, since
    the raw base-paper ts is sub-second and unrealistic for a real valve.
    """
    c = max(0.0, min(1.0, float(coefficient)))
    if c <= 0.0:
        return 0.0
    return SPRAY_TS_MIN + c * (SPRAY_TS_MAX - SPRAY_TS_MIN)


if __name__ == "__main__":
    # quick self-check of the disease-specific dosing (no image needed).
    # Same severity, different disease -> different effective coefficient.
    names = ["Bacterial_Blight", "Brown_Spot", "Leaf_Smut"]
    for i, nm in enumerate(names):
        probs = [0.1, 0.1, 0.1]
        probs[i] = 0.9
        # force a fixed severity via the no-image fallback (confidence*100=90 -> High)
        pred = build_prediction(names, probs)
        d = pred.to_dict()
        print(f"{nm:18s} base_C={d['base_coefficient']} x factor={d['disease_factor']} "
              f"-> effective_C={d['reduction_coefficient']}")
```

Appendix 2: Single Image Command Line Prediction (model/predict.py)

```
"""
Single-image inference:
upload one rice leaf photo
-> disease + severity
level.

Pipeline:
    1. Load the trained
EfficientNetV2-B0 model
and class_names.txt.
    2. Classify the image
-> disease class +
confidence.
    3. If diseased,
MEASURE severity by
segmentation (severity.py)
-> % of leaf
       area that is
diseased.
    4. Map severity % ->
Healthy / Low / Medium /
High.

Usage:
    python predict.py --
image leaf.jpg
    python predict.py --
image leaf.jpg --model
rice_disease_effnetv2b0.ke
ras
    python predict.py --
image leaf.jpg --
save_overlay overlay.png

Outputs a JSON object with
disease_class, confidence,
severity_percent,
category, category_code,
reduction_coefficient.
"""

import argparse
import json
import os

import numpy as np

import severity as sev

IMG_SIZE = 224
HERE =
os.path.dirname(os.path.ab
spath(__file__))
DEFAULT_MODEL =
os.path.join(HERE,
"rice_disease_effnetv2b0.k
eras")
DEFAULT_CLASSES =
os.path.join(HERE,
"class_names.txt")



def
load_class_names(path):
    if
os.path.exists(path):
        with open(path) as
f:
            names =
[ln.strip() for ln in f if
ln.strip()]
        if names:
            return names
    print(f"[warn] {path}
not found; using
DEFAULT_CLASS_NAMES from
severity.py")
    return
list(sev.DEFAULT_CLASS_NAM
ES)



def
load_image_for_model(image
_path):
    """Load and preprocess
an image for
EfficientNetV2 inference.

    Returns
(batched_tensor,
pil_rgb_image). The PIL
image (original, unscaled)
    is passed to the
severity segmenter so it
measures on real pixels.
    """
    import tensorflow as
tf
    from
tensorflow.keras.applicati
ons.efficientnet_v2 import
preprocess_input
    from PIL import Image

    pil =
Image.open(image_path).con
vert("RGB")
    resized =
pil.resize((IMG_SIZE,
IMG_SIZE))
    arr =
np.asarray(resized,
dtype=np.float32)
    arr =
preprocess_input(arr)
    batch =
np.expand_dims(arr,
axis=0)
    return batch, pil



def
save_overlay(image_path,
out_path):
    """Save a
visualization: leaf
outline + diseased pixels
highlighted."""
    import cv2
    from PIL import Image

    pil =
Image.open(image_path).con
vert("RGB")
    bgr =
sev._to_bgr_uint8(pil)
    leaf =
sev.segment_leaf_mask(bgr)
    diseased =
sev.segment_disease_mask(b
gr, leaf)

    overlay = bgr.copy()
    overlay[diseased] =
(0, 0, 255)  # red for
diseased pixels
    blended =
cv2.addWeighted(bgr, 0.6,
overlay, 0.4, 0)
    cv2.imwrite(out_path,
blended)
    print(f"[ok] overlay
saved: {out_path}")



def predict(image_path,
model_path, classes_path):
    import tensorflow as
tf

    class_names =
load_class_names(classes_p
ath)
    model =
tf.keras.models.load_model
(model_path)

    batch, pil =
load_image_for_model(image
_path)
    probs =
model.predict(batch,
verbose=0)[0]

    # Combine
classification with
segmentation-based
severity (pass the image).
    pred =
sev.build_prediction(class
_names, probs, image=pil)
    return pred



def main():
    parser =
argparse.ArgumentParser(de
scription="Predict rice
disease + severity for one
image.")
    parser.add_argument("-
-image", required=True,
help="Path to the leaf
image.")
    parser.add_argument("-
-model",
default=DEFAULT_MODEL,
help="Path to trained
.keras model.")
    parser.add_argument("-
-classes",
default=DEFAULT_CLASSES,
help="Path to
class_names.txt.")
    parser.add_argument("-
-save_overlay",
```

```
default=None,
help="Optional path to
save a diseased-area
overlay image.")
    args =
parser.parse_args()

    if not
os.path.exists(args.image)
:
        raise
FileNotFoundError(f"Image
not found: {args.image}")
    if not
os.path.exists(args.model)
:
        raise
FileNotFoundError(
            f"Model not
found: {args.model}\n"
            "Train first
with
train_efficientnetv2.py or
pass --model."
        )

    pred =
predict(args.image,
args.model, args.classes)

print(json.dumps(pred.to_d
ict(), indent=2))

    if args.save_overlay:

save_overlay(args.image,
args.save_overlay)


if __name__ == "__main__":
    main()
```

Appendix 3: Flask Inference API (app/server.py)

```
"""
Flask backend for the Rice Leaf Disease + Severity demo UI.

Serves the static frontend (app/static/index.html) and a /predict endpoint.

Endpoints:
    GET  /            -> the demo web UI
    GET  /health      -> {"status": "ok", "model_loaded": bool, "mode": ...}
    POST /predict     -> multipart "image" OR raw body bytes; returns JSON:
        {
          "disease_class", "confidence", "severity_percent",
          "leaf_pixels", "diseased_pixels",
          "category", "category_code", "reduction_coefficient",
          "spray_time_s", "spray_params": {A, d, Qs, a_ml_per_hill},
          "overlay_png": "<base64>",       # mask overlay image
          "mode": "model" | "demo"
        }

MODES
    - "model": a trained EfficientNetV2 model is found and used for the disease
      class. Severity is always measured by segmentation (severity.py).
    - "demo":  no model file present. The disease class is inferred by a simple
      heuristic so the whole UI (mask + formulas + animation) is fully
      demonstrable before training finishes. Severity is still real (measured).

Run locally:
    python app/server.py
    # then open http://127.0.0.1:5000

Colab: see app/launch_colab.py for a public share URL.
"""

import io
import os
import sys

from flask import Flask, jsonify, request, send_from_directory

# Make the shared severity module importable regardless of CWD.
HERE = os.path.dirname(os.path.abspath(__file__))
MODEL_DIR = os.path.join(HERE, "..", "model")
sys.path.append(MODEL_DIR)

import severity as sev  # noqa: E402

IMG_SIZE = 224
STATIC_DIR = os.path.join(HERE, "static")

MODEL_PATH = os.environ.get(
    "MODEL_PATH", os.path.join(MODEL_DIR, "rice_disease_effnetv2b0.keras")
)
CLASS_NAMES_PATH = os.environ.get(
    "CLASS_NAMES_PATH", os.path.join(MODEL_DIR, "class_names.txt")
)

# Spray equation defaults (from the base paper). Overridable via query/env.
DEFAULT_APP_RATE_LPH = float(os.environ.get("APP_RATE_LPH", "500"))   # A (L/ha)
DEFAULT_HILL_DENSITY = float(os.environ.get("HILL_DENSITY", "25"))    # d (hills/m^2)
DEFAULT_FLOW_RATE_MLS = float(os.environ.get("FLOW_RATE_MLS", "3.5"))  # Qs (mL/s)

try:
    from flask_cors import CORS
    _HAS_CORS = True
except ImportError:
    _HAS_CORS = False

app = Flask(__name__, static_folder=None)
if _HAS_CORS:
    CORS(app)

_model = None
_class_names = None
_mode = None  # "model" or "demo"


def _load_class_names():
    if os.path.exists(CLASS_NAMES_PATH):
        with open(CLASS_NAMES_PATH) as f:
            names = [ln.strip() for ln in f if ln.strip()]
        if names:
            return names
    return list(sev.DEFAULT_CLASS_NAMES)


def _try_load_model():
    """Attempt to load a trained model. Set mode to 'model' or 'demo'."""
    global _model, _class_names, _mode
    _class_names = _load_class_names()
    if os.path.exists(MODEL_PATH):
        try:
            import tensorflow as tf
            _model = tf.keras.models.load_model(MODEL_PATH)
            _mode = "model"
            print(f"[server] loaded model: {MODEL_PATH}")
            return
        except Exception as e:  # noqa: BLE001
            print(f"[server] model present but failed to load ({e}); demo mode.")
    else:
        print("[server] no model file found; running in DEMO mode "
              "(severity is still measured by segmentation).")
    _mode = "demo"


def _pil_from_bytes(image_bytes):
    from PIL import Image
    return Image.open(io.BytesIO(image_bytes)).convert("RGB")


def _pil_from_rgb565(raw, width, height):
    """Build an RGB image from a raw big-endian RGB565 frame.
```

```
    The ESP32-CAM clone
sensor used in this
project has no hardware
JPEG and
    the on-device software
JPEG encoder exhausts the
ESP32's internal RAM, so
    the firmware ships the
raw RGB565 frame instead.
Each pixel is 2 bytes,
    big-endian (the esp32-
camera default): RRRRRGGG
GGGBBBBB. We expand to 8
    bits per channel and
let PIL take over from
there.
    """
    import numpy as np
    from PIL import Image

    expected = width *
height * 2
    if len(raw) <
expected:
        raise ValueError(
            f"RGB565 body
too short: got {len(raw)}
bytes, "
            f"need
{expected} for
{width}x{height}"
        )
    px =
np.frombuffer(raw[:expecte
d],
dtype=">u2").astype(np.uin
t32)
    r = ((px >> 11) &
0x1F) << 3
    g = ((px >> 5) & 0x3F)
<< 2
    b = (px & 0x1F) << 3
    rgb = np.stack([r, g,
b], axis=-
1).astype(np.uint8).reshap
e((height, width, 3))
    return
Image.fromarray(rgb,
mode="RGB")



def
_classify_with_model(pil):
    import numpy as np
    from
tensorflow.keras.applicati
ons.efficientnet_v2 import
preprocess_input

    img =
pil.resize((IMG_SIZE,
IMG_SIZE))
    arr = np.asarray(img,
dtype="float32")
    arr =
preprocess_input(arr)
    batch =
np.expand_dims(arr,
axis=0)
    probs =
_model.predict(batch,
verbose=0)[0]
    return probs


def _classify_demo(pil,
severity_percent):
    """Heuristic pseudo-
classification for demo
mode.

    Returns a
probabilities vector
aligned with _class_names.
If the leaf is
    essentially clean,
favor Healthy; otherwise
favor the first non-
healthy
    class. This is ONLY to
make the UI complete
without a trained model;
the
    severity number itself
is real (measured by
segmentation).
    """
    n = len(_class_names)
    probs = [0.0] * n
    healthy_idx = next(
        (i for i, c in
enumerate(_class_names) if
c.lower() == "healthy"),
None
    )
    if severity_percent <=
0.5 and healthy_idx is not
None:
        probs[healthy_idx]
= 0.98
        for i in range(n):
            if i !=
healthy_idx:
                probs[i] =
0.02 / max(1, n - 1)
    else:
        # pick first non-
healthy class as the
"detected" disease
        disease_idx =
next(
            (i for i in
range(n) if i !=
healthy_idx), 0
        )
        probs[disease_idx]
= 0.9
        for i in range(n):
            if i !=
disease_idx:
                probs[i] =
0.1 / max(1, n - 1)
    return probs


def
_extract_image_bytes(req):
    # Multipart upload
(web UI) first.
    if req.files and
"image" in req.files:
        return
req.files["image"].read()
    # Raw body (ESP32
posts application/octet-
stream). Read it straight
from
    # the input stream so
it works regardless of
content type and is not
    # affected by any
earlier access to
request.data/form.
content_length
    # tells us how much to
read.
    try:
        n =
req.content_length or 0
        if n > 0:
            raw =
req.stream.read(n)
            if raw:
                return raw
    except Exception:  #
noqa: BLE001
        pass
    # Fallbacks.
    raw =
req.get_data(cache=True,
as_text=False)
    if raw:
        return raw
    return None


def _parse_box(req, size):
    """Parse an optional
leaf bounding box from the
request.

    Accepts a 'box'
field/param as
"x1,y1,x2,y2". Values may
be pixel coords or
    normalized [0,1]
(auto-detected: if all <=
1, scale by image size).
    Returns (x1, y1, x2,
y2) in pixels, or None.
    """
    raw = None
    if req.form and "box"
in req.form:
        raw =
req.form["box"]
    elif "box" in
req.args:
        raw =
req.args["box"]
    if not raw:
        return None
    try:
        parts = [float(v)
for v in raw.split(",")]
        if len(parts) !=
4:
            return None
        w, h = size
        if all(0.0 <= p <=
1.0 for p in parts):
            x1, y1, x2, y2
= parts[0] * w, parts[1] *
h, parts[2] * w, parts[3]
* h
        else:
            x1, y1, x2, y2
= parts
        x1, x2 =
sorted((max(0, min(w,
x1)), max(0, min(w, x2))))
        y1, y2 =
sorted((max(0, min(h,
y1)), max(0, min(h, y2))))
        if x2 - x1 < 5 or
y2 - y1 < 5:
```

```
            return None
        return (x1, y1,
x2, y2)
    except Exception:  #
noqa: BLE001
        return None


def _parse_polygon(req,
size):
    """Parse an optional
free-form leaf polygon
from the request.

    Accepts a 'polygon'
field/param as flat
"x1,y1,x2,y2,...,xn,yn".
Values may
    be pixel coords or
normalized [0,1] (auto-
detected). Returns a list
of
    (x, y) pixel tuples
(>= 3 points), or None.
    """
    raw = None
    if req.form and
"polygon" in req.form:
        raw =
req.form["polygon"]
    elif "polygon" in
req.args:
        raw =
req.args["polygon"]
    if not raw:
        return None
    try:
        nums = [float(v)
for v in raw.split(",") if
v.strip() != ""]
        if len(nums) < 6
or len(nums) % 2 != 0:
            return None
        w, h = size
        normalized =
all(0.0 <= v <= 1.0 for v
in nums)
        pts = []
        for i in range(0,
len(nums), 2):
            x, y =
nums[i], nums[i + 1]
            if normalized:
                x, y = x *
w, y * h
            x = max(0,
min(w, x))
            y = max(0,
min(h, y))
            pts.append((x,
y))
        return pts if
len(pts) >= 3 else None
    except Exception:  #
noqa: BLE001
        return None


@app.route("/")
def index():
    return
send_from_directory(STATIC
_DIR, "index.html")

@app.route("/static/<path:
filename>")
def
static_files(filename):
    return
send_from_directory(STATIC
_DIR, filename)


@app.route("/health")
def health():
    if _mode is None:
        _try_load_model()
    return jsonify({
        "status": "ok",
        "model_loaded":
_model is not None,
        "mode": _mode,
        "classes":
_class_names,
    })


@app.route("/predict",
methods=["POST"])
def predict():
    if _mode is None:
        _try_load_model()

    fmt =
(request.args.get("fmt")
          or
request.headers.get("X-
Image-Format")
          or "").lower()

    def _diag(msg):
        line = f"{msg}\n"
        try:
            with
open(os.path.join(HERE,
"predict_debug.log"), "a")
as _f:

_f.write(line)
        except Exception:
# noqa: BLE001
            pass
        print(line,
end="", flush=True)

    # Read the body BEFORE
touching
request.data/form, so the
input stream is
    # not consumed out
from under us.
    image_bytes =
_extract_image_bytes(reque
st)

    _diag(f"[predict]
from={request.remote_addr}
fmt={fmt!r} "

f"args={dict(request.args)
}
content_length={request.co
ntent_length} "

f"extracted={len(image_byt
es) if image_bytes else
0}")

    if not image_bytes:
        _diag("[predict]
400: no image bytes
extracted")
        return
jsonify({"error": "no
image provided"}), 400

    # Raw RGB565 path: the
ESP32-CAM firmware sends
the raw frame (no JPEG)
    # when it declares
format=rgb565 with
width/height, either as
query params
    #
(?fmt=rgb565&w=160&h=120)
or the X-Image-Format / X-
Image-Width /
    # X-Image-Height
headers. Everything else
is treated as an encoded
image
    # (JPEG/PNG) exactly
as before.
    try:
        if fmt in
("rgb565", "raw565"):
            w =
int(request.args.get("w")
or request.headers.get("X-
Image-Width"))
            h =
int(request.args.get("h")
or request.headers.get("X-
Image-Height"))

_diag(f"[predict] rgb565
decode w={w} h={h}
bytes={len(image_bytes)}")
            pil =
_pil_from_rgb565(image_byt
es, w, h)
            # Debug aid:
keep the last frame the
ESP32 sent so colours /
framing
            # can be
checked by eye (wrong byte
order would swap colours).
            try:

pil.save(os.path.join(HERE
, "last_esp32_frame.png"))
            except
Exception as save_err:  #
noqa: BLE001

print(f"[server] could not
save debug frame:
{save_err}")
        else:
            pil =
_pil_from_bytes(image_byte
s)
    except Exception as e:
# noqa: BLE001
        _diag(f"[predict]
400: decode failed: {e}")
        return
jsonify({"error": f"could
not decode image: {e}"}),
400

    # Optional leaf
isolation. A free-form
```

```
'polygon' (traced outline) takes
    # priority; otherwise a 'box' (with MobileSAM); otherwise auto color mask.
    box = _parse_box(request, pil.size)
    polygon = _parse_polygon(request, pil.size)

    # Measure severity (real, from segmentation) once; reused for the overlay.
    try:
        sres = sev.measure_severity(pil, box=box, polygon=polygon)
        seg_method = getattr(sres, "method", "unknown")
    except Exception as e:  # noqa: BLE001
        return jsonify({"error": f"severity segmentation failed: {e}"}), 500

    # Classify disease.
    try:
        if _mode == "model":
            probs = _classify_with_model(pil)
        else:
            probs = _classify_demo(pil, sres.severity_percent)
    except Exception as e:  # noqa: BLE001
        return jsonify({"error": f"classification failed: {e}"}), 500

    # Build the prediction using the measured severity (avoids re-segmenting).
    pred = sev.build_prediction_from_severity(
        _class_names, probs, sres.severity_percent
    )

    # Spray-time equation from the base paper.
    app_rate = float(request.args.get("A", DEFAULT_APP_RATE_LPH))
    hill_density = float(request.args.get("d", DEFAULT_HILL_DENSITY))
    flow_rate = float(request.args.get("Qs", DEFAULT_FLOW_RATE_MLS))
    a_ml_per_hill = (pred.reduction_coefficient * app_rate) / (hill_density * 10.0)
    ts = sev.spray_time(pred.reduction_coefficient, app_rate, hill_density, flow_rate)

    # Overlay PNG. The web UI wants it, but the ESP32 does not and cannot hold
    # the large base64 string (its JSON buffer overflows -> NoMemory). Clients
    # can skip it with ?overlay=0 (the firmware does this); it is also skipped
    # automatically for the raw RGB565 path.
    want_overlay = (request.args.get("overlay", "1") != "0") and fmt not in (
        "rgb565", "raw565")
    if want_overlay:
        try:
            overlay_b64 = sev.overlay_png_base64(pil, box=box, polygon=polygon)
        except Exception as e:  # noqa: BLE001
            overlay_b64 = None
            print(f"[server] overlay generation failed: {e}")
    else:
        overlay_b64 = None

    result = pred.to_dict()
    result.update({
        "leaf_pixels": sres.leaf_pixels,
        "diseased_pixels": sres.diseased_pixels,
        "seg_method": seg_method,
        "spray_time_s": round(ts, 3),
        "spray_params": {
            "A_lph": app_rate,
            "d_hills_per_m2": hill_density,
            "Qs_mls": flow_rate,
            "a_ml_per_hill": round(a_ml_per_hill, 4),
            "ts_min": sev.SPRAY_TS_MIN,
            "ts_max": sev.SPRAY_TS_MAX,
        },
        "overlay_png": overlay_b64,
        "mode": _mode,
    })
    return jsonify(result)


if __name__ == "__main__":
    _try_load_model()
    port = int(os.environ.get("PORT", "5000"))
    app.run(host="0.0.0.0", port=port, debug=False)
```

Appendix 4: ESP32-CAM Firmware (hardware_evidence/firmware/esp32cam_sprayer/esp32cam_sprayer.ino)

```
/*
  esp32cam_sprayer.ino
  ------------------------
--------------------------
------------------------
  BCE497J - Deep Learning
and IoT-Based Real-Time
Variable-Rate Chemical
  Spraying System for
Disease Control in Paddy
Crop.

  AI-Thinker ESP32-CAM
controller. One cycle:
    1. Wait for a trigger
(push button on GPIO13 to
GND, or Blynk button).
    2. Capture one RGB565
QVGA (320x240) frame.
    3. POST the RAW RGB565
bytes to the Flask
/predict API with

?fmt=rgb565&w=<w>&h=<h>.
This board's clone OV2640
has NO hardware
       JPEG
(PIXFORMAT_JPEG -> error
0x106), so the server
converts the frame
       instead of the
ESP32 (app/server.py,
_pil_from_rgb565).
    4. Parse the JSON
reply and light the
matching status LED.
    5. Send
"S<ms>,<cat>\n" to the
Arduino Nano to open the
valve.
    6. (Optional) push
severity/category/usage to
Blynk.

  HARD RULE: never use
GPIO16 or GPIO17. GPIO16
is the PSRAM chip-select
on
  the AI-Thinker ESP32-CAM
(CONFIG_D0WD_PSRAM_CS_IO=1
6). Reconfiguring it
  disables PSRAM, and
malloc() > 4 KB is served
from PSRAM, so the camera
  driver, HTTP stack and
any large buffer then
crash (LoadProhibited).

  Tested toolchain:
arduino-esp32 core 2.0.17,
ArduinoJson 7.4.3,
  Blynk 1.4.0 (only if
BLYNK_ENABLED 1). Blynk is
OFF by default.
*/

// ------------------------
--------------------------
-------------------- //
// BLYNK CONFIG - must be
defined before any
include. Set BLYNK_ENABLED
to 1
// to re-enable the
dashboard once the core
loop is confirmed working.
// ------------------------
--------------------------
-------------------- //
#define BLYNK_ENABLED    0
#if BLYNK_ENABLED
  #define
BLYNK_TEMPLATE_ID
"TMPL3Yo1RlZPt"
  #define
BLYNK_TEMPLATE_NAME
"PaddyGuard"
  #define BLYNK_AUTH_TOKEN
"jvPkWFFePNLhTNqAsWHgjpXfN
qN38XNl"
  #define BLYNK_PRINT
Serial
#endif

#include "esp_camera.h"
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#if BLYNK_ENABLED
  #include
<BlynkSimpleEsp32.h>
#endif

// ------------------------
--------------------------
-------------------- //
// FILL IN BEFORE FLASHING
// ------------------------
--------------------------
-------------------- //
#define WIFI_SSID
"Sourish\u2019s iPhone"
// curly apostrophe
(U+2019) to match the
iPhone hotspot name
exactly
#define WIFI_PASSWORD
"sourishid27112004"
// The Flask server's
/predict URL on the iPhone
hotspot. If your PC's IP
on
// the hotspot changes
(reconnect, reboot),
update this and reflash.
#define PREDICT_URL
"http://172.20.10.4:5000/p
redict"

// ------------------------
--------------------------
-------------------- //
// Pin map (AI-Thinker
ESP32-CAM). Do not change
the camera pins.
// ------------------------
--------------------------
-------------------- //
#define PWDN_GPIO_NUM
32
#define RESET_GPIO_NUM
-1
#define XCLK_GPIO_NUM
0
#define SIOD_GPIO_NUM
26
#define SIOC_GPIO_NUM
27
#define Y9_GPIO_NUM
35
#define Y8_GPIO_NUM
34
#define Y7_GPIO_NUM
39
#define Y6_GPIO_NUM
36
#define Y5_GPIO_NUM
21
#define Y4_GPIO_NUM
19
#define Y3_GPIO_NUM
18
#define Y2_GPIO_NUM
5
#define VSYNC_GPIO_NUM
25
#define HREF_GPIO_NUM
23
#define PCLK_GPIO_NUM
22

// !!! NEVER use GPIO16 or
GPIO17 on this board. In
the core's sdkconfig
//
CONFIG_D0WD_PSRAM_CS_IO=16
/ CLK_IO=17: GPIO16 is the
PSRAM chip-select.
// Calling pinMode() on it
disables PSRAM, and
because malloc() > 4 KB is
// served from PSRAM, that
caused every
LoadProhibited crash /
malloc failure
// seen during bring-up
(camera init, JPEG encode,
HTTP POST).
// Free pins on this board
with camera + PSRAM + no
SD card: 2, 12, 13, 14, 15
// (GPIO4 drives the
onboard flash LED; GPIO0
is the camera XCLK).
#define TRIGGER_BUTTON_PIN
13  // push button to GND
(INPUT_PULLUP). Was 16.
#define LED_HEALTHY_PIN
12  // category 0
(strapping pin:
LED+resistor to GND is
fine)
#define LED_LOW_PIN
2  // category 1 (was 13,
now used by the button)
#define LED_MEDIUM_PIN
14  // category 2
```

```
#define LED_HIGH_PIN
15  // category 3

#define TRIGGER_PERIOD_MS
0        // 0 =
button/Blynk only, no auto
timer
#define
SERIAL_BAUD_TO_NANO 9600
#define HTTP_TIMEOUT_MS
15000

#if BLYNK_ENABLED
#define VPIN_SEVERITY
V0
#define VPIN_CATEGORY
V1
#define VPIN_CHEMICAL_LOG
V2
#define VPIN_ALERT
V3
#define
VPIN_REMOTE_TRIGGER V4
bool
remoteTriggerRequested =
false;
BLYNK_WRITE(VPIN_REMOTE_TR
IGGER) {
  if (param.asInt() == 1)
{
    remoteTriggerRequested
= true;
  }
}
#endif

unsigned long
lastAutoTrigger = 0;

// ----------------------
--------------------------
-------------------- //
// Camera init. This
board's clone OV2640 has
no hardware JPEG
// (PIXFORMAT_JPEG ->
error 0x106), so frames
are captured as RGB565 and
sent
// raw; the Flask server
converts them. Every
config field is defined
(the
// struct is value-
initialised) and the frame
buffer lives in PSRAM,
which is
// healthy now that GPIO16
(PSRAM chip-select) is no
longer touched.
// ----------------------
--------------------------
-------------------- //
bool initCamera() {
  camera_config_t config =
{};
  config.ledc_channel =
LEDC_CHANNEL_0;
  config.ledc_timer   =
LEDC_TIMER_0;
  config.pin_d0 =
Y2_GPIO_NUM;
  config.pin_d1 =
Y3_GPIO_NUM;
  config.pin_d2 =
Y4_GPIO_NUM;
  config.pin_d3 =
Y5_GPIO_NUM;
  config.pin_d4 =
Y6_GPIO_NUM;
  config.pin_d5 =
Y7_GPIO_NUM;
  config.pin_d6 =
Y8_GPIO_NUM;
  config.pin_d7 =
Y9_GPIO_NUM;
  config.pin_xclk =
XCLK_GPIO_NUM;
  config.pin_pclk =
PCLK_GPIO_NUM;
  config.pin_vsync =
VSYNC_GPIO_NUM;
  config.pin_href =
HREF_GPIO_NUM;
  config.pin_sccb_sda =
SIOD_GPIO_NUM;
  config.pin_sccb_scl =
SIOC_GPIO_NUM;
  config.pin_pwdn =
PWDN_GPIO_NUM;
  config.pin_reset =
RESET_GPIO_NUM;
  config.xclk_freq_hz =
20000000;
  config.pixel_format =
PIXFORMAT_RGB565;   //
clone sensor: no HW JPEG
  config.frame_size   =
FRAMESIZE_QVGA;     //
320x240 (153,600 bytes
RGB565)
  config.jpeg_quality =
12;                 //
unused for RGB565
  config.fb_count     = 1;
  config.fb_location  =
CAMERA_FB_IN_PSRAM;
  config.grab_mode    =
CAMERA_GRAB_WHEN_EMPTY;

  esp_err_t err =
esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("[cam]
init failed, error
0x%x\n", err);
    return false;
  }
  return true;
}

void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID,
WIFI_PASSWORD);
  Serial.print("[wifi]
connecting");
  unsigned long start =
millis();
  while (WiFi.status() !=
WL_CONNECTED && millis() -
start < 20000) {
    delay(300);
    Serial.print(".");
  }
  Serial.println();
  if (WiFi.status() ==
WL_CONNECTED) {
    Serial.print("[wifi]
connected, IP ");
Serial.println(WiFi.localI
P());
  } else {
    Serial.println("[wifi]
could not connect within
20 s, will retry on next
trigger");
  }
}

void setStatusLed(int
categoryCode) {

digitalWrite(LED_HEALTHY_P
IN, categoryCode == 0 ?
HIGH : LOW);

digitalWrite(LED_LOW_PIN,
categoryCode == 1 ? HIGH :
LOW);

digitalWrite(LED_MEDIUM_PI
N,  categoryCode == 2 ?
HIGH : LOW);

digitalWrite(LED_HIGH_PIN,
categoryCode == 3 ? HIGH :
LOW);
}

// Send "S<ms>,<cat>\n" to
the Nano.
void
sendSprayCommandToNano(flo
at sprayTimeS, int
categoryCode) {
  long ms =
(long)(sprayTimeS * 1000.0
+ 0.5);
  if (ms < 0) {
    ms = 0;
  }
  Serial.print('S');
  Serial.print(ms);
  Serial.print(',');

Serial.print(categoryCode)
;
  Serial.print('\n');
}

// Capture an RGB565 frame
and POST the RAW bytes to
Flask. We do NOT encode
// JPEG on the ESP32: this
clone sensor has no
hardware JPEG, and the
software
// encoder (frame2jpg)
exhausts the ESP32's
scarce internal RAM and
crashes
// (JPG buffer malloc
failed / LoadProhibited).
Instead the server accepts
the
// raw RGB565 frame
(fmt=rgb565 with
width/height) and does the
conversion on
// the PC, which has ample
memory. The raw frame
already lives in the
camera
```

```
// frame buffer, so no
extra allocation is needed
here.
bool
captureAndPredict(JsonDocu
ment &outJson) {
  camera_fb_t *fb =
esp_camera_fb_get();
  if (!fb) {
    Serial.println("[cam]
capture failed");
    return false;
  }

  Serial.printf("[cam]
captured RGB565 %ux%u, %u
bytes\n",
                fb->width,
fb->height, (unsigned)fb-
>len);

  bool ok = false;
  if (WiFi.status() ==
WL_CONNECTED) {
    // Tell the server
this is a raw RGB565 frame
and its dimensions, and
ask
    // it to omit the
large overlay_png field
(overlay=0): the ESP32
only needs
    // the small decision
fields, and the base64 PNG
overflowed the JSON
    // buffer (JSON parse
error: NoMemory).
    String url =
String(PREDICT_URL) +
"?fmt=rgb565&overlay=0&w="
+

String(fb->width) + "&h="
+ String(fb->height);
    HTTPClient http;

http.setTimeout(HTTP_TIMEO
UT_MS);
    http.begin(url);

http.addHeader("Content-
Type", "application/octet-
stream");
    int code =
http.POST(fb->buf, fb-
>len);
    if (code == 200) {
      String body =
http.getString();
      DeserializationError
derr =
deserializeJson(outJson,
body);
      if (!derr) {
        ok = true;
      } else {

Serial.print("[api] JSON
parse error: ");

Serial.println(derr.c_str(
));
      }
    } else {
      Serial.print("[api]
POST failed, HTTP code ");

Serial.println(code);
    }
    http.end();
  } else {
    Serial.println("[api]
WiFi not connected,
skipping this cycle");
  }


esp_camera_fb_return(fb);
// release the frame
buffer
  return ok;
}

#if BLYNK_ENABLED
void
publishToBlynk(JsonDocumen
t &j) {
  float severity =
j["severity_percent"] |
0.0;
  const char *category =
j["category"] | "Healthy";
  float volumeMl =
j["spray_params"]["a_ml_pe
r_hill"] | 0.0;

Blynk.virtualWrite(VPIN_SE
VERITY, severity);

Blynk.virtualWrite(VPIN_CA
TEGORY, category);

Blynk.virtualWrite(VPIN_CH
EMICAL_LOG, volumeMl);
  if (severity > 20.0) {

Blynk.virtualWrite(VPIN_AL
ERT, "High severity
detected, full dose
sprayed");
  }
}
#endif

bool triggerRequested() {
  static bool
lastButtonState = HIGH;
  bool requested = false;

  bool buttonState =
digitalRead(TRIGGER_BUTTON
_PIN);
  if (lastButtonState ==
HIGH && buttonState ==
LOW) {
    requested = true;  //
falling edge, active LOW
pull-up
  }
  lastButtonState =
buttonState;

  if (TRIGGER_PERIOD_MS >
0 && millis() -
lastAutoTrigger >=
(unsigned
long)TRIGGER_PERIOD_MS) {
    lastAutoTrigger =
millis();
    requested = true;
  }

#if BLYNK_ENABLED
  if
(remoteTriggerRequested) {
    remoteTriggerRequested
= false;
    requested = true;
  }
#endif

  return requested;
}

void runOneCycle() {
  Serial.println("[cycle]
triggered");
  // 4 KB is plenty for
the decision fields now
that overlay_png is
omitted
  // (see
captureAndPredict: the
request sends overlay=0).
Dynamic so it lives
  // on the heap, not the
stack.
  DynamicJsonDocument
doc(4096);
  if
(!captureAndPredict(doc))
{

Serial.println("[cycle] no
result this time, LEDs
unchanged, no code sent to
Nano");
    return;
  }

  const char *diseaseClass
= doc["disease_class"] |
"Unknown";
  float confidence =
doc["confidence"] | 0.0;
  float severity =
doc["severity_percent"] |
0.0;
  int categoryCode =
doc["category_code"] | 0;
  float sprayTime =
doc["spray_time_s"] | 0.0;

  Serial.printf("[cycle]
class=%s conf=%.2f
severity=%.2f%%
category_code=%d
ts=%.2fs\n",

diseaseClass, confidence,
severity, categoryCode,
sprayTime);


setStatusLed(categoryCode)
;

sendSprayCommandToNano(spr
ayTime, categoryCode);

#if BLYNK_ENABLED
  publishToBlynk(doc);
#endif
}
```

```
void setup() {

Serial.begin(SERIAL_BAUD_TO_NANO);
  delay(200);
  Serial.println();
  Serial.println("[boot] PaddyGuard ESP32-CAM sprayer controller");

  // Trigger button on GPIO13 (NOT 16: GPIO16 is the PSRAM chip-select).

pinMode(TRIGGER_BUTTON_PIN, INPUT_PULLUP);

  Serial.printf("[trace] psramFound=%d freeHeap=%u psramSize=%u\n",

(int)psramFound(), (unsigned)ESP.getFreeHeap(),

(unsigned)ESP.getPsramSize());
  Serial.println("[trace] about to call initCamera()");
  Serial.flush();

  if (!initCamera()) {
    Serial.println("[boot] camera init failed, halting (check ribbon cable seating)");
    while (true) {
      delay(1000);
    }
  }
  Serial.println("[trace] initCamera() returned OK");
  Serial.flush();

  // Status LEDs (GPIO12/2/14/15 are SD-card lines, unused here).
  pinMode(LED_HEALTHY_PIN, OUTPUT);
  pinMode(LED_LOW_PIN, OUTPUT);
  pinMode(LED_MEDIUM_PIN, OUTPUT);
  pinMode(LED_HIGH_PIN, OUTPUT);
  setStatusLed(-1);  // all off until first result

  connectWiFi();

#if BLYNK_ENABLED
  // Only start Blynk if WiFi actually came up. Calling Blynk.connect() without
  // a working network connection crashed (LoadProhibited) in Blynk 1.4.0 on
  // the ESP32 3.x core. With WiFi confirmed up first, the connect path is
  // safe; if WiFi is down, we skip Blynk entirely and the spray loop still
  // runs. loop() retries Blynk once WiFi reconnects.
  if (WiFi.status() == WL_CONNECTED) {

Blynk.config(BLYNK_AUTH_TOKEN);
    Blynk.connect(5000);
  } else {
Serial.println("[blynk] WiFi down, skipping Blynk connect for now");
  }
#endif

  Serial.println("[boot] ready, waiting for trigger");
}

void loop() {
#if BLYNK_ENABLED
  // Only service Blynk when it is actually connected; running it while
  // disconnected can fault on the 3.x core.
  if (Blynk.connected()) {
    Blynk.run();
  }
#endif

  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
  }

  if (triggerRequested()) {
    runOneCycle();
  }

  delay(20);
}
```

Appendix 5: Arduino Nano Firmware (hardware_evidence/firmware/valve_controller/valve_controller.ino)

```
/*
  valve_controller.ino
  ------------------------
-------------------------
-------------------------
  BCE497J - Deep Learning
and IoT-Based Real-Time
Variable-Rate Chemical
  Spraying System for
Disease Control in Paddy
Crop.

  Runs on an Arduino Nano
V3.0 (ATmega328P).
Receives a spray command
from
  the ESP32-CAM over
SoftwareSerial and opens
the relay-driven solenoid
  valve for the requested
duration.

  SERIAL PROTOCOL (matches
esp32cam_sprayer.ino)
  ------------------------
-----------------------
  Real-cycle message, one
line, from the ESP32-CAM:

      S<ms>,<cat>\n

      <ms>  : integer, the
solenoid open time in
MILLISECONDS, already
              computed by
the Flask API (Chapter 4,
section 4.6:
              ts = t_min +
C_eff * (t_max - t_min),
where C_eff already
              includes the
per-disease factor).
Sending the final value
              here,
instead of re-deriving it
from just a 0-3 category
              code, is a
deliberate choice:
recomputing from category
              alone would
drop the disease-specific
scaling that

model/severity.py applies,
and the equations are
already
              implemented
once, correctly, on the
server. 0 means no
              spray
(Healthy).
      <cat> : integer 0-3,
the category code, kept
only so this board can
              log or react
to the category (e.g. for
a serial debug
              printout);
it does not drive the
timing.

  Bench/dry-run message
(matches the hardware
plan, section "Phase 2:
  Pairwise integration",
point 2 and Chapter 5,
section 5.8, step 3):
  typing a single bare
digit '0'..'3' into the
Serial Monitor (over the
  USB hardware serial, NOT
the SoftwareSerial link
from the ESP32) opens
  the relay for a fixed
test duration, so the
relay and solenoid wiring
  can be checked with no
camera, no WiFi and no API
call at all.

  SAFETY CAP
  ----------
  MAX_VALVE_OPEN_MS below
hard-limits how long the
valve can stay open in
  any single cycle,
exactly as recommended in
hardware_implementation_pl
an.md
  section 11 ("Safety cap
on max solenoid open time
(e.g. 5 s) so a bad
  reading can't dump the
whole tank"). Any
requested duration above
this
  cap is clamped down to
it, both for the real
protocol and the bench
  test digits.

  Pin usage (Chapter 5,
Table 5.2):
    D10  SoftwareSerial RX
<- ESP32-CAM GPIO1 (TX)
    D11  SoftwareSerial TX
-> (unused today, reserved
for future two-way

comms, matching the
hardware plan)
    D7   Relay IN (HIGH =
relay energised = solenoid
open)
    D13  Onboard LED, lit
while the valve is open

  Firmware constants to
set once Phase 5 (flow
calibration) is complete:
    QS_MLS_DESIGN below is
the DESIGN value (3.5
mL/s) used throughout the
    report until
Capstone/hardware_evidence
/HARDWARE_TEST_DATA.md
section D
    is filled in with a
measured flow rate. It is
only used for the
    optional serial debug
printout of the expected
dispensed volume; it
    does not change the
valve open time, because
that arrives already
    computed from the
server (see protocol note
above).
*/

#include
<SoftwareSerial.h>

#define RX_PIN
10   // from ESP32-CAM
GPIO1 (TX)
#define TX_PIN
11   // reserved, unused
#define RELAY_PIN
7
#define STATUS_LED_PIN
13

#define
SERIAL_BAUD_TO_ESP32 9600
#define MAX_VALVE_OPEN_MS
5000UL   //
hardware_implementation_pl
an.md section 11

// Design-value constants,
for the optional debug
printout only (see header
// note above). A and d
match model/severity.py
and app/server.py
defaults.
#define APP_RATE_LPH
500.0
#define HILL_DENSITY_M2
25.0
#define QS_MLS_DESIGN
3.5   // [[REPLACE WITH
MEASURED Qs, SECTION D]]

// Bench/dry-run durations
for the legacy single-
digit test mode, roughly
// matching the worked
examples in Chapter 9,
Table 9.1 (so the dry run
// "feels" like the real
system even with no API
call).
const unsigned long
TEST_DURATION_MS[4] = {0,
1250, 1875, 2500};

SoftwareSerial
espLink(RX_PIN, TX_PIN);

String rxBuffer = "";

void openValveFor(unsigned
long ms, int categoryCode)
{
  if (ms >
MAX_VALVE_OPEN_MS) {
```

```
    Serial.print("[safety]
requested ");
    Serial.print(ms);
    Serial.print(" ms
exceeds the ");

Serial.print(MAX_VALVE_OPE
N_MS);
    Serial.println(" ms
cap, clamping");
    ms =
MAX_VALVE_OPEN_MS;
  }

  if (ms == 0) {

Serial.println("[valve]
category Healthy (or
ts=0), no spray this
cycle");
    return;
  }

  float expectedMl = ms /
1000.0 * QS_MLS_DESIGN;
  Serial.print("[valve]
opening for ");
  Serial.print(ms);
  Serial.print(" ms,
category code ");

Serial.print(categoryCode)
;
  Serial.print(", expected
volume at design Qs: ");
  Serial.print(expectedMl,
2);
  Serial.println(" mL
(replace QS_MLS_DESIGN
with the measured value
once available)");

  digitalWrite(RELAY_PIN,
HIGH);

digitalWrite(STATUS_LED_PI
N, HIGH);
  delay(ms);
  digitalWrite(RELAY_PIN,
LOW);

digitalWrite(STATUS_LED_PI
N, LOW);

  Serial.println("[valve]
closed");
}

// Parse one completed
line from the ESP32 link,
e.g. "S1250,2".
void handleEspLine(const
String &line) {
  if (line.length() < 2 ||
line.charAt(0) != 'S') {
    Serial.print("[link]
ignored unrecognised line:
");
    Serial.println(line);
    return;
  }
  int commaIdx =
line.indexOf(',');
  if (commaIdx < 0) {
    Serial.println("[link]
malformed message, missing
comma, ignored");
    return;
  }
  long ms =
line.substring(1,
commaIdx).toInt();
  int cat =
line.substring(commaIdx +
1).toInt();
  if (ms < 0) {
    Serial.println("[link]
negative duration
received, ignored");
    return;
  }
  openValveFor((unsigned
long)ms, cat);
}

// Bench/dry-run: a single
bare digit typed into the
Serial Monitor (USB).
void handleBenchDigit(char
c) {
  if (c < '0' || c > '3')
{
    return;
  }
  int cat = c - '0';
  Serial.print("[bench]
test category ");
  Serial.println(cat);

openValveFor(TEST_DURATION
_MS[cat], cat);
}

void setup() {

Serial.begin(SERIAL_BAUD_T
O_ESP32);     // USB, for
Serial Monitor bench tests
and debug

espLink.begin(SERIAL_BAUD_
TO_ESP32);    //
SoftwareSerial, from the
ESP32-CAM

  pinMode(RELAY_PIN,
OUTPUT);
  pinMode(STATUS_LED_PIN,
OUTPUT);
  digitalWrite(RELAY_PIN,
LOW);           //
solenoid off by default
(NC valve, safe state)

digitalWrite(STATUS_LED_PI
N, LOW);

  Serial.println("[boot]
PaddyGuard valve
controller ready");
  Serial.println("[boot]
bench test: type 0, 1, 2
or 3 in this Serial
Monitor to dry-run the
relay");
}

void loop() {
  // Real protocol,
arriving over the
SoftwareSerial link from
the ESP32-CAM.
  while
(espLink.available()) {
    char c =
espLink.read();
    if (c == '\n') {
      if
(rxBuffer.length() > 0) {

handleEspLine(rxBuffer);
      }
      rxBuffer = "";
    } else if (c != '\r')
{
      rxBuffer += c;
      if
(rxBuffer.length() > 32) {
        rxBuffer = "";  //
guard against a
stuck/noisy line filling
memory
      }
    }
  }

  // Bench/dry-run digits,
typed directly into the
USB Serial Monitor.
  if (Serial.available())
{
    char c =
Serial.read();
    handleBenchDigit(c);
  }
}
```